



Universitat d'Alacant
Universidad de Alicante

APLICACIONES DE
VISIÓN ARTIFICIAL PARA
AYUDA A PERSONAS
CON DIFICULTADES
VISUALES

Samer Refat Hamzeh Alashhab

Tesis **Doctorales**

Unitat de Digitalització UA
Unidad de Digitalización UA

UNIVERSIDAD de ALICANTE



Universitat d'Alacant
Universidad de Alicante

APLICACIONES DE VISIÓN ARTIFICIAL PARA AYUDA A PERSONAS CON DIFICULTADES VISUALES

Universitat d'Alacant
Universidad de Alicante

Samer Refat Hamzeh Alashhab

Tesis doctoral

Alicante, Junio 2022



Universitat d'Alacant
Universidad de Alicante

DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN E
INTELIGENCIA ARTIFICIAL

APLICACIONES DE VISIÓN ARTIFICIAL PARA AYUDA A PERSONAS CON DIFICULTADES VISUALES

Samer Refat Hamzeh Alashhab

DISERTACIÓN PRESENTADA PARA ASPIRAR AL GRADO DE DOCTOR
POR LA UNIVERSIDAD DE ALICANTE (*DOCTOR OF PHILOSOPHY*)
DOCTORADO EN INFORMÁTICA

Universidad de Alicante

Directores

Dr. Antonio Javier Gallego Sánchez
Dr. Miguel Ángel Lozano Ortega

ALICANTE, Junio, 2022

Esta tesis está registrada bajo una [licencia internacional Creative Commons \(CC\) BY-NC-SA](#) (*Creative Commons Attribution NonCommercial-ShareAlike 4.0 International License*). Usted es libre de: (a) **Compartir**: copiar y distribuir el material en cualquier medio y formato; (b) **Adaptar**: modificar, transformar y crear a partir de este material. El licenciador no puede revocar estas libertades mientras cumpla con los términos de la licencia.

- **Reconocimiento**: Debe reconocer adecuadamente la autoría, proporcionar un enlace a la licencia e indicar si se han realizado cambios.
- **No Comercial**: No puede utilizar el material con finalidad comercial.
- **Compartir Igual**: Si modifica, transforma o crea a partir de este material, deberá difundir sus contribuciones bajo la misma licencia que el original.

Universitat d'Alacant
Universidad de Alicante

Documento realizado por
Samer Refat Hamzeh Alashhab
salashhab@ua.es



Al alma de mi madre y de mi hermano, y a mi padre.



Universitat d'Alacant
Universidad de Alicante



*En el nombre de Al-lah, el clemente, el misericordioso.
Pues nosotros elevamos en grados de dignidad a quien nos place; y sobre cualquier poseedor de
conocimiento está el único, el sumo sabedor.*

Universitat d'Alacant
Universidad de Alicante

Sagrado Corán, Yusuf (José) 12:76

AGRADECIMIENTOS

Agradecer solo con palabras a aquellas personas que me han dado todo el apoyo para llegar donde estoy nunca es suficiente. Aun así, como un signo de agradecimiento, dedico esta tesis doctoral:

♡ **A Dios:**

Por la salud, la fuerza, la sabiduría y por todo lo que me ha dado.

♡ **A mis padres:**

Por su motivación, comprensión y apoyo. Por hacerme crecer y por todo el esfuerzo que han puesto, desde el primer momento de mi vida, para hacer de mí lo que soy. Mamá, cuánto siento no haber podido terminar esta tesis antes de que te fueras, hace tan solo unos meses. Deseaba correr a buscarte con el título en las manos para mostrártelo y hacerte sentir feliz y orgullosa de mí. A pesar de la lástima que me genera, sé que tu alma me acompaña y que estará a mi lado el día de la defensa. Te quiero, mamá, estés donde estés. Papá, te quiero también, eres el pilar de todo esto.

♡ **Al alma de mi hermano mayor:**

Por irse antes de tiempo. Querido Nihad, siento que te fueras a tan solo un mes de terminar esta tesis. Has sido nuestro hermano mayor, nuestro maestro, quien nos cuidaba a todos cuando éramos pequeños y quien nos ayudaba en nuestros deberes. Qué gran dolor perderte. Te dedico todo mi trabajo. Descansa en paz, hermano.

♡ **A mi familia:**

Por rodearme siempre con tanto amor y cariño. Sois mi referencia.

♡ **A esas personas más especiales y cercanas:** Por estar a mi lado y darme todo el apoyo y esperanza para seguir hacia adelante. Siempre seréis alguien especial en mi vida.

♡ **A todas aquellas personas** que me han aportado algo durante esta tesis o los trabajos publicados.

♡ **A la Universidad de Alicante:**

Por abrirme sus puertas y sus aulas para poder aprender todo aquello que quería y mucho más. Un día fuiste aeropuerto militar y ahora nos permites aterrizar en tu corazón a través de tu maravillosa gente, de los libros y del conocimiento. Gracias por enseñarnos a ser militantes de la paz y la educación y por permitirnos volver a despegar más fuertes que nunca. Eres mi segunda casa.

♡ **A mi Alicante:**

Por acogerme y tratarme como un alicantino más. Eres la maravillosa ciudad donde he tenido la suerte de vivir más que una década de mi vida junto a tu fabulosa gente. Eres una parte muy importante de mi vida.

♡ **A Pablo Gil:**

Por todo lo que me ha enseñado y por haberme ayudado a conseguir trabajo, esta motivación económica ha sido fundamental para poder seguir estudiando. Muchas gracias, Pablo.

♡ **A mis grandes profesores Antonio Javier Gallego y Miguel Ángel Lozano:**

Quería a acabar el agradecimiento con vosotros, porque sin vosotros no hubiese podido empezar, ni seguir, ni terminar. Me siento sin palabras y desarmado por no saber cómo agradecer todo lo que me habéis dado, todo lo que me habéis enseñado y todo lo que habéis hecho por mí. Todo se va y se pierde en un solo momento, pero el conocimiento permanece, y vosotros me habéis enseñado mucho. Sois un gran ejemplo a seguir como profesores y como personas. Muchísimas gracias, siempre.

Alicante, 25 de mayo de 2022
Samer Alashhab

RESUMEN

La discapacidad visual es considerada la mayor discapacidad sensorial, la cual determina en gran medida la vida de una persona tanto en la interacción con su entorno como con la sociedad. La Organización Mundial de la Salud (OMS) informa de que al menos 2.200 millones de personas en todo el mundo padecen deficiencia visual o ceguera, lo cual convierte en una cuestión prioritaria la investigación en soluciones que sirvan de ayuda para que estas personas superen los retos con los que se enfrentan en su día a día.

Existen ayudas para las personas ciegas o con dificultades visuales, como los perros guía, los bastones, el sistema braille, etc. Sin embargo, aún así, hay multitud de tareas que les resultan difíciles o incluso imposibles, como por ejemplo orientarse en entornos desconocidos o leer textos no adaptados, entre otros. Para estas tareas también podemos encontrar herramientas basadas en la tecnología, como por ejemplo aplicaciones de teléfonos móviles de ayuda a la lectura o a la orientación, aunque habitualmente estas aplicaciones resultan muy básicas y están desarrolladas para un solo propósito, por ejemplo, ampliar un texto, detectar un color, etc. Los recientes avances producidos en el campo de la Inteligencia Artificial y, más concretamente, en el *Deep Learning*, han creado nuevas posibilidades para el desarrollo de aplicaciones avanzadas de ayuda en tareas en las que antes no era posible. Estos avances se han unido al incremento en la capacidad hardware de los teléfonos móviles, los cuales han pasado de ser simples medios de comunicación a tener una potencia de cálculo casi equivalente a la de un ordenador. Todo esto ha producido que estos dispositivos constituyan en la actualidad una herramienta muy importante para la ayuda a este colectivo.

La cuestión central que se aborda en esta tesis doctoral es la investigación en métodos de visión artificial que permitan el reconocimiento de gestos realizados con las manos y, en función del gesto, facilitar diferentes tipos de información. El objetivo es el desarrollo de un sistema eficiente y de bajo coste destinado a dispositivos móviles que permita interactuar mediante gestos con el dispositivo y sea capaz de realizar múltiples acciones de ayuda a personas con discapacidad visual. Se busca con ello un método de interacción humano-máquina que resulte natural, rápido, intuitivo y accesible, y que integre

diferentes acciones sin necesidad de utilizar la pantalla táctil para activarlas o cambiar de aplicación.

Para ello se define un conjunto de gestos con los que interactuar con la aplicación, cada uno de los cuales desencadena una acción diferente, por ejemplo apuntar a un objeto para obtener una descripción del mismo, apuntar con dos dedos para centrar y arrastrar a la posición señalada, hacer la forma de una lupa con la mano para obtener la descripción de la escena, o hacer un gesto de tipo “Pellizco” para hacer *zoom*. En este último caso tendríamos un gesto dinámico, ya que abriendo y cerrando los dedos podremos ajustar el nivel de *zoom*, mientras que los otros casos son ejemplos de gestos estáticos.

Para gestionar todas estas acciones se propone un método eficiente que realiza de forma conjunta tanto la clasificación y la localización de gestos como las acciones asociadas a cada gesto. Este método se basa en una red *multi-head* compuesta por un *backbone* común al que se conectan diferentes cabezas (*heads*) para realizar las tareas especializadas (como por ejemplo la descripción del objeto o la escena, el control de los niveles de *zoom*, etc.). Las cabezas aprovechan las características comunes extraídas por el *backbone* y además solo se activan si se detecta su correspondiente gesto, lo que resulta en una arquitectura altamente eficiente. Además, para la tarea de la localización de los gestos se propone un nuevo enfoque débilmente supervisado que permite transformar una red de clasificación en un método para la localización de objetos, resultando también en una solución eficiente al no requerir una nueva arquitectura.

Para evaluar la metodología propuesta se han creado tres conjuntos de datos diferentes con un total de unas 44 mil imágenes, incluyendo imágenes reales y sintéticas de gestos, y un conjunto de datos que contiene descripciones de las escenas. Todas estas imágenes se han etiquetado a varios niveles, indicando la categoría de toda la imagen, la posición de la mano y de la punta del dedo índice, y la posición y descripción de los objetos señalados.

Para cada uno de los pasos del método propuesto se ha realizado un conjunto de experimentos, tanto para ajustar sus parámetros como para compararlo con alternativas del estado del arte, incluyendo redes neuronales convolucionales, redes de detección de objetos, redes de segmentación, así como la evaluación de diferentes tamaños de entrada, técnicas de inicialización y de aumentado de datos. La experimentación realizada muestra buenos resultados tanto a nivel de precisión como de eficiencia del método. Al comparar los resultados de cada una de las cabezas especializadas con otros enfoques del estado del arte, incluyendo opciones específicas para esas mismas tareas, los mejores resultados (o casi los mejores) se obtienen en todos los casos mediante la arquitectura propuesta. Además, este método también ha mostrado un buen desempeño en los dispositivos móviles actuales reportando tiempos de procesamiento promedio de entre 3-4 FPS en pruebas realizadas en dispositivos Samsung A51 y Huawei P30 lite.

ÍNDICE GENERAL

Agradecimientos	IX
Resumen	XII
Índice de figuras	XVIII
Índice de tablas	XIX
1 Introducción	1
1.1 Motivación	2
1.2 Propuesta y objetivos	4
1.3 Contribuciones	5
1.4 Publicaciones	6
1.5 Estructura de la tesis	7
2 Fundamentos del aprendizaje automático	9
2.1 Inteligencia Artificial	10
2.2 Machine Learning	12
2.2.1 Tipos de aprendizaje	13
2.2.2 Redes neuronales	15
2.3 Deep Learning	19
2.3.1 Redes Neuronales Convolucionales	21
2.3.2 Redes Neuronales Recurrentes	22
2.3.3 Técnicas de regularización	24
3 Estado del arte	29

3.1	Reconocimiento de imágenes y objetos	30
3.1.1	Métodos de clasificación	31
3.1.2	Detección de objetos	31
3.2	Reconocimiento de gestos	36
3.2.1	Métodos basados en hardware	36
3.2.2	Métodos basados en visión artificial	36
3.3	Image captioning	42
4	Clasificación y localización de gestos	44
4.1	Introducción	45
4.2	Método	46
4.2.1	Detalles sobre el entrenamiento	48
4.3	Datasets	49
4.3.1	ImageNet	49
4.3.2	Conjunto de datos de gestos	49
4.4	Experimentación	51
4.4.1	Métricas de evaluación	51
4.4.2	Resultados de clasificación	53
4.4.3	Evaluación de la eficiencia	54
4.4.4	Evaluación de la posición	55
4.5	Conclusiones	57
5	Selección de filtros para la detección de objetos	59
5.1	Introducción	60
5.2	Método	61
5.2.1	Esquema del método propuesto	62
5.2.2	Paso 1: Entrenar la CNN	63
5.2.3	Paso 2: Selección de filtros	64
5.2.4	Paso 3: etapa de inferencia	66
5.3	Datasets	66
5.3.1	MASATI v1	67
5.3.2	MASATI v2	67
5.4	Experimentos	69
5.4.1	Evaluación de la clasificación	69
5.4.2	Resultados de la selección de filtros	70
5.4.3	Comparación con otros métodos	75
5.5	Conclusiones	80
6	Sistema para el procesamiento eficiente de gestos	82
6.1	Introducción	83
6.2	Interfaz de la aplicación	84
6.3	Datasets	86
6.3.1	Dataset de imagen real	86
6.3.2	Dataset de imagen sintética	88

6.3.3	Flickr8k ampliado	89
6.4	Método	90
6.4.1	Backbone y clasificación	92
6.4.2	Gestos de señalar y arrastrar	93
6.4.3	Gesto de lupa	95
6.4.4	Gesto de pellizco	96
6.4.5	Etapas de entrenamiento	96
6.5	Experimentación	98
6.5.1	Evaluación de la clasificación de gestos	98
6.5.2	Evaluación de gestos de señalar y arrastrar	104
6.5.3	Evaluación del gesto de lupa	109
6.5.4	Evaluación del gesto de pellizco	110
6.6	Conclusiones	111
7	Conclusiones	113
7.1	Conclusiones generales	114
7.2	Trabajo futuro	117
	Apéndices	137
A	Librerías	137
B	Herramientas	139
C	Plataformas	141
D	Arquitecturas CNN	144
	Lista de acrónimos	147

ÍNDICE DE FIGURAS

2.1	Principales hitos en la historia de la IA.	11
2.2	Relación entre IA, ML y DL.	12
2.3	Clasificación de los diferentes tipos de métodos utilizados en ML.	14
2.4	Estructura de una neurona del cerebro humano.	15
2.5	Representación de una neurona artificial	16
2.6	Ejemplo de una ANN.	17
2.7	Arquitectura ANN vs. DNN	20
2.8	Arquitectura CNN	21
2.9	Ilustración de la arquitectura de una neurona recurrente	23
2.10	Esquema de una LSTM	24
2.11	Ejemplos de <i>word embeddings</i>	25
2.12	Ejemplo de <i>overfitting</i> y <i>underfitting</i>	25
2.13	Esquema del proceso de <i>dropout</i>	27
3.1	Diferencia entre clasificación, localización y detección de objetos.	30
3.2	Detección de gestos mediante guantes con sensores.	37
3.3	El proyecto Soli utiliza un radar para detectar gestos.	37
3.4	Cámara ToF integrada y mapa de profundidad obtenido.	38
3.5	Imágenes capturadas mediante cámaras termográficas y cámaras de visión nocturna.	38
3.6	Datos obtenidos mediante la cámara <i>Intel RealSense</i>	40
3.7	Reconocimiento basado en colores mediante guantes con marcadores.	41
4.1	Descripción gráfica de los gestos propuestos.	45
4.2	Esquema gráfico de la metodología propuesta.	46
4.3	Ejemplos de la jerarquía de ImageNet.	50

4.4	Imágenes de ejemplo de nuestro conjunto de datos.	51
4.5	Análisis MOP de F_1 vs. eficiencia.	55
4.6	Mapas de calor calculados usando Grad-CAM.	56
5.1	Ejemplos de <i>saliency maps</i> obtenidos con Grad-CAM y BP.	62
5.2	Ejemplo de activaciones obtenidas.	63
5.3	Esquema del método propuesto.	64
5.4	Ejemplos de imágenes del conjunto de datos MASATI v2	68
5.5	Resultados de localización al variar el tamaño del conjunto de entrena- miento y la capa seleccionada.	72
5.6	Resultados de localización e histogramas del IoU obtenido.	73
5.7	Resultados al variar el umbral β y el tamaño de s	74
5.8	Ejemplo del proceso realizado para calcular la localización de un barco.	75
5.9	Ejemplos de los resultados obtenidos por los diferentes métodos.	81
6.1	<i>Google Cardboard</i> y esquema de la interfaz de usuario propuesta.	84
6.2	Gestos propuestos para interactuar con la interfaz.	85
6.3	Ejemplos del conjunto de datos con imágenes reales.	87
6.4	Ejemplos de imágenes del conjunto de datos sintético.	89
6.5	Ejemplos del <i>dataset</i> Flickr8k junto a sus anotaciones.	90
6.6	Esquema de la red <i>multi-head</i> propuesta.	91
6.7	Modelo de <i>captioning</i> usado para la cabeza que procesa el gesto de lupa.	95
6.8	Arquitectura propuesta para procesar el gesto de pellizco.	96
6.9	Ejemplos de las detecciones obtenidas con MediaPipe.	102
6.10	Análisis de eficiencia vs. eficacia como problema de optimización multi- objetivo.	103
6.11	Evaluación de la consistencia temporal para la tarea de clasificación.	104
6.12	Resultados de localización al variar la capa de la red, el número de filtros, el umbral β , y el tamaño de s	106
6.13	Ejemplo del proceso realizado para calcular la ubicación.	108
6.14	Resultados de <i>captioning</i> con el conjunto de datos propuesto.	111
A.1	Resultado obtenido mediante la librería MediaPipe.	138
A.2	Arquitectura de la plataforma Core ML.	139
B.1	Herramienta LibHand para la generación de modelos 3D de manos.	140
B.2	Herramienta LabelImg para la anotación de imágenes.	141
C.1	Proceso de conversión de TensorFlow Lite.	143

ÍNDICE DE TABLAS

4.1	Número de parámetros de cada red en orden descendente.	47
4.2	Descripción de las capas adicionales.	48
4.3	Descripción del conjunto de datos utilizado para la evaluación.	50
4.4	Comparación del F_1 obtenido por las redes CNN.	53
4.5	Comparación de resultados al utilizar aumentado de datos.	54
4.6	Evaluación (en términos de IoU) de la localización obtenida.	56
5.1	Distribución de las clases del conjunto de datos MASATI v2.	69
5.2	Resultados de clasificación obtenidos con las redes consideradas.	70
5.3	Configuración del método de selección de filtros para cada una de las redes.	74
5.4	Comparación de los resultados obtenidos para el conjunto simple.	77
5.5	Comparación de los resultados obtenidos para el conjunto complejo.	78
5.6	Evaluación de las capacidades de generalización de los métodos analizados.	79
6.1	Resumen de los conjuntos de datos utilizados.	88
6.2	Resultados obtenidos para la clasificación del <i>dataset</i> sintético.	99
6.3	Resultados de clasificación del <i>dataset</i> de imagen real.	100
6.4	Resumen de resultados para la tarea de clasificación de gestos.	101
6.5	Comparación de los resultados para la tarea de localización.	107
6.6	Resultados obtenidos para el <i>captioning</i> de imágenes.	110
6.7	Comparación de los resultados obtenidos para el gesto de pellizco.	111

CAPÍTULO 1

INTRODUCCIÓN

Resumen

Esta tesis doctoral presenta el resultado de los estudios de doctorado desarrollados dentro del programa Doctorado en Informática, realizados entre 2016 y 2022 en la Universidad de Alicante.

En este primer capítulo introducimos el tema central de la presente tesis, así como su motivación, la propuesta desarrollada a lo largo del trabajo y los principales objetivos y retos a superar.

Contenidos

1.1	Motivación	2
1.2	Propuesta y objetivos	4
1.3	Contribuciones	5
1.4	Publicaciones	6
1.5	Estructura de la tesis	7

1.1. Motivación

La discapacidad visual es considerada la mayor discapacidad sensorial, la cual determina en gran medida la vida de una persona, tanto en la interacción con su entorno como con la sociedad. De hecho, el 80 % de la información necesaria para nuestra vida diaria proviene del sentido de la vista (ONCE, 2022). Según la Organización Mundial de la Salud (WHO), al menos 2.200 millones de personas en todo el mundo padecen deficiencia visual o ceguera (WHO, 2020). Asimismo, la Organización Nacional de Ciegos Españoles (ONCE) indica que en España, en 2021, había más de 70.462 miembros registrados en su organización, de los cuales el 13,93 % son totalmente ciegos o solo perciben luz, el 4,98 % son personas con visión con imposibilidad de valoración, y el 81,09 % mantienen resto visual funcional para la vida diaria (ONCE, 2022).

Las personas con discapacidad visual se apoyan en los sentidos del oído, el tacto y el olfato para orientarse dentro de su entorno habitual. Además suelen valerse de sistemas de accesibilidad para ayudarse en tareas cotidianas, como por ejemplo la lectura, para la que es común el uso del sistema braille o de aplicaciones de dispositivos móviles, como *Voice Over* o *Voice Assistant*. Sin embargo, les resulta difícil interactuar con entornos no habituales, ya que moverse en lugares cambiantes o desconocidos requiere de habilidades de orientación y capacidades para detectar obstáculos, señales o indicaciones (como por ejemplo letreros, semáforos, etc.). Para ello, estas personas se apoyan tanto en herramientas y ayudas a la movilidad, como son los bastones y los perros guía, como en dispositivos tecnológicos que les facilitan la orientación.

También es común que las personas con discapacidad se ayuden de expresiones y gestos corporales para la comunicación, incluyendo expresiones faciales, el contacto visual, los movimientos y posturas corporales y los gestos, lo que constituye un tipo de lenguaje no verbal cuyo estudio es cada vez más habitual (Pansare et al., 2012; Neto et al., 2019). En particular, los gestos resultan útiles en multitud de contextos, siendo en ocasiones más efectivos que una explicación verbal. Por ejemplo, son muy útiles para reconocer el estado de ánimo de una persona, para saber si está enferma o incluso para averiguar si está diciendo la verdad (Cheng et al., 2020). Además, en la actualidad se están empezando a utilizar para la interacción persona-máquina (Rajesh et al., 2012), para por ejemplo controlar videojuegos, dispositivos móviles (Kaur and Rani, 2016), sistemas robóticos (Van den Bergh et al., 2011), domótica (Desai and Desai, 2017) o interfaces de realidad virtual (Wang and Popović, 2009). Por lo tanto, un sistema de interacción de este tipo podría resultar bastante adecuado para el desarrollo de aplicaciones de ayuda a personas con dificultades visuales.

Para interactuar con la tecnología a través de gestos es necesario desarrollar en primer lugar sistemas especializados capaces de detectar este tipo de expresiones. Este proceso se puede realizar siguiendo estrategias muy diversas, encontramos por ejemplo soluciones basadas en el uso de guantes especiales, sensores o marcadores que puedan ser detectados

fácilmente, o enfoques basados en el empleo de técnicas de visión artificial. Cada una de estas aproximaciones tiene sus ventajas y desventajas, por ejemplo, las basadas en equipamiento especial resultan más costosas y además pueden ser incómodas y aparatosas, aunque a cambio consiguen una alta precisión. Por otro lado, las basadas en visión artificial suelen ser más económicas y menos voluminosas, por lo que también resultan más cómodas, aunque sin embargo requieren de una tecnología que continúa en desarrollo y que no ha sido hasta recientemente, gracias a los avances en *Machine Learning* (ML) y *Deep Learning* (DL), que ha conseguido buenos resultados (LeCun et al., 2015).

En el caso de la visión artificial, para el desarrollo de un dispositivo de este tipo solo es necesario que incluya una cámara y que sea capaz de procesar el algoritmo en cuestión. A este respecto, los teléfonos móviles actuales resultan especialmente adecuados por varios motivos: su reducido tamaño y peso (lo que los hace muy cómodos de utilizar), su alta capacidad de computación (en ocasiones equivalente a la de un ordenador portátil) y su alta disponibilidad, ya que son los dispositivos más comunes (en 2021, el 99,5 % de la población de España tiene al menos un teléfono móvil). Todo esto unido a la facilidad para instalar nuevas aplicaciones hace que actualmente sean comúnmente utilizados para la realización de tareas avanzadas que involucran el uso de visión artificial e incluso de modelos de ML y DL, para por ejemplo realizar reconocimiento facial.

Gracias a todos estos avances, los teléfonos móviles han pasado a ser un elemento clave para el desarrollo de aplicaciones de ayuda a personas con discapacidad visual. Entre estas podemos encontrar ejemplos como los de *Indoor Navigation System* (Rituerto et al., 2016), *Crosswalk Localization* (Ahmetovic et al., 2017) y *The vOICe* (Meijer, 2017), mediante las que se puede usar una brújula para orientarse en interiores, un detector facial y un localizador textos que además los lee. Cabe mencionar también algunos trabajos llevados a cabo por equipos de la Universidad de Alicante, como *Aerial Obstacle Detection* (Sáez et al., 2015), que utiliza *smartphones* dotados de una cámara 3D para detectar la distancia y altura de obstáculos para alertar al usuario, o *SuperVision for Cardboard*¹, donde se combina un teléfono inteligente junto con un módulo de *Google Cardboard* para conseguir un dispositivo de gafas electrónicas de bajo coste que permite ampliar la imagen capturada por la cámara. Más recientemente podemos destacar también la aplicación *Navilens*² que facilita la orientación a personas ciegas mediante el uso de etiquetas visuales densas de largo alcance (Sáez et al., 2020).

Siguiendo esta misma línea, la presente tesis investiga en metodologías que puedan servir de ayuda a personas con dificultad visual para realizar sus tareas cotidianas con más facilidad. Para esto nos basaremos en dispositivos comunes, fáciles de utilizar y asequibles, como son los teléfonos móviles, para crear un sistema que pueda ser manejado de una manera muy sencilla e intuitiva a través de gestos.

¹<http://supervisioncardboard.com>

²<https://www.navilens.com>

1.2. Propuesta y objetivos

La cuestión central sobre la que se centra esta tesis doctoral es el desarrollo de sistemas intuitivos para la ayuda a personas con discapacidad visual que les proporcionen información útil sobre el entorno y, de este modo, les faciliten tareas cotidianas de la vida diaria. Para esto se propone la investigación en interfaces persona-máquina que permitan la interacción con un dispositivo móvil de una manera natural basada en el uso de gestos. En concreto, se pretende desarrollar sistemas eficientes basados en visión artificial que puedan ser ejecutados en un teléfono móvil y sean capaces de detectar los gestos realizados y de llevar a cabo una serie de acciones en función de estos. Por lo tanto, además del reconocimiento de los gestos, el método ha de ser capaz de realizar otras operaciones que ofrezcan información adicional dependiendo del tipo de gesto efectuado. De esta forma, según se realice un gesto u otro el sistema realizará una acción diferente. Por ejemplo, el usuario podrá señalar con el dedo índice para reconocer el objeto señalado o hacer un gesto de lupa con la mano para obtener una breve descripción de la escena.

En línea con la motivación que hemos expuesto en el apartado anterior, buscamos que el método desarrollado pueda formar parte de un asistente digital que proporcione a las personas con discapacidad visual una mayor independencia. Es decir, sin la necesidad de usar ningún dispositivo adicional que en algunas ocasiones puede resultar caro, complicado de usar o incluso puede hacerles sentir incómodos por llevar un aparato especial en público ([Golubova et al., 2021](#)) (en nuestro caso solo sería necesario el teléfono móvil del que dispone casi todo el mundo). Del mismo modo, favoreciendo su acceso a los puestos de trabajo y ayudándoles a realizar tareas cotidianas, como buscar un objeto o ampliarlo para verlo mejor.

El requerimiento de poder ejecutar el método propuesto en teléfonos móviles nos enfrenta a una serie de retos. A pesar de que estos dispositivos cuentan actualmente con una gran capacidad de cómputo, la ejecución de las soluciones actuales basadas en ML y DL suelen tener unos altos requisitos hardware, lo que además de crear una ejecución poco fluida (o incluso inviable para estos dispositivos) implica un gran consumo de batería y el sobrecalentamiento del equipo. Otro reto que hay que enfrentar es la gran cantidad de datos etiquetados que requieren los métodos basados en aprendizaje, lo que implica tener que etiquetar manualmente miles de ejemplos, lo cual es una tarea muy costosa, sobre todo para tareas como la detección de la posición del gesto o de los dedos, ya que no solo requiere indicar el tipo de gesto sino también la posición concreta en la imagen del mismo.

Por tanto, los principales objetivos a alcanzar durante el desarrollo de esta tesis han sido:

- Estudiar el desarrollo de métodos eficientes capaces de realizar las distintas ac-

ciones propuestas. Para ello exploraremos modelos ligeros, buscando siempre un compromiso entre precisión y rendimiento del sistema. Si bien podría orientarse hacia la precisión utilizando modelos más complejos con un elevado número de parámetros, estos conducirían a una alta latencia que no haría viable aplicar estos métodos en dispositivos actuales.

- Explorar soluciones que requieran pocos datos etiquetados para su entrenamiento. Esto, además de facilitar el proceso de entrenamiento, también permitirá la generación de modelos más robustos, ya que permitirá recopilar una mayor variabilidad de datos sin necesidad de etiquetarlos.
- Investigar en el desarrollo de metodologías capaces de realizar múltiples tareas de manera eficiente, como podría ser la clasificación y localización de gestos, la descripción de un objeto o de una escena, etc., sin requerir la ejecución de un método específico para cada acción, ya que esto implicaría unos mayores requisitos hardware.
- Validar la metodología propuesta con datos reales que incluyan todas las posibles casuísticas en las que tendría que funcionar correctamente el método, incluyendo por ejemplo ambientes de interior y exterior, diferentes iluminaciones, diferentes tipos de gestos, etc.

En la siguiente sección se resumen las principales contribuciones realizadas a lo largo de este trabajo de investigación.

1.3. Contribuciones

Las principales contribuciones realizadas durante el desarrollo de esta tesis han sido las siguientes:

- Un método eficiente para la clasificación y localización de gestos realizados con las manos a partir de imágenes basado en DL y visión artificial. El objetivo de este método es proporcionar una forma intuitiva y accesible de interactuar con aplicaciones móviles destinadas a ayudar a las personas con discapacidad visual.
- Un método débilmente supervisado para la detección de objetos en imágenes. El método propuesto se entrena inicialmente para la clasificación categórica de los objetos, sin especificar su posición. A partir del resultado obtenido y utilizando un conjunto muy reducido de imágenes etiquetadas con la localización, se realiza un proceso para adaptar el modelo aprendido a un método que detecte la posición de los objetos que inicialmente había aprendido a clasificar. Esta metodología tiene

varias ventajas, ya que además de superar el estado del arte para la detección precisa de objetos pequeños, resulta eficiente al no añadir capas adicionales y facilita el proceso de entrenamiento al ser débilmente supervisado.

- Un sistema orientado al control de dispositivos móviles mediante gestos de la mano destinado a la ayuda a personas con discapacidad visual. Este sistema permite a los usuarios interactuar con el dispositivo con simples gestos estáticos y dinámicos, cada uno de los cuales desencadena una acción diferente, como describir la escena o el objeto señalado, hacer *zoom*, etc. Con el objetivo de desarrollar un sistema eficiente capaz de funcionar en un teléfono móvil se propone una arquitectura neuronal que optimiza los recursos necesarios para realizar las diferentes tareas, las cuales además solo se activan si se detecta su correspondiente gesto.
- Otra de las contribuciones son los conjuntos de datos elaborados y publicados para su uso por la comunidad científica³. Se han recopilado y etiquetado manualmente más de 10.000 imágenes de gestos con diferentes fondos de interiores y exteriores, con variaciones de iluminación y usando para esto diferentes modelos de teléfonos. Estas imágenes se etiquetaron a varios niveles, indicando la categoría, la posición de la mano y de la punta del dedo índice, la posición y clase de los objetos señalados, y la descripción general de la escena.
- Por último destacar que otra contribución ha sido la liberación del código desarrollado en repositorios de acceso abierto⁴. Además de la metodología propuesta, también se ha implementado una herramienta para la generación sintética de manos realistas que permite la generación aleatoria de variaciones de un conjunto de gestos⁵. De esta forma, a partir de la definición de la posición y los grados de libertad permitidos para las articulaciones de cada gesto, el sistema posteriormente es capaz de generar transformaciones aleatorias de este gesto, introduciendo además cambios en el color de la piel y permitiendo variar el fondo de la imagen. La herramienta además etiqueta automáticamente la posición tanto de la mano como de la punta del dedo índice. De esta forma es posible generar grandes conjuntos de datos sintéticos con los que ajustar inicialmente los modelos propuestos.

1.4. Publicaciones

Como resultado de la investigación realizada en esta tesis doctoral se han obtenido las siguientes tres publicaciones:

³<https://www.dlsi.ua.es/~jgallego/datasets/gestures/>

⁴<https://github.com/salashhab/gestos>

⁵Esta herramienta se basa en LibHand y se puede descargar desde: <https://github.com/malozano/libhand/>

Artículos de revista:

- **Autores:** Alashhab, S., Gallego, A.J., Pertusa, A. & Gil, P.
Título: Precise ship location with cnn filter selection from optical aerial images
Revista: IEEE Access
Volumen: 7
Año de publicación: 2019
Páginas: 96567-96582
DOI: 10.1109/ACCESS.2019.2929080
Indicios de Calidad: Indexada en el Journal Citation Reports (JCR) en el cuartil Q1. Factor de impacto 3,745
- **Autores:** Alashhab, S., Gallego, A.J. & Lozano, M.Á.
Título: Efficient Gesture Recognition for the Assistance of Visually Impaired People using Multi-Head Neural Networks
Revista: Engineering Applications of Artificial Intelligence
Año de publicación: *Artículo en revisión*
Indicios de Calidad: Indexada en el Journal Citation Reports (JCR) en el cuartil Q1. Factor de impacto 6,212

Artículos de congreso:

- **Autores:** Alashhab, S., Gallego, A.J. & Lozano, M.Á.
Título: Hand gesture detection with convolutional neural networks
Congreso: International Symposium on Distributed Computing and Artificial Intelligence
Año de publicación: 2018
Páginas: 45-52
Editorial: Springer International Publishing
Indicios de Calidad: Clasificado como clase C según la Spanish Computer-Science Society (SCIE).

1.5. Estructura de la tesis

El resto de contenidos de esta tesis se estructuran en los siguientes capítulos:

- El Capítulo 2 realiza una introducción a los fundamentos del aprendizaje automático relacionados con los conceptos utilizados a lo largo de esta tesis doctoral.
- El Capítulo 3 revisa el estado del arte que ha servido de punto de partida para el

desarrollo de esta tesis.

- Los tres siguientes capítulos investigan el problema central de esta tesis, empezando con el Capítulo 4, en el cual se presenta una primera aproximación para el reconocimiento de gestos mediante visión artificial y el uso de DL y de *heat maps* o mapas de calor.
- El Capítulo 5 propone un método para mejorar la precisión y eficiencia de la técnica usada en el capítulo previo para calcular la localización de los gestos.
- El Capítulo 6 presenta el sistema propuesto en esta tesis para la ayuda a personas con deficiencias visuales, el cual se basa en los métodos desarrollados en los capítulos anteriores y los extiende para crear una sistema completo con el que detectar de manera eficiente un conjunto de gestos y realizar una acción para cada uno de ellos.
- Por último, el Capítulo 7 incluye las conclusiones generales extraídas de esta tesis y las posibles líneas de trabajo futuro.

CAPÍTULO 2

FUNDAMENTOS DEL APRENDIZAJE AUTOMÁTICO

Resumen

Este capítulo introduce los conceptos fundamentales sobre aprendizaje automático utilizados en los diferentes temas abordados de esta tesis doctoral, como la Inteligencia Artificial, incluyendo sus orígenes e historia, el *Machine Learning* y sus principales métodos, y por último las redes neuronales y el *Deep Learning*.

Contenidos

2.1	Inteligencia Artificial	10
2.2	Machine Learning	12
2.3	Deep Learning	19

2.1. Inteligencia Artificial

La Inteligencia Artificial (IA) se refiere a la simulación de la inteligencia humana en máquinas programadas para imitar nuestras capacidades de resolución de problemas y de toma de decisiones. Actualmente se trata de un campo de investigación muy activo en el que se están logrando grandes avances en el desarrollo de nuevas técnicas y aplicaciones. Si volvemos atrás en el tiempo, encontramos que la idea de IA no es nueva, ya que existen ejemplos en la antigua Grecia, en China o en Egipto de la idea de objetos inanimados que cobraban vida o incluso de la construcción de algunos autómatas sencillos. Sin embargo, no fue hasta bastantes siglos después, en el año 1642, que Blaise Pascal construyó a base de ruedas y engranajes la primera máquina compleja destinada a calcular, la cual se considera el antepasado remoto del actual ordenador. Dos siglos más tarde, en concreto en el año 1837, Babbage y Ada Lovelace propusieron el primer diseño de máquina programable.

Si bien sus raíces son largas, la historia de la IA tal y como la conocemos hoy en día lleva menos de un siglo desarrollándose. La Figura 2.1 resume algunos de los eventos más importantes en la historia de la IA del último siglo. El primer trabajo relacionado con la IA actual fue realizado por McCulloch and Pitts (1943), en el cual propusieron un modelo teórico de red neuronal. En 1949, Donald Hebb propuso un método para modificar la fuerza de conexión entre estas neuronas, el cual se conoce actualmente como “aprendizaje hebbiano”. En 1950, Alan Turing, matemático inglés pionero en el *Machine Learning* (ML), publicó el artículo *Computing Machinery and Intelligence*, en el que reflexionaba sobre la IA y sus capacidades para exhibir un comportamiento inteligente equivalente al humano. Es aquí donde propuso la famosa “Prueba de Turing” para verificar esta capacidad.

Cinco años más tarde, Allen Newell y Herbert A. Simon crearon el primer programa de IA, al que llamaron “*Logic Theorist*”, que consiguió probar 38 teoremas matemáticos y además encontró nuevas demostraciones para algunos de ellos. En 1956, tras su uso en la conferencia de Dartmouth por el científico estadounidense John McCarthy, se adoptó la expresión “inteligencia artificial” para referirse desde entonces a este nuevo campo académico. En 1966 Joseph Weizenbaum creó el primer *chatbot*, que recibió el nombre de “ELIZA”, y unos años más tarde se construyó en Japón el primer robot humanoide inteligente, al que se llamó “WABOT-1”.

A partir de 1974 los científicos se enfrentaron a una grave escasez de inversión por parte de los gobiernos en la investigación relacionada con la IA. Como consecuencia se redujeron significativamente los esfuerzos durante este periodo, hasta que en 1980 se recobró el interés con la aparición de los sistemas expertos y sus buenos resultados. Corporaciones y multinacionales adoptaron estos sistemas y se empezó a fabricar ordenadores especializados. Sin embargo, siete años más tarde se produjo un colapso repentino de esta industria especializada que dio lugar al segundo “invierno” en la IA, que duró

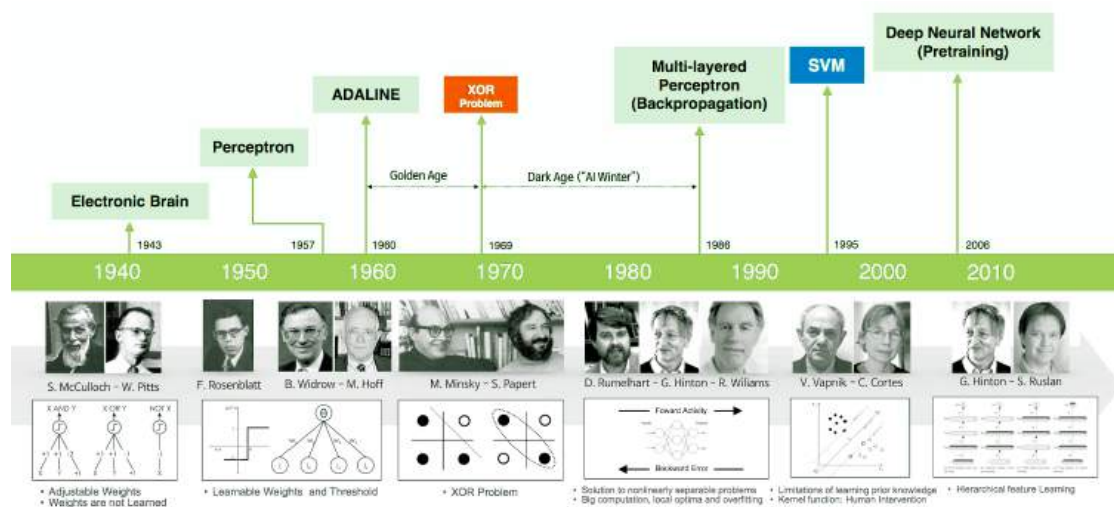


Figura 2.1.: Principales hitos en la historia de la IA. Fuente sefiks.com.

desde 1987 y hasta 1993. Gracias a la evolución del *hardware* y a la aparición de nuevos métodos se volvió a recobrar el interés, teniendo lugar algunos eventos históricos como, por ejemplo, la derrota en 1997 del campeón mundial de ajedrez Gary Kasparov por el ordenador de IBM llamado *Deep Blue*, la introducción de la IA en el hogar en 2002 con el robot aspirador Roomba, o el sistema Watson de IBM que en el año 2011 ganó Jeopardy, un concurso de televisión donde tenía que resolver preguntas complejas y acertijos.

A partir del año 2012 se inicia una nueva era en la IA debido a la aparición del *Deep Learning* (DL), ya que en este año se presentaron varios trabajos que superaron por una notable ventaja al resto de métodos del estado del arte (Krizhevsky et al., 2012). Esto supuso una revolución por la que toda la investigación en IA se ha centrado desde entonces en este tipo de técnicas, incluyendo la visión por computador y el procesamiento del lenguaje natural. Actualmente se han desarrollado nuevos métodos que han llegado a superar el nivel de precisión de expertos humanos en determinadas tareas, como por ejemplo en la clasificación de imágenes, y además se ha habilitado el desarrollo de aplicaciones que permiten comunicarse con un dispositivo en lenguaje natural, como Alexa, Google Home o Siri. Recientemente se están desarrollando modelos, como GPT-3 basado en la arquitectura Transformer, que están muy cerca de superar el test de Turing.

En las siguientes secciones se revisan en detalle los fundamentos de estas áreas de investigación, empezando por el ML y las redes neuronales, y terminando con las nociones relacionadas con el DL.

2.2. Machine Learning

Machine Learning (ML) (aprendizaje automático) es el subconjunto de técnicas de la IA capaces de realizar predicciones sin que sean programados explícitamente para ello, sino que se basan en el aprendizaje a partir de datos (Samuel, 1959; Mitchell et al., 1997). En la Figura 2.2 se puede ver la relación existente entre la IA y ML, el cual a su vez contiene otra gran área de investigación que es el DL (la cual abordaremos en la Sección 2.3).



Figura 2.2.: Relación entre IA, ML y DL.

Dos áreas de la IA en las que el ML juega un papel muy importante son la visión por computador y el procesamiento del lenguaje natural. Muchas de las herramientas que hoy nos facilitan la vida son posibles gracias a estas tecnologías, pudiéndose usar para ayudar, por ejemplo, en la detección de fraude cibernético (Ch et al., 2020), para el reconocimiento facial (Mohammed et al., 2011), en la conducción de vehículos autónomos (Daily et al., 2017) o incluso para sugerir amigos de Facebook (Hazelwood et al., 2018), entre otros. Aunque en la literatura existen numerosos métodos de VC y PLN que no están basados en ML (Koskenniemi, 1984; Guida and Mauri, 1986), actualmente los enfoques más prometedores son aquellos basados en aprendizaje, y más concretamente en *Deep Learning* (DL), como veremos más adelante.

Dependiendo de la características de los datos utilizados durante el entrenamiento, este tipo de modelos matemáticos pueden aprender a realizar diferentes tipo de tareas, como la clasificación o la agrupación de los datos a partir de sus características. A continuación se revisan los diferentes tipos de aprendizaje y los principales métodos de ML.

2.2.1. Tipos de aprendizaje

Los algoritmos de ML tradicionalmente se clasifican según la naturaleza de la **retroalimentación** utilizada durante la etapa de aprendizaje, también llamada proceso de entrenamiento. El principal criterio seguido para esta diferenciación es si usan o no datos etiquetados, aunque también existen otros enfoques que son una combinación de estos. A continuación se resumen los diferentes tipos de aprendizaje:

- **Aprendizaje supervisado:** Los modelos se entrenan con conjuntos de datos etiquetados en los que para cada entrada se especifica al sistema la salida o resultado que debe generar. Estas etiquetas se utilizan durante el proceso de entrenamiento para comparar la salida generada por el modelo con la salida esperada y de esta forma ajustar su configuración para minimizar el error cometido. El principal inconveniente con este tipo de aprendizaje es que el etiquetado es un proceso costoso, tanto temporal como económicamente, ya que en ocasiones tiene que ser realizado por expertos.
- **Aprendizaje no supervisado:** En este caso no se requiere etiquetado, sino que el método aprende a identificar patrones, tendencias o a crear agrupaciones a partir de las características comunes de los datos.
- **Aprendizaje semi-supervisado:** Es una combinación de aprendizaje supervisado y no supervisado, mezclando una pequeña cantidad de datos etiquetados con una gran cantidad de datos no etiquetados para entrenar los modelos.
- **Aprendizaje débilmente supervisado:** Este tipo de aprendizaje se divide a su vez en **supervisión incompleta**, cuando solo está etiquetado un subconjunto de los datos (normalmente de tamaño reducido, sería equivalente al aprendizaje semi-supervisado), **supervisión inexacta**, cuando los datos se etiquetan con una precisión menor a la deseada (por ejemplo, a nivel de imagen en lugar de a nivel de objeto), y **supervisión errónea**, cuando en los datos de entrenamiento hay algunas etiquetas con errores o incorrectamente etiquetadas. Estos tipos de aprendizaje, incluyendo el semi-supervisado, son muy útiles, ya que obtener grandes cantidades de datos es relativamente fácil, sin embargo, etiquetarlas es un proceso costoso, por lo que etiquetar solo una parte o realizar un etiquetado rápido, con errores o inexacto, facilita mucho esta tarea.
- **Aprendizaje por refuerzo:** Son los algoritmos orientados a objetivos o metas, los cuales aprenden a resolver una tarea a partir de penalizaciones y recompensas. Se penaliza cuando falla y se recompensa cuando acierta, de modo que el algoritmo progresivamente irá modificando su predicción para maximizar la recompensa obtenida.

Dentro de cada uno de estos paradigmas de aprendizaje podemos encontrar multitud de propuestas metodológicas. En la Figura 2.3 se resumen algunos de los algoritmos más populares de ML clasificados según diferentes categorías. Por ejemplo, dentro de los basados en aprendizaje no supervisado encontramos los métodos de agrupación o *clustering* como *k*-Means (Likas et al., 2003), usados para detectar asociaciones a partir de características similares. En esta categoría también podríamos incluir los métodos de reducción de dimensionalidad, como *Principal Component Analysis* (PCA) (Risvik, 2007) o *t-Stochastic Neighbor Embedding* (t-SNE) (Van der Maaten and Hinton, 2008).

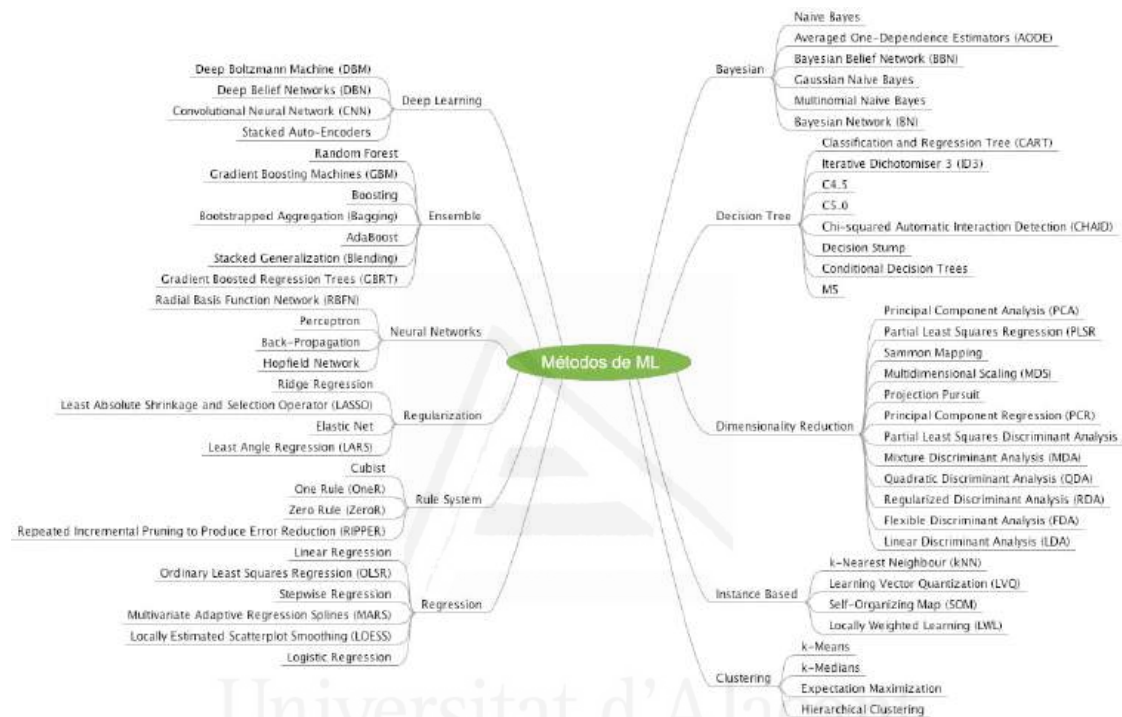


Figura 2.3.: Clasificación de los diferentes tipos de métodos utilizados en ML. Fuente MachineLearningMastery.com.

Dentro de los basados en aprendizaje supervisado tenemos el aprendizaje bayesiano, los árboles de decisión (*decision trees*), los métodos de regresión (como la regresión lineal (Yan and Su, 2009) o la logística (Kleinbaum et al., 2002)), los basados en instancias (como *k*NN) o en la construcción de *ensemble* de clasificadores (como *Random Forest* o *AdaBoost*), así como algunas de las técnicas citadas para la definición de redes neuronales y el Deep Learning, sobre los que hablaremos en detalle en las siguientes secciones.

2.2.2. Redes neuronales

Un tipo de algoritmo de ML particularmente importante para nuestro trabajo son las redes neuronales artificiales (*Artificial Neural Networks* o ANN), las cuales se inspiran en la estructura del cerebro humano, compuesto principalmente de neuronas interconectadas entre sí, y en su funcionamiento mediante la transmisión de señales electroquímicas. La Figura 2.4 representa la estructura de una de estas neuronas. Este tipo de células son estimuladas mediante una serie de impulsos nerviosos que envían señales por medio de las **dendritas**. Estas señales se procesan en el **cuerpo** de la neurona y, posteriormente, se envía una nueva señal a las siguientes neuronas a través del **axón**. Las neuronas se conectan en una región llamada **sinapsis**, en donde una célula (presináptica) se pone en contacto con otra u otras (postsinápticas) mediante la transmisión de estímulos electroquímicos.

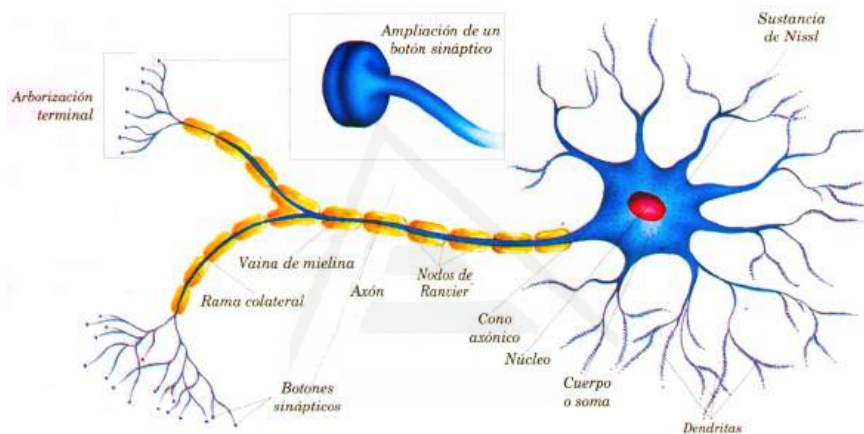


Figura 2.4.: Estructura de una neurona del cerebro humano.

Con el fin de imitar este funcionamiento neurobiológico, en 1943 [McCulloch and Pitts \(1943\)](#) propuso un modelo matemático de red neuronal en términos de un proceso computacional inspirado en esta actividad nerviosa. Posteriormente, [Rosenblatt \(1958\)](#) planteó el primer modelo de neurona artificial, más conocido como *perceptrón*. Y en 1985, [Rumelhart et al.](#) propusieron la primera arquitectura de ANN, originalmente denominada como perceptrón multicapa (*Multilayer Perceptron* o MLP).

La Figura 2.5 muestra un ejemplo de una de estas neuronas artificiales. Como se puede ver, la neurona recibe una serie de entradas x_i y a partir de ellas produce una salida. Para esto multiplica cada valor de entrada por un peso w_i , a continuación suma los resultados de esta ponderación, le añade un bias b y por último procesa este resultado mediante una función de activación f , la cual determina el valor de salida. Formalmente, la operación que realiza cada una de estas neuronas sería $\hat{y} = f(\sum_{i=1}^m w_i x_i + b)$, donde los pesos w_i y el bias b son los parámetros a aprender durante el entrenamiento.

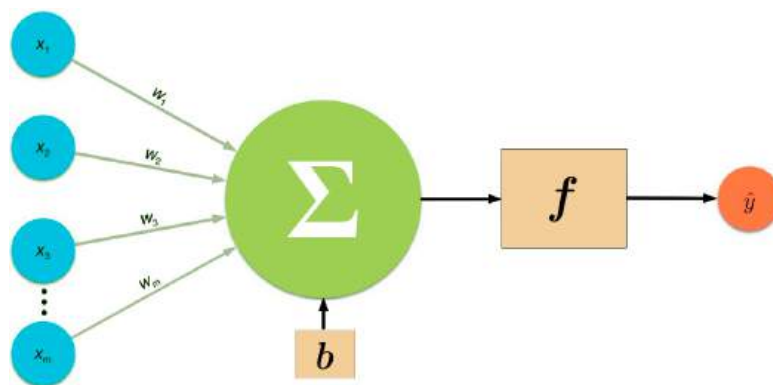


Figura 2.5.: Representación de una neurona artificial, también llamada perceptrón. w_i es el peso por el que se multiplica el valor de cada una de las entradas recibidas, a lo que se añade el bias b y seguidamente se procesa mediante la función de activación f para calcular el resultado final de salida.

Una ANN se compone de un conjunto de estas neuronas artificiales organizadas en capas conectadas entre sí. En la Figura 2.6 se representa esta arquitectura, la cual se compone de tres partes básicas: la capa de entrada, que es la que recibe los datos, una o más capas ocultas, que son las que procesan los datos, y por último la capa de salida, usada para calcular la predicción final de la red. Cada neurona de esta arquitectura recibe la salida de todas las neuronas de la capa anterior y, una vez procesa estos datos, envía su resultado a todas las neuronas de la capa siguiente. Es por esto que las capas que conforman una ANN o un MLP también se denominan “capas totalmente conectas” (del inglés *fully connected layers*).

Siguiendo el esquema de la Figura 2.6, el dato recibido como entrada se procesa capa tras capa en función de los pesos aprendidos durante el entrenamiento. Esto permite ir transformando los datos hasta una representación linealmente relacionada con la predicción a realizar, pudiendo por tanto calcular a partir de esta la predicción devuelta como salida.

Como hemos indicado, además de estos pesos, cada neurona aplica una función de activación para modificar su resultado y, de esta forma, maximizar o minimizar determinados valores. Esto nos permite utilizar diferentes tipos de funciones según el tipo de distribución a aprender, pudiendo seleccionar entre: (i) las de activación lineal, que simplemente devuelven el mismo valor sin aplicar ninguna transformación, (ii) las de paso binario, que umbralizan el resultado y devuelven, por tanto, solo los valores discretos 0 o 1, y (iii) las de activación no lineal, que son las más utilizadas ya que permiten aprender distribuciones de datos complejas. Algunas de las funciones de activación no lineales más comunes son la sigmoide, la logística y la tangente hiperbólica ($\tanh$), las cuales, aunque obtienen buenos resultados, provocan que las predicciones apenas varíen para valores de entrada muy altos o muy bajos y que el proceso de entrenamiento sea costoso.

Actualmente se utiliza más la función de activación *Rectified Linear Unit (ReLU)* (Glorot et al., 2011) y sus variaciones, como *Leaky ReLU* o *Parametric ReLU*, las cuales resultan más eficientes computacionalmente y además evitan problemas con los valores del gradiente durante el entrenamiento (*vanishing and exploding gradients* (Goodfellow et al., 2016)). Además de estas, cuando se trata de problemas de clasificación, se suele emplear la función Softmax para la activación de la capa de salida, en la cual tendremos tantas neuronas como clases posibles. Esta función calcula una distribución de probabilidades normalizada considerando todas las categorías y haciendo destacar la predicción a realizar.

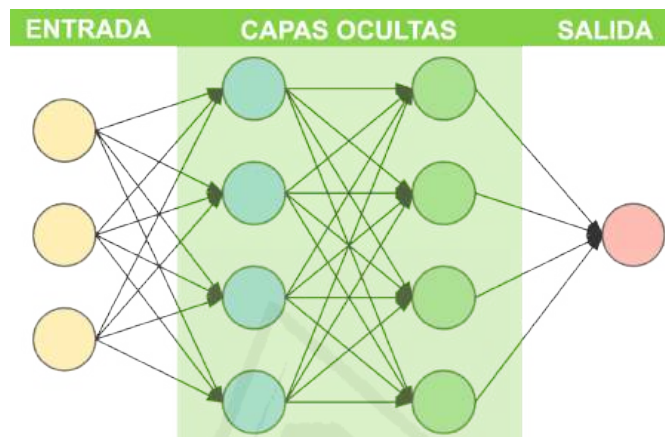


Figura 2.6.: Representación de una ANN con una capa de entrada, 2 capas ocultas y una capa de salida.

2.2.2.1. Proceso de entrenamiento

Para que estas redes neuronales aprendan a clasificar un conjunto de datos determinado se realiza un proceso de entrenamiento en el que, básicamente, se ajustan los parámetros de la red progresivamente para minimizar el error cometido en las predicciones.

Para esto lo primero que necesitamos es el conjunto de datos a utilizar. Este conjunto de datos inicialmente se suele dividir en dos partes, una para entrenar el modelo (con la mayoría de los datos) y otra para evaluarlo. Es importante que la partición de evaluación (también denominada partición de *test*) no se solape con la de entrenamiento y que no se use para ajustar ningún parámetro ni hiperparámetro (aquellos que controlan la configuración de la red y el proceso de aprendizaje) (James et al., 2013). De esta forma, después de entrenar, podremos evaluar las capacidades de generalización del modelo ante datos que no ha visto antes. De manera opcional también podemos crear una partición de validación (a partir de los datos de entrenamiento) que nos permita ajustar los hiperparámetros (Xu and Goodacre, 2018).

Existen diferentes propuestas para realizar este particionado, si bien, cuando contamos con un conjunto de datos grande, al considerarse lo suficientemente representativo del problema, se suele realizar un único conjunto de particiones. Sin embargo, con conjuntos pequeños se pueden considerar otras alternativas, como la validación cruzada (*Cross Validation*) y sus variantes (Kohavi, 1995).

Otra cuestión importante antes de entrenar y evaluar un método es preparar y revisar adecuadamente los datos utilizados. Esto incluye corregir errores en los datos y en su etiquetado, eliminar duplicados, tratar etiquetados incompletos, clases desbalanceadas, etc. Además se suele realizar un proceso de conversión de los datos a formato numérico y de normalización, con el fin de redimensionar los valores para que sigan una escala común que facilite el proceso de entrenamiento (por ejemplo normalizando entre 0 y 1 o aplicando normalización estándar, entre otras alternativas).

Durante el entrenamiento se utiliza el conjunto de datos (en concreto la partición destinada de entrenamiento) para ir ajustando los parámetros de la red (los pesos w , el bias b , así como otros parámetros según el tipo de capa, como veremos más adelante). Para este proceso se suele emplear el método de retropropagación (*backpropagation*), el cual consta de dos pasos:

- Paso hacia adelante (*forward*): Se suministran las muestras de entrenamiento a la red y se calcula la predicción utilizando la configuración de parámetros actual (que inicialmente serán valores aleatorios). Esta salida se compara con el etiquetado del conjunto de entrenamiento y se calcula el error cometido usando una función de pérdida (*loss*), como por ejemplo el error medio al cuadrado o en valor absoluto, entre otras (esto dependerá de la tarea a resolver por la red).
- Paso hacia atrás (*backward*): El error calculado en el paso *forward* se utiliza para ir ajustando los pesos de cada una de las capas de la red empezando desde la capa de salida y propagándolo en sentido inverso. Para este proceso se utiliza el método de optimización de descenso por gradiente, el cual calcula para cada neurona la pendiente formada por el error cometido en función de su configuración de parámetros. De esta forma se determina el ajuste a realizar, modificando los parámetros en la dirección opuesta al gradiente calculado.

Este proceso se repite durante un número determinado de épocas (o iteraciones), pudiéndose utilizar también otros criterios para detener el entrenamiento (como veremos más adelante). En cada iteración los parámetros se modifican solo un factor del error cometido (determinado por el valor del *learning rate* establecido), para de esta forma conseguir un entrenamiento más estable en el que progresivamente se vaya reduciendo el error. Una vez entrenada la red se fijan los parámetros aprendidos y se utilizan durante la etapa de inferencia para realizar predicciones con nuevas muestras.

2.3. Deep Learning

Las redes neuronales han ido evolucionando con el paso de los años debido a la aparición de nuevas técnicas, como mejoras en el proceso de entrenamiento, nuevas funciones de activación, tipos de capas, mecanismos de regularización, etc., las cuales han permitido que progresivamente se hayan ido obteniendo mejores resultados. Esto, unido al gran incremento en las capacidades *hardware* de los ordenadores, tanto de potencia de cálculo como de almacenamiento, y a las ingentes cantidades de datos de las que disponemos actualmente, ha producido una revolución en este tipo de metodologías, llegando a un punto en el actualmente se obtienen resultados comparables (y en algunos casos superiores) a los que obtendría un experto humano.

Históricamente las redes neuronales han presentado problemas cuando tenían muchos parámetros o muchas capas, principalmente debido a que no llegaban a converger durante el proceso de entrenamiento o a que aprendían características muy específicas y en ocasiones no relacionadas con la tarea. A raíz de una serie de trabajos publicados en 2006 y 2007 ([Hinton et al., 2006](#); [Bengio et al., 2006](#); [Ranzato et al., 2007](#)) se demostró que era posible entrenar redes neuronales con muchas capas evitando estos problemas. Este nuevo tipo de arquitecturas profundas pasó a denominarse *Deep Learning* (DL) o aprendizaje profundo y, como se puede ver en el esquema previo de la Figura 2.2, representa un subconjunto de ML, basado en aprendizaje, que a su vez también puede ser supervisado, semi-supervisado y no supervisado.

En concreto, el DL se basa en el concepto de *feature learning* mediante el cual la red aprende a detectar automáticamente las características necesarias para resolver la tarea para la que se ha entrenado. Como se ilustra en la Figura 2.7, esto crea una jerarquía de características según aumenta la profundidad de la red. Por ejemplo, en el procesamiento de imágenes, las capas iniciales se centran en identificar características de bajo nivel, como bordes, gradientes, etc., y progresivamente estas características se van combinando para formar otras de mayor nivel. Si por ejemplo estuviésemos identificando caras, la red aprendería a detectar partes del rostro, como narices, ojos, etc. y posteriormente, en base a estos, a detectar las caras.

Uno de los hitos más relevantes en la historia del DL fue la publicación de la arquitectura AlexNet ([Krizhevsky et al., 2012](#)), con la cual se ganó por una notable diferencia el concurso de clasificación de imágenes ImageNet ([Russakovsky et al., 2015](#)). Hasta ese año los métodos presentados competían por mejoras inferiores al 1 %, esta arquitectura consiguió superarlos en un 10 %. Debido a este hecho, y otros hitos similares, el DL ha centrado desde entonces todos los esfuerzos de investigación en campos tan dispares como la visión artificial ([Russakovsky et al., 2015](#); [He et al., 2016](#); [Sandler et al., 2018](#)), el reconocimiento de voz ([Mustafa et al., 2019](#)), el procesamiento del lenguaje natural ([Xu et al., 2018](#); [Chen et al., 2021](#)), la música ([Condit-Schultz et al., 2018](#)) o la bioinformática y la medicina ([Hannun et al., 2019](#); [Seo et al., 2020](#); [Zhou et al., 2020](#)). Además, muchas

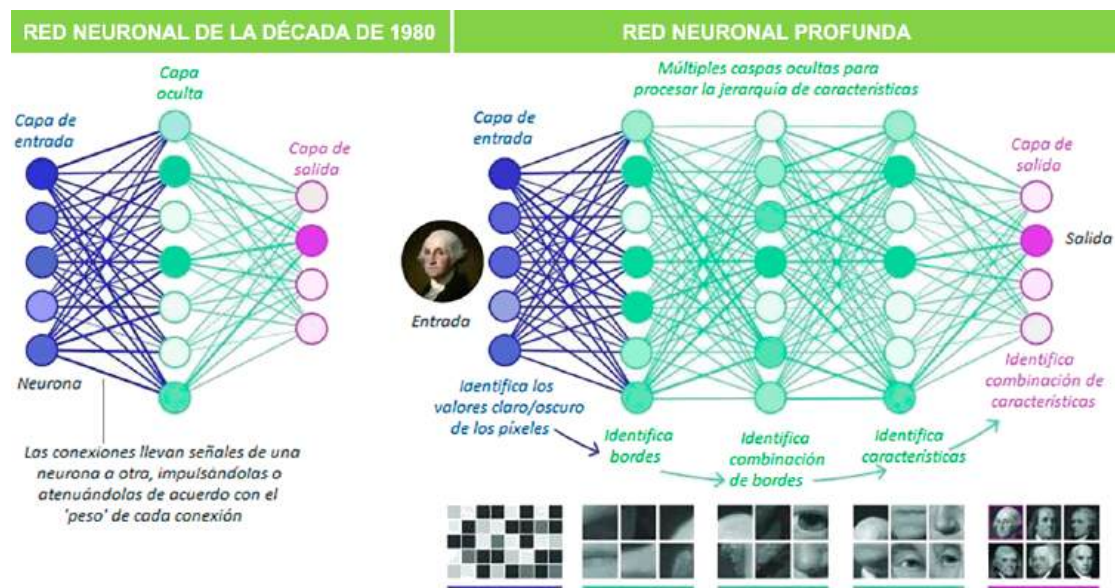


Figura 2.7.: Comparación de la arquitectura de una ANN con la utilizada en las DNN.
Fuente: [Waldrop \(2019\)](#).

multinacionales también han apostado por esta tecnología, como Google, Microsoft, Apple, Netflix o Amazon, las cuales destinan gran parte de sus recursos a la investigación y desarrollo de nuevas técnicas basadas en aprendizaje. Esto también ha provocado que surjan numerosas librerías, herramientas y plataformas (las cuales revisaremos en los Apéndices A, B y C) que facilitan y aceleran el trabajo en desarrollos que involucren este tipo de técnicas.

En DL también se utiliza el método de *backpropagation* que hemos visto previamente para el entrenamiento de los modelos, aunque este se suele combinar con mecanismos de regularización y con el uso de otros tipos de capas en su construcción. Esto nos permite generar diferentes arquitecturas según la tarea que se pretenda resolver y de la naturaleza de los datos que se vayan a tratar. Entre las arquitecturas más utilizadas encontramos, por ejemplo, las redes neuronales profundas (*Deep Neural Networks* o DNN), basadas en los MLP vistos previamente, las Redes Neuronales Convolucionales (*Convolutional Neural Networks* o CNN), aplicadas principalmente al procesamiento de imágenes, y las Redes Neuronales Recurrentes (*Recurrent Neural Networks* o RNN), usadas para tratar con datos secuenciales. En las siguientes secciones revisaremos en detalle algunas de estas técnicas.

2.3.1. Redes Neuronales Convolucionales

Las CNN son un tipo de red neuronal en las que las neuronas utilizadas imitan los campos receptivos de las neuronas biológicas de la corteza visual primaria de nuestro cerebro. De este modo, estas arquitecturas están enfocadas principalmente al procesamiento de imágenes (datos bidimensionales, aunque podemos encontrar variaciones para 1D y 3D), resultando muy efectivas en tareas como la clasificación y la segmentación de imágenes o la detección de objetos (LeCun et al., 2015).

Estas redes reciben su nombre del tipo de operación que utilizan, la convolución discreta, la cual se basa en aplicar un filtro o *kernel* sobre los datos que recibe. Formalmente, esta función se podría definir como $\hat{y} = f(b + \sum_{i=1}^m K \otimes x_i)$, donde K es el *kernel* que se aplica mediante la operación de convolución $\otimes$ sobre las entradas x_i recibidas, y b y f representan, respectivamente, el bias y la función de activación utilizada. Esta operación resulta similar a la que vimos para el perceptrón pero, en este caso, en lugar de aprender los pesos con los que se pondera la entrada, se aprenden los valores del *kernel*. Usualmente este *kernel* tiene un tamaño reducido (por ejemplo, de 3×3 , 5×5 , 7×7 , etc.), por lo que para procesar una imagen completa se tiene que desplazar por sus píxeles¹ aplicando el producto escalar por los valores del filtro. El resultado generado se almacena en una nueva matriz denominada mapa de activación o de características.

La arquitectura de una red CNN, de manera similar a las ANN vistas previamente, se compone de una serie de capas. Sin embargo, en este caso, y como se representa en el esquema de la Figura 2.8, se suelen utilizar diferentes tipos de capas. Inicialmente se utilizan capas de convolución alternadas con capas de *subsampling* (para reducir el tamaño de la entrada), posteriormente se aplica una operación de *flatten* (que “aplana” o transforma en un vector unidimensional los mapas de activación obtenidos) y se conecta con una o más capas tipo *fully connected* para obtener el resultado final.

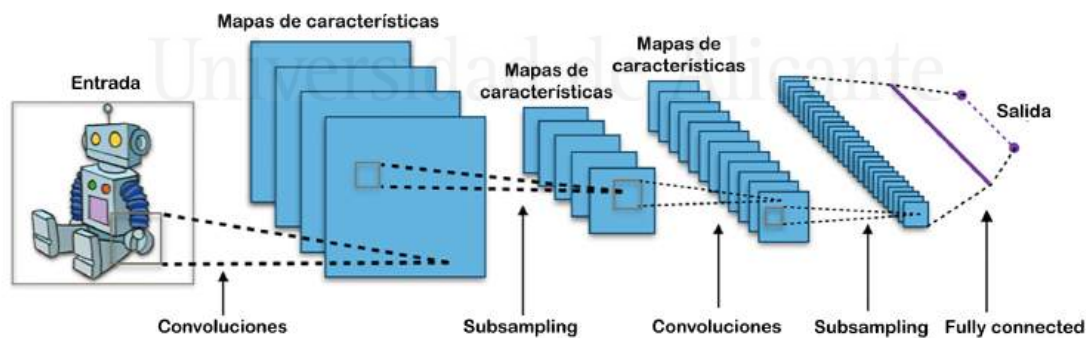


Figura 2.8.: Ejemplo de una arquitectura CNN. Fuente: [Wikipedia Commons \(2015\)](#).

¹El desplazamiento del *kernel* sobre la imagen se puede ajustar para dar pasos o *strides* más o menos grandes.

Las capas de convolución extraen diferentes tipos de características de la imagen en función de la parametrización de los *kernels* aprendida durante el entrenamiento. Como se ha explicado previamente, la red crea una jerarquía de características en sus capas, empezando por descriptores de bajo nivel, como bordes, esquinas, etc., hasta llegar a descriptores de más alto nivel formados a partir de la combinación de los anteriores. Estos últimos dependerán de la tarea objetivo para la que se entrenó la red, por ejemplo, para la clasificación de imágenes se obtendrían descriptores que representarían los objetos relevantes a clasificar. Estas características finalmente obtenidas permiten, por tanto, calcular fácilmente la predicción de la red en sus últimas capas.

La operación de *subsampling* utilizada en estas arquitecturas está destinada a reducir el tamaño de la imagen de entrada pero conservando su información más importante. Gracias a esta operación, los *kernels* aplicados en las capas de convolución pueden progresivamente operar sobre un campo receptivo mayor, es decir, al irse reduciendo la imagen, el mismo tamaño de *kernel* cada vez será capaz de “ver” una porción mayor de la imagen. Podemos encontrar diferentes tipos de *subsampling*, como el *max pooling*, que selecciona el valor máximo de la región, el *average pooling*, que calcula la media, y el *global pooling*, el cual realiza la reducción considerando también los canales (número de filtros aplicados), quedándose con el valor máximo del canal (en el caso de *global max pooling*) o con la media (en el *global average pooling*).

2.3.2. Redes Neuronales Recurrentes

Las redes neuronales que hemos visto hasta ahora no nos permiten trabajar con secuencias de datos, ya que solo consideran la entrada actual para realizar la predicción. Es decir, ante una entrada siempre nos darán la misma predicción, sin considerar el contexto.

Para este tipo de tareas, cuando por ejemplo se requiere trabajar con cadenas de texto o valores secuenciales, resultan más adecuadas las Redes Neuronales Recurrentes (RNN) (Grossberg and Levine, 1975). Este tipo de arquitecturas se construyen mediante *neuronas recurrentes* (ilustradas en la parte izquierda de la Figura 2.9), las cuales son muy similares a la neurona artificial que hemos visto previamente pero incorporando un bucle o conexión consigo misma que le permite mantener un estado. De este modo, la salida ya no depende solamente de la entrada actual sino también del resultado previo (el cual dependía a su vez de los anteriores). En la parte derecha de la Figura 2.9 se ilustra cómo funcionaría una neurona de este tipo al procesar una secuencia de valores.

Sin embargo, debido al problema del *vanishing gradients* o gradiente evanescente ocasionado durante el entrenamiento, este tipo de neuronas solo consiguen aprovechar la información más reciente de la secuencia. Para solucionar esto se han propuesto alternativas como las *Long Short-Term Memory* (LSTM) (Hochreiter and Schmidhuber, 1997),

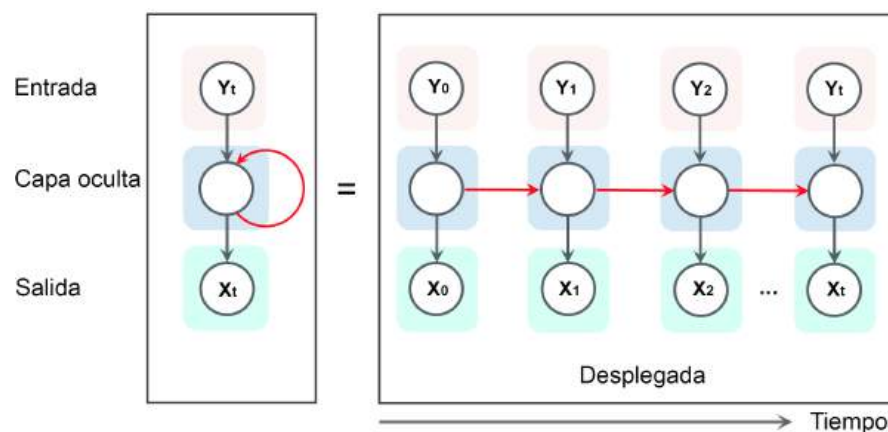


Figura 2.9.: Ilustración de la arquitectura de una neurona recurrente. A la derecha se incluye la forma desplegada de la neurona, en la que se representa cómo se utiliza la salida obtenida previamente para calcular la predicción del elemento actual de la secuencia.

las cuales incorporan una memoria de la cual pueden leer, escribir y borrar información. Para esto la neurona incorpora tres puertas (ver Figura 2.10): puerta de entrada (*input gate*), la cual determina si se permite o no una nueva entrada, puerta de olvido (*forget gate*), para eliminar el estado almacenado, y puerta de salida (*output gate*), para decidir si modificar la salida del instante actual. Otra alternativa similar a estas son las *Gated Recurrent Units* (GRU) (Chung et al., 2014), las cuales son una simplificación de las LSTM en las que solo se utilizan dos puertas, una de reinicio (*reset gate*) y otra de actualización (*update gate*).

Otra diferencia entre las redes recurrentes y las que hemos visto previamente es el formato de las entradas y salidas. Mientras que en las redes convencionales cada entrada solo produce una salida, en las RNN podemos tener arquitecturas de tres tipos: i) uno-a-muchos, es decir, para una entrada se produce una secuencia como salida (veremos alguna aplicación de esto en el siguiente capítulo, como las redes de *captioning*), ii) muchos-a-uno, para a partir de una secuencia devolver un solo valor (sería el caso de la clasificación de sentimientos (Tang et al., 2015) o de voz (Son et al., 2019)), y iii) muchos-a-muchos, que sería la aplicada habitualmente en tareas de procesamiento del lenguaje natural para, por ejemplo, la traducción automática (Vathsala and Holi, 2020).

2.3.2.1. Word embeddings

Los modelos de ML son sumamente hábiles cuando tratan con datos numéricos, pero se convierten en métodos inútiles si les proporcionamos datos en formato texto. Por este motivo se hace necesario codificar esta información numéricamente, para lo cual

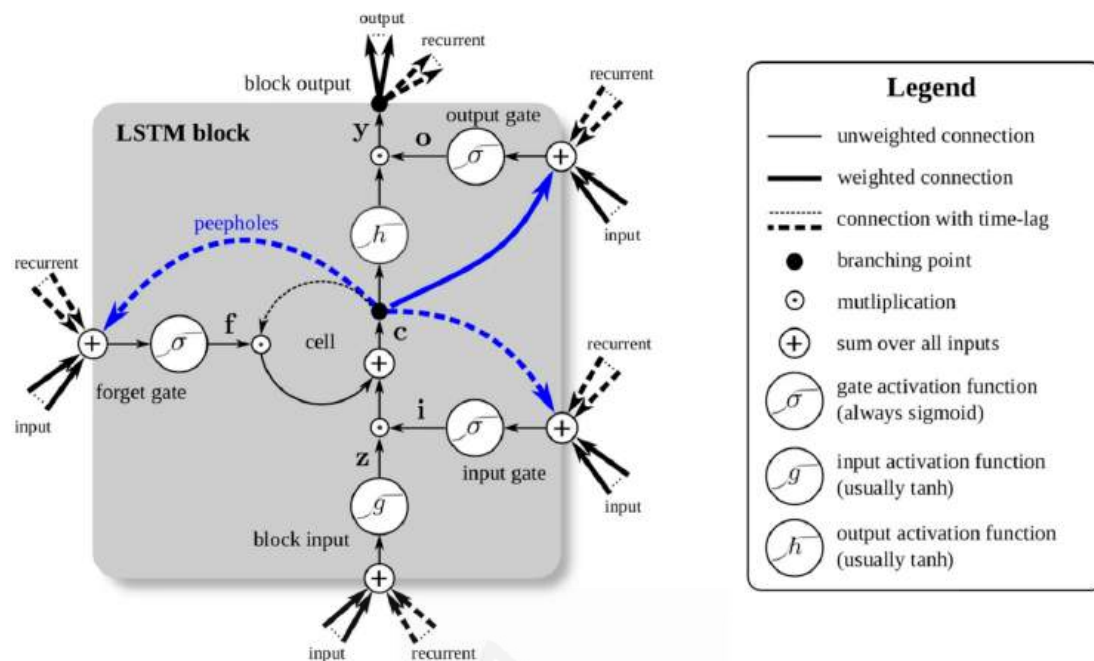


Figura 2.10.: Esquema de una unidad LSTM. La celda de memoria de la estructura LSTM está compuesta de una puerta de entrada, una puerta de salida y una puerta de olvido. Fuente ([Greff et al., 2017](#)).

podríamos seguir varias estrategias. La más sencilla sería asignar identificadores únicos a cada palabra, sin embargo, de este modo el modelo podría aprender relaciones entre los valores, lo cual podría ser erróneo, ya que estos identificadores no consideran el significado de las palabras. Otra estrategia históricamente utilizada es codificar las palabras usando *one-hot*, de este modo la distancia con otras palabras siempre valdrá 1 (y 0 consigo misma). Pero de nuevo esta estrategia pierde el significado de las palabras.

Para esto se utilizan precisamente los *word embeddings* (ver Figura 2.11), que son vectores numéricos que sí que preservan el significado semántico de las palabras (por ejemplo, en esta representación las palabras “hombre” y “mujer” aparecerían cerca, pero alejadas de otras que no estén relacionadas, como por ejemplo con “casa”). Para obtener estos *embeddings* se emplean redes neuronales que se entrenan para ajustar estos valores, siendo Word2Vec una de las más populares ([Mikolov et al., 2013](#)).

2.3.3. Técnicas de regularización

Un problema inherente al proceso de entrenamiento de los algoritmos de aprendizaje automático es el *overfitting* o sobreajuste. Este problema ocurre cuando los parámetros

Tesis	→	1.2	-0.1	4.3	3.2
Doctorado	→	0.4	2.5	-0.9	0.5
Alicante	→	2.1	0.3	0.1	0.4
...					...

Figura 2.11.: Ejemplo de *word embeddings* para las palabras “Tesis, Doctorado y Alicante” en el que cada palabra se representa mediante un vector de valores decimales con 4 dimensiones.

aprendidos se especializan en la detección de características muy específicas del conjunto de datos usado para el entrenamiento, en lugar de basarse en patrones generales que puedan aplicarse también a otras muestras no vistas durante este proceso. En la Figura 2.12 se ejemplifica este problema, representando también el caso contrario de “sobregeneralización” (*underfitting*).



Figura 2.12.: Ejemplo de sobreajuste (*overfitting*), sobregeneralización (*underfitting*) y del ajuste óptimo deseado durante el proceso de entrenamiento.

El *overfitting* es un problema especialmente frecuente al entrenar modelos de Deep Learning debido a la gran cantidad de datos utilizados y a la alta capacidad de aprendizaje de estos modelos. Para solucionarlo podemos aplicar diferentes técnicas, siendo la más sencilla el uso de un conjunto de datos de entrenamiento más variado, con un mayor número de muestras que sean representativas de todos los posibles casos a considerar. Otra estrategia sencilla es ajustar la arquitectura de red utilizada, reduciendo, por ejemplo, el número de capas o de neuronas por capa, para de esta forma reducir los parámetros a aprender y forzar a que el modelo tenga que generalizar. Además de estas, existen muchas otras formas de regularización en el campo del aprendizaje profundo (también aplicables al aprendizaje automático en general), a continuación revisamos algunas de las más populares, las cuales utilizaremos también durante el desarrollo de esta tesis.

2.3.3.1. Aumentado de datos

Una forma de abordar el problema de la falta de datos es mediante el uso de métodos de aumentado de datos, el cual consiste en generar artificialmente variaciones en los datos utilizados durante el proceso de entrenamiento sin cambiar el significado semántico de las clases/etiquetas (Krizhevsky et al., 2012; Chatfield et al., 2014a). Por ejemplo, en el caso de CNN podemos aplicar transformaciones lineales y no lineales sobre las imágenes de entrenamiento para crear más imágenes reales, sintéticas o una mezcla de estas (Rozantsev et al., 2015). Las transformaciones típicas incluyen traslación, rotación, escalado, reflexión (*flips*), deformación (como el desenfocado), cambios de iluminación o color, e inyección de ruido, entre otros.

2.3.3.2. Aprendizaje por transferencia

Al igual que el aumentado de datos, el aprendizaje por transferencia (*Transfer Learning*) (Weiss et al., 2016) es otro paradigma para evitar el sobreajuste. En este caso se basa en aprovechar el modelo aprendido para otra tarea, con otro conjunto de datos, como podría ser ImageNet (Russakovsky et al., 2015), para utilizar los pesos aprendidos como inicialización a la hora de aprender una nueva tarea. Al entrenar un modelo desde cero los parámetros se inicializan de manera aleatoria o siguiendo algún tipo de distribución, por lo que partir de los pesos aprendidos para otra tarea suele ser un buen punto de partida que ayuda tanto a reducir el tiempo de entrenamiento como a mejorar los resultados.

Para entrenar para la nueva tarea es habitual tener que modificar las últimas capas de la arquitectura de la que se parte para adaptarlas a la nueva tarea. Siguiendo con el ejemplo anterior, si la red fue entrenada para ImageNet para clasificar más de mil clases, es necesario modificar esta última capa para configurar el número de clases (o la tarea en concreto a aprender) de nuestro conjunto de datos.

Una vez inicializada y adaptada la red se realiza un proceso denominado como *fine-tune* (ajuste fino) en el que se entrena la red para ajustarla a la nueva tarea. Para esto se pueden seguir varias estrategias, como ajustar solo las últimas capas de la red, entrenar todo el modelo, o hacer una combinación de estas técnicas.

2.3.3.3. Pre-entrenamiento

Esta es una estrategia muy similar a la anterior, ya que también persigue partir de una buena inicialización de los pesos de la red, pero en este caso se consigue realizando un

entrenamiento previo mediante otro conjunto de datos. Con este fin a menudo se preparan bases de datos sintéticas que imitan a la base de datos real para la que se pretende generar el modelo. Este proceso, de manera similar al *transfer learning*, ha demostrado ayudar notablemente a acelerar el entrenamiento y a obtener mejores resultados.

2.3.3.4. Dropout

Se trata de un método de regularización muy sencillo y eficaz para reducir el sobreajuste y mejorar la generalización de los modelos (Srivastava et al., 2014). Esta técnica se basa en eliminar o ignorar algunas unidades de la red de forma aleatoria durante cada época de entrenamiento. En concreto, se descarta una unidad junto con sus conexiones con una probabilidad p (normalmente $0,2 \leq p \leq 0,5$) (Srivastava et al., 2014). La idea es prevenir la coadaptación, en la que la red se vuelve demasiado dependiente de conexiones particulares, forzando de esta manera a que tenga que generalizar y no pueda aprender características específicas. Esta técnica se puede utilizar con la mayoría de los tipos de capas, así como en cualquiera (o en todas) de las capas de la red. La Figura 2.13 ilustra un ejemplo de *dropout* aplicado a una ANN.

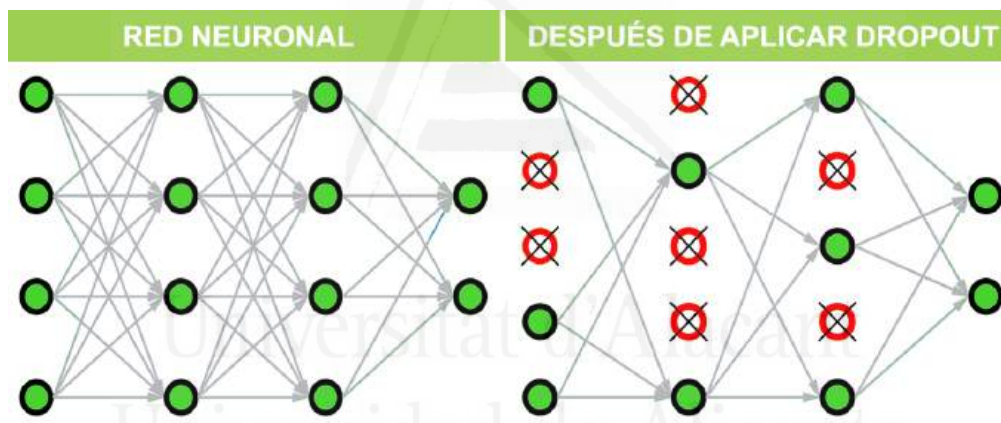


Figura 2.13.: Esquema del proceso de *dropout* aplicado a un modelo de red. En esta figura se puede ver una comparación de una arquitectura ANN estándar con 2 capas ocultas y el efecto de aplicar sobre esta la técnica de regularización conocida como *dropout*.

2.3.3.5. Parada anticipada

La parada anticipada (o *Early Stopping*) consiste en detener el proceso de entrenamiento cuando las actualizaciones de parámetros ya no producen mejoras en el conjunto de validación. Está demostrado que detener el entrenamiento antes de que se adapte al

conjunto de datos utilizado puede reducir el sobreajuste y mejorar la generalización ([Prechelt, 1998](#)).



Universitat d'Alacant
Universidad de Alicante

CAPÍTULO 3

ESTADO DEL ARTE

Resumen

Este capítulo revisa el estado del arte relacionado con los principales temas en los que se basa esta tesis doctoral. Dado que buscamos desarrollar un método basado en visión por computador para el reconocimiento de gestos, que además nos ofrezca información sobre la escena y sobre los objetos que aparecen en ella, revisaremos en primer lugar el estado del arte relacionado con el reconocimiento de imágenes y objetos de carácter general. Posteriormente, estudiaremos los métodos específicos para el reconocimiento de gestos, y finalizaremos con los métodos para obtener la descripción textual de una imagen (métodos de *captioning*).

Contenidos

3.1	Reconocimiento de imágenes y objetos	30
3.2	Reconocimiento de gestos	36
3.3	Image captioning	42

3.1. Reconocimiento de imágenes y objetos

Gracias a los recientes avances en los modelos de visión por computador basados en aprendizaje profundo, actualmente podemos encontrar numerosos métodos y recursos para desarrollar sistemas de clasificación de imágenes y de detección de objetos. Uno de los principales recursos con los que contamos son grandes conjuntos de imágenes con los que entrenar los modelos. Además, las mejoras de rendimiento en las plataformas de ejecución y en los modelos permiten que muchos de estos puedan funcionar en tiempo real, o casi sin retraso, incluso en dispositivos móviles.

Los escenarios de clasificación de imágenes, localización y detección de objetos son diferentes pero pueden llegar a confundirse. Con **clasificación de imagen** nos referiremos a asignar una determinada categoría a una imagen completa. Además de la clasificación, hablaremos de **localización** cuando seamos capaces de determinar la región de la imagen en la que se encuentra el objeto que hace que identifiquemos la imagen con esa determinada categoría, pero considerando siempre un único objeto para cada imagen. Por otro lado, hablaremos de **detección de objetos** para referirnos a métodos que al mismo tiempo localicen e identifiquen diferentes objetos dentro de una imagen, delimitando la región en la que se encuentran e identificando su categoría. La Figura 3.1 muestra mediante un ejemplo la diferencia entre la clasificación de imágenes, la localización y la detección de objetos.



Figura 3.1.: Diferencia entre clasificación, localización y detección de objetos. La imagen izquierda muestra un ejemplo de clasificación de una mano, la imagen central ilustra la localización de la mano usando un cuadro delimitador y la tercera imagen muestra la detección de varios objetos (una mano y una señal de tráfico) usando cuadros delimitadores etiquetados para cada objeto.

A continuación, debido a su extenso uso en esta tesis, procedemos a revisar el estado del arte de los métodos de clasificación de imágenes y de detección de objetos.

3.1.1. Métodos de clasificación

Entre los métodos tradicionales para la clasificación de imágenes, gran parte de los enfoques pertenecen al subgrupo de métodos icónicos. Estos métodos tienen en cuenta las propiedades de la imagen de forma global, como es el caso de los modelos basados en histograma (Barla et al., 2003) o en *Principal Component Analysis* (PCA) (Al-Azzawi and Abdullah, 2011; Risvik, 2007). En general, los métodos clásicos se basaban en la extracción de características o descriptores de las imágenes seleccionadas manualmente (a lo que actualmente se conoce como *handcrafted features*), para posteriormente realizar la clasificación mediante métodos como, por ejemplo, SVM (*Support Vector Machines*), kNN (*k-Nearest Neighbors*), árboles de decisión (Rajendran and Madheswaran, 2010), o incluso mediante el uso de sistemas expertos (Zarandi et al., 2011).

Como se indicó en el capítulo anterior, la clasificación de imágenes es uno de los campos en los que las técnicas de DL han demostrado ser especialmente efectivas, por lo que en los últimos años la investigación se ha centrado principalmente en este tipo de técnicas. En el estado del arte podemos encontrar numerosas propuestas de redes profundas para la clasificación de imágenes, como por ejemplo **Darknet-53** (Redmon and Farhadi, 2018), **DenseNet** (Huang et al., 2017), **ResNet** (*Residual Network*) (He et al., 2016), **Inception v3** (Szegedy et al., 2015), **VGG-16 / VGG-19** (Simonyan and Zisserman, 2014), y **Xception** (Chollet, 2016).

Encontramos también arquitecturas diseñadas para obtener una mayor eficiencia, lo cual facilita por ejemplo que sean ejecutadas en dispositivos móviles o incluso en chips tipo *Field Programmable Gate Array* (FPGA). Entre estas arquitecturas, destacamos modelos como **EfficientNet** (Tan and Le, 2019), **MobileNet v1** (Howard et al., 2017) (incluyendo sus versiones **v2** (Sandler et al., 2018) y **v3** (Howard et al., 2019) más recientes), y **SqueezeNet** (Iandola et al., 2016).

En el desarrollo de esta tesis usaremos y compararemos varios de estos modelos, por lo que en el Apéndice D se ha incluido una descripción extendida de las mismas.

3.1.2. Detección de objetos

De forma similar al caso de la clasificación de imágenes, se acepta ampliamente que el progreso de la detección de objetos ha pasado por dos grandes períodos históricos: el período de detección de objetos tradicional (antes del año 2014) y el período de detección basado en el aprendizaje profundo (después del año 2014) (Zou et al., 2019).

Durante la era tradicional, la mayoría de los algoritmos de detección de objetos se construían sobre la base de funciones diseñadas *ad hoc*, extrayendo características “he-

chas a mano” (*handcrafted features*). A su vez podemos organizar estas técnicas en dos grandes grupos: métodos basados en apariencia y métodos basados en características. En el primer grupo encontramos propuestas como los detectores de Viola Jones (Viola and Jones, 2001), para la detección de caras, descriptores basados en *Histogram of Oriented Gradients* (HOG) (Dalal and Triggs, 2005), que utilizan la orientación del gradiente en regiones de la imagen, o el método *Deformable Part-based Model* (DPM) (Felzenszwalb et al., 2008), que sigue la filosofía de divide y vencerás identificando diferentes partes del objeto y reconociendo el todo a partir de sus partes. En el segundo grupo (métodos basados en características) encontramos propuestas como SIFT (*Scale-Invariant Feature Transform*) (Lowe, 2004), que nos proporciona descriptores que identifican puntos característicos de la imagen, o Haar-like (Lienhart and Maydt, 2002), que entrena una función en cascada a partir de una gran cantidad de imágenes positivas y negativas.

No obstante, debido a la gran diversidad de posibles apariencias, condiciones de iluminación y fondos, es difícil diseñar manualmente descriptores de características robustos que describan perfectamente todo tipo de objetos (Zhao et al., 2019). Por este motivo, los resultados mediante estos métodos eran limitados y se habían estancado, y no fue hasta la aparición del DL que se consiguió seguir avanzando en el área. En concreto, en 2014 se produjo un gran avance en la detección de objetos debido a la publicación del modelo *Region Based Convolutional Neural Network* (RCNN) (Girshick et al., 2014, 2015). Desde entonces, la detección de objetos comenzó a evolucionar a una velocidad sin precedentes con métodos basados en DL.

En el presente estudio nos centramos en los principales modelos presentados en los últimos años, y que por lo tanto se encuentran englobados en la era del aprendizaje profundo. Dentro de este conjunto de modelos, distinguiremos entre métodos de detección de objetos supervisados y débilmente supervisados.

3.1.2.1. Aproximaciones supervisadas

Nos centraremos ahora en revisar el estado del arte de los métodos de detección de objetos supervisados, los cuales los agruparemos en los que se basan en realizar **dos etapas** o pasos, los que realizan solamente **una etapa**, y **otros métodos** de detección de objetos específicos.

Con respecto a los primeros, los detectores basados en **dos etapas** utilizan dos módulos para realizar este proceso:

- Un primer módulo para la **generación de propuestas de regiones**, donde se busca encontrar un número arbitrario de objetos en la imagen de entrada y las regiones que ocupan.

- Un segundo módulo para la **clasificación del objeto de cada región**, donde además de obtener la categoría de cada objeto propuesto en la etapa anterior, puede refinarse también la región que ocupa para obtener su cuadro delimitador (*bounding box*) así como eliminar posibles propuestas de objetos repetidas.

Entre los métodos de dos etapas, encontramos R-CNN ([Girshick et al., 2014, 2015](#)), Fast R-CNN ([Girshick, 2015](#)) y Faster R-CNN ([Ren et al., 2015](#)), en las que se han propuesto mejoras sucesivas sobre esta arquitectura hasta conseguir un sistema capaz de funcionar casi en tiempo real. En las dos primeras arquitecturas utilizaron lo que se conoce como método de búsqueda selectiva (*selective search*) para realizar las propuestas de regiones para una imagen de entrada determinada. La búsqueda selectiva es un proceso lento que afecta el rendimiento de la red, por lo que Faster R-CNN eliminó este algoritmo gracias a la incorporación de una red convolucional que proporciona un mapa de características denominado *Region Proposal Network* (RPN), a partir del cual se obtienen las propuestas de regiones. Además, con esta red se reduce el número de regiones propuestas a la vez que se mantiene la precisión de la detección de objetos.

Además de estos también encontramos otros métodos de dos etapas que utilizan mapas de características para la detección de objetos. Por ejemplo, [Li et al. \(2018\)](#) propuso una topología similar a Faster R-CNN, llamada HSF-Net y orientada a la detección de barcos, que emplea una red de propuesta de regiones para generar candidatos a partir de mapas de características. [Huang et al. \(2019a\)](#) propuso una nueva arquitectura denominada *Squeeze Excitation Skip-connection Path Networks* (SESPNets), basada en una red piramidal con la que se busca mejorar la localización multi-escala, ya que estos métodos suelen funcionar bien con objetos de tamaño medio o grande, pero presentan problemas para la detección de objetos de tamaño reducido. Otra propuesta que sigue esta vía es *You Only Look Twice* (YOLT) ([Etten, 2018](#)), la cual, en este caso, se centra en la detección de objetos pequeños en imágenes grandes, ya que se aplica para el procesamiento de imágenes aéreas.

El problema de todos estos métodos es que, al tener que realizar dos pasos de procesamiento separados, generalmente requieren más tiempo para generar propuestas y tienen una arquitectura más compleja. Los métodos de **una sola etapa**, por el contrario, presentan un mayor rendimiento y un diseño más simple. En este caso, la clasificación y la obtención de los *bounding boxes* de los objetos se obtienen directamente, sin realizar de forma previa una propuesta de regiones.

Dentro de la categoría de métodos basados en una sola etapa podemos destacar YOLO (*You Only Look Once*) ([Redmon et al., 2016](#)) como uno de los más populares. Esta arquitectura aborda la detección de objetos como un problema de regresión, con el que para cada objeto se obtiene la región que lo delimita y la probabilidad de pertenencia a las diferentes clases de objetos consideradas. Sus sucesores, YOLO v2 ([Redmon and Farhadi, 2017](#)) y v3 ([Redmon and Farhadi, 2018](#)), introducen mejoras en la eficiencia,

la precisión y la capacidad de detección multi-escala respecto a la primera versión. Otra propuesta basada en una sola etapa y que también es bastante utilizada por sus buenos resultados es RetinaNet (Lin et al., 2018). Esta arquitectura propone el uso de *Feature Pyramid Networks* (FPN), para extraer y procesar característica a distintas escalas, y de la pérdida focal (*focal loss*) para mejorar la precisión de la detección. Esta pérdida se introduce como una mejora de *Cross-Entropy Loss* para abordar el problema del gran desequilibrio que existe entre los datos de primer plano (los objetos a detectar) y la cantidad de datos de fondo.

Aparte de los métodos mencionados anteriormente de una y dos etapas, también podemos encontrar **otros métodos** previos a estos, los cuales se solían basar en el uso de descriptores seleccionados a mano (Lure and Rau, 1994; Weiss et al., 1997). En ellos, a partir de las características extraídas se clasificaba el objeto aplicando una medida de similitud. Ejemplos más recientes que siguen esta misma aproximación incluyen el uso de un detector de objetos basado en HOG-LBP (Zhang et al., 2011), un procedimiento de aprendizaje de instancias múltiples (Cinbis et al., 2014), el uso de pistas implícitas a partir de etiquetas de la imagen (Hwang and Grauman, 2012) o de probabilidades de intensidad de píxeles combinadas con descriptores LBP (Yang et al., 2017).

En general, los métodos de detección de objetos supervisados tiene una principal desventaja, que es el hecho de que necesitan una gran cantidad de datos etiquetados para el entrenamiento. Esto supone una tarea muy costosa en términos de tiempo, recursos y esfuerzo. Este es el problema que se pretende solucionar con los métodos débilmente supervisados que trataremos en la siguiente sección.

3.1.2.2. Métodos débilmente supervisados

Recientemente, el aprendizaje débilmente supervisado ha recibido una gran atención dentro de la comunidad de visión por computador (Shao et al., 2021). Enfoques de este tipo han sido propuestos en la última década para abordar diferentes retos, como son la detección de objetos, la segmentación semántica (Chen et al., 2017) y la reconstrucción en 3D (Fan et al., 2017), entre otros.

En la tarea de localización de objetos que nos atañe, una aproximación habitualmente seguida para el enfoque débilmente supervisado es el uso de los mapas de características de una CNN estándar entrenada para tareas de clasificación. Siguiendo este enfoque se analizan las activaciones de salida de cada filtro para estimar la localización de los objetos. Algunos de estos métodos también consideran los gradientes de error para obtener las localizaciones que han hecho una mayor contribución a la predicción de una clase en particular.

Uno de los primeros métodos débilmente supervisados para la localización de objetos

basado en redes neuronales fue propuesto por [Simonyan et al. \(2013\)](#). Este enfoque, a partir de una predicción dada para una determinada clase, realiza una pasada de retro-propagación para obtener el gradiente con respecto a la imagen de entrada. El resultado, que marca la posición del objeto en la imagen, se calcula como el promedio de los gradientes de los filtros cuando el valor del mapa de características es positivo. Posterior a este encontramos la técnica denominada *Class Activation Mapping* (CAM) ([Zhou et al., 2016](#)). En este caso, los mapas de características de la última capa convolucional se agrupan espacialmente utilizando una operación de *Global Average Pooling* (GAP) ([Lin et al., 2014b](#)) y, utilizando los pesos aprendidos en la capa final, se transforman para obtener el mapa de activación. Sin embargo, el principal problema de CAM es la necesidad de adaptar la arquitectura y de tener que re-entrenarla para utilizar el método. *Gradient-weighted Class Activation Mapping* (Grad-CAM) ([Selvaraju et al., 2016](#)) se introdujo para superar estas limitaciones y permitir su uso con cualquier arquitectura CNN sin tener que adaptarla.

Destacamos en este punto que en el Capítulo 4 el método Grad-CAM es aplicado para la tarea de localización de gestos, y en el Capítulo 5 proponemos un nuevo método débilmente supervisado basado en un proceso de selección de filtros para, a partir de sus mapas de características, estimar de forma precisa la localización de los objetos de interés en la imagen.

Encontramos también otros métodos débilmente supervisados que siguen otros tipos de aproximaciones, como por ejemplo el modelo *UntrimmedNets* para el reconocimiento de acciones ([Wang et al., 2017](#)), que aprende a partir de vídeos sin recortar y sin el requisito de contar con anotaciones temporales de instancias de acción. También en esta categoría, encontramos un método para la localización de objetos, que separa el primer plano (el objeto de interés) del fondo para proporcionar a las redes de clasificación información de correlación espacial de píxeles ([Zhang et al., 2018](#)). Otro enfoque que podría aplicarse a cualquier red diseñada para la localización de objetos consiste en modificar la imagen de entrada ocultando parches al azar, y así forzar la red a buscar otras partes relevantes cuando la parte más discriminativa está oculta ([Singh and Lee, 2017](#)). En el ámbito de la aplicación industrial, [Ge et al. \(2020\)](#) propone un marco de segmentación-agregación para aprender detectores de objetos a partir de datos visuales con anotaciones débiles (utilizando solo etiquetas de categoría a nivel de imagen, sin cuadros delimitadores).

En esta sección hemos hablado sobre los métodos de detección de objetos de una forma general. No obstante, existen numerosos métodos especializados en la detección de gestos de la mano. Dado que nuestro trabajo en la presente tesis forma parte de ellos, en las próximas secciones revisaremos el estado del arte de dichos métodos.

3.2. Reconocimiento de gestos

Para el reconocimiento de gestos de las manos encontramos tanto métodos basados en *hardware* dedicado (Sonkusare et al., 2015) (o en otro tipo de accesorios) como métodos basados en visión por computador (Manduchi and Coughlan, 2012). A continuación, presentamos el estado del arte de cada categoría, centrándonos especialmente en el segundo grupo.

3.2.1. Métodos basados en hardware

Los sistemas dentro de este grupo dependen de dispositivos o accesorios específicos para la detección de los gestos. Destacan en este caso las soluciones basadas en guantes equipados con sensores (Mazumdar et al., 2013; Mummadi et al., 2018) o guantes etiquetados con marcas de colores (Wang and Popović, 2009; Lamberti and Camastra, 2011). La Figura 3.2 muestra un ejemplo de un guante con sensores utilizado para reconocer el lenguaje de signos. En el caso del segundo tipo de guantes, cada color es utilizado para identificar las diferentes partes de la mano, facilitando de este modo que mediante una cámara podamos detectarlas y hacer un seguimiento de las mismas, sin necesitar ningún *hardware* adicional.

Además de guantes, existen otros tipos de sensores cuyo propósito es también reconocer gestos de la mano, como dispositivos *wearables* que monitorizan la actividad muscular basándose en la electromiografía de superficie (Moin et al., 2021) o sensores Doppler ultrasónicos (Raj et al., 2012). En la Figura 3.3 se muestra el uso de un radar para detectar los gestos de la mano.

3.2.2. Métodos basados en visión artificial

Los métodos basados en visión artificial tienen la ventaja de que únicamente requieren una cámara, lo cual es un elemento con el que en la actualidad contamos en la práctica totalidad de dispositivos móviles, por lo que en este caso no es necesario usar un *hardware* especial para el reconocimiento. Esto hace que este tipo de métodos sean una solución natural, no intrusiva y sin contacto, a diferencia de los métodos vistos previamente que sí que necesitan que el usuario use dispositivos que en ocasiones pueden ser incómodos. Lo más común es encontrar cámaras RGB (*Red Green Blue*), que nos permiten adquirir imágenes 2D en color. No obstante, en los últimos años cada vez más dispositivos incorporan cámaras ToF (*Time of Flight*) (Oggier et al., 2006). Este tipo de cámaras utilizan luz infrarroja para obtener información de profundidad. De esta manera, con ellas podemos obtener imágenes RGB-D, que además de los canales con información de color



Figura 3.2.: Detector de gestos en tiempo real mediante guantes con sensores. La detección se realiza en un sistema integrado en la parte posterior del guante, el cual se conecta a través de *Bluetooth* con un teléfono móvil para mostrar los gestos detectados. Fuente (Mummadi et al., 2018).

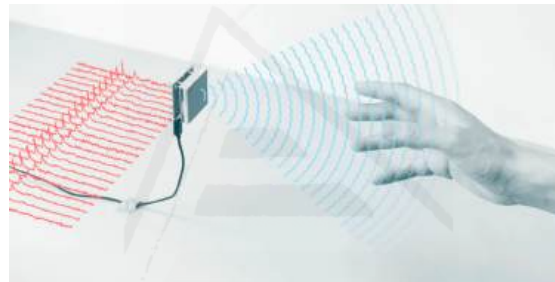


Figura 3.3.: El proyecto Soli utiliza un radar en miniatura para detectar gestos de las manos. Imagen adaptada de *Google's Advanced Technology and Projects Group* (ATAP).

RGB incorporan un canal con el mapa de profundidad de la escena (ver Figura 3.4). Existen también otros tipos de cámaras, como son las cámaras de visión nocturna o las cámaras termográficas (Williams, 2009) (ver Figura 3.5), pero su uso se limita a campos más especializados.

Además de la cámara, necesitaremos contar con un método para reconocer los gestos a partir de la imagen adquirida y un sistema con la capacidad de cómputo necesaria para ejecutarlo. Como ya hemos comentado previamente, los dispositivos móviles actuales, además de tener cámaras de alta calidad, cuentan con una alta capacidad de cómputo, lo que nos permite ejecutar en ellos la mayoría de estos métodos. Con respecto al algoritmo en cuestión a utilizar, en general todos ellos suelen realizar uno o varios de los siguientes pasos a la hora de llevar a cabo el proceso de detección:

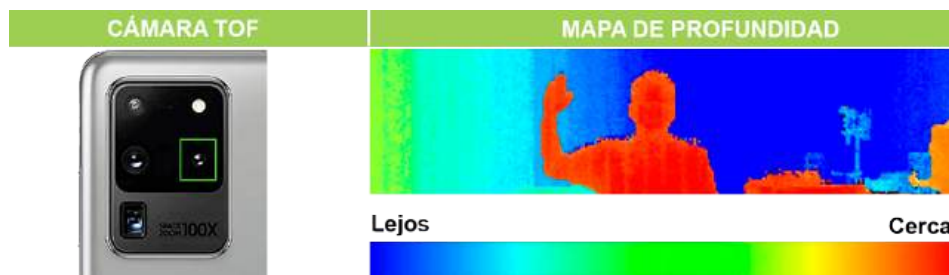


Figura 3.4.: Ejemplo de cámara ToF integrada en un teléfono móvil (izquierda) y del mapa de profundidad obtenido (derecha). El color azul representa objetos lejanos y el rojo los cercanos. Fuente (Yang et al., 2019).



Figura 3.5.: Ejemplo de imágenes capturadas mediante una cámara termográfica (izquierda), donde marca claramente la temperatura corporal de las personas que se ven en la imagen, y mediante una cámara nocturna (derecha) de uso militar.

- **Pre-procesamiento de la imagen.** Según el enfoque, se pueden aplicar operaciones para segmentar la mano, eliminar ruido o adaptar las propiedades de la imagen a los requisitos de entrada del algoritmo.
- **Extracción de características.** Encontramos métodos que se basan en determinadas características de las manos para su detección, como podría ser el esqueleto de la mano, información geométrica 3D, información de profundidad, el color de la piel o características extraídas por redes de aprendizaje profundo (Pisharady and Saerbeck, 2015).
- **Método de clasificación.** Por último se aplica un algoritmo de clasificación, bien sobre las características previamente extraídas o directamente sobre la entrada. Para esto es habitual el uso de redes neuronales o de otros métodos de clasificación, como Random Forest, SVM o kNN (Narayanan et al., 2017).

En este grupo de enfoques para el reconocimiento de gestos de la mano basados en visión por computador distinguimos a su vez dos subcategorías dependiendo del tipo de características utilizadas: los basados en características 3D y los basados en apariencia. Revisaremos a continuación el estado del arte de cada una de estas aproximaciones.

3.2.2.1. Métodos basados en información 3D

Aunque encontramos aproximaciones que procesan directamente los puntos 3D o que extraen información volumétrica (Ge et al., 2019), no obstante, la mayoría de enfoques basados en datos 3D, utilizan el esqueleto de la mano. Este esqueleto podría obtenerse mediante *hardware* específico, como guantes con sensores, aunque este caso entraría dentro de la categoría de soluciones basadas en *hardware*. Otra opción son los dispositivos basados en cámaras de profundidad que incorporan funciones para la obtención y seguimiento del esqueleto. Este es el caso de Microsoft Kinect (Xi et al., 2018) y de Leap Motion Controller¹, que han sido aplicadas, por ejemplo, para la realización de ejercicios de rehabilitación de la mano (Xiao et al., 2021).

Existen también librerías como OpenPose Hand Detection² (Cao et al., 2019; Simon et al., 2017) y MediaPipe Hands³ (Zhang et al., 2020a), que nos permiten obtener el esqueleto 2D o 3D de la mano a partir de una imagen RGB. También encontramos métodos que estiman el esqueleto 3D de la mano a partir de imágenes RGB-D (Ge et al., 2019) o a partir de una secuencia de imágenes RGB (Tekin et al., 2019).

Estos datos 3D sobre el esqueleto de la mano, que consisten habitualmente en las coordenadas de las articulaciones, posteriormente son procesados mediante diferentes tipos de clasificadores para reconocer los gestos, como por ejemplo Modelos Ocultos de Markov (HMM) y Dynamic Time Warping (DTW) en Raheja et al. (2015) o CNN en Devineau et al. (2018). De Smedt et al. (2017) presenta una comparativa entre un clasificador SVM que utiliza el esqueleto de la mano y una CNN que procesa imágenes RGB-D como entrada (ver Figura 3.6), donde se muestra que el primer enfoque, para el tipo de aplicación propuesto, presenta mejores resultados en el reconocimiento de gestos.

Sin embargo, el uso de librerías como OpenPose o MediaPipe para recuperar los datos del esqueleto a partir de imágenes RGB tiene ciertas limitaciones, dado que se ha comprobado que para ciertas poses no es posible reconocer de forma precisa los puntos del esqueleto (Amaliya et al., 2021a).

¹<https://www.ultraleap.com/product/leap-motion-controller/>

²<https://github.com/CMU-Perceptual-Computing-Lab/openpose>

³<https://mediapipe.dev>



Figura 3.6.: Datos de profundidad y esqueleto de la mano obtenidos mediante la cámara *Intel RealSense*. Las 22 articulaciones devueltas incluyen el centro de la palma, la muñeca y cuatro articulaciones para cada dedo (las dos articulaciones más la punta y la base) (De Smedt et al., 2017).

3.2.2.2. Métodos basados en apariencia

Este tipo de métodos no utilizan una representación geométrica de la mano, sino que realizan el reconocimiento directamente a partir de las imágenes o vídeos. En este grupo encontramos numerosas técnicas basadas en segmentar la mano por color a partir de imágenes RGB (Pun et al., 2011; Prakash and Gautam, 2019). Posteriormente, para detectar el gesto, calculan el radio de la mano segmentada y consideran que los *blobs* que queden fuera de este radio son los dedos extendidos (Perimal et al., 2018). Este tipo de métodos tiene la ventaja de que no necesitan una base de datos de gestos para ser entrenados (Rajesh et al., 2012), pero tienen la gran limitación de que solo pueden reconocer gestos que consistan en dedos plegados o extendidos. En Prakash et al. (2017) proponen detectar la posición de los dedos extendidos a partir de los vértices del recubrimiento convexo de la mano. Otros métodos utilizan un procedimiento parecido, pero añaden información de profundidad obtenida de imágenes RGB-D para segmentar la mano (Kim and Lee, 2016). Existen también soluciones similares que analizan la curva de la mano para identificar los dedos desplegados (Ren et al., 2011) y las posiciones de las puntas de los dedos (Lai et al., 2016).

También podemos encontrar enfoques que utilizan algún tipo de marcador para facilitar el proceso de detección, como aquellos basados en guantes de colores (ver Figura 3.7). Estos guantes permiten identificar cada parte de la mano mediante el uso de diferentes colores, de este modo, aplicando una segmentación por color se puede realizar un seguimiento preciso del movimiento de la mano (Lamberti and Camastra, 2011). Por ejemplo, Hasan and Mishra (2012) usa guantes de colores para la interacción con modelos 3D, permitiendo diferentes funciones, como hacer *zoom*, mover, dibujar y escribir utilizando un teclado virtual. En Balbin et al. (2016) encontramos otro ejemplo del uso de guantes de colores, en este caso desarrollan un traductor de palabras en lenguaje de signos.

En general, los métodos basados en segmentación por color resultan poco robustos ante cambios de iluminación y posibles coincidencias con colores del fondo.



Figura 3.7.: Reconocimiento basado en colores mediante guantes con marcadores (Wang and Popović, 2009).

Por el contrario, los métodos basados en profundidad no se ven afectados por cambios de iluminación y son más robustos ante variaciones del fondo. En este caso podemos encontrar diversos enfoques. Por ejemplo, en Dinh et al. (2014), la información de profundidad se utiliza en primer lugar para segmentar la silueta de la mano y eliminar el fondo, y posteriormente se aplica un clasificador de tipo Random Forest para reconocer las partes de la mano. Otra propuesta similar para el reconocimiento de gestos estáticos se basa en obtener un histograma basado en profundidad y aplicar ANN y SVM para la clasificación (Bamwenda and Özerdem, 2019).

También encontramos propuestas para el reconocimiento de gestos dinámicos. Por ejemplo, Lu et al. (2016) calcula un vector de características con información de profundidad a partir de imágenes obtenidas mediante *Leap Motion Controller* y, sobre esta información, aplica un clasificador *Hidden Conditional Neural Field* (HCNF) para detectar este tipo de gestos. En Molina et al. (2017) se utilizan secuencias de imágenes RGB-B para reconocer patrones de movimiento y, de esta forma, identificar gestos dinámicos. Para mejorar tanto la eficiencia como la precisión en los casos en los que como entrada tenemos una secuencia de imágenes, en Tang et al. (2019) se propone seleccionar una serie de fotogramas clave para reducir el número de muestras a procesar. Las características en este caso se obtienen a partir de la apariencia y del movimiento entre dos fotogramas clave consecutivos. Este método posteriormente aplica un enfoque de tipo *Bag of Features* (BoF) para clasificar los gestos.

En general, para el reconocimiento de gestos a partir de imágenes RGB y RGB-D, los métodos que se han mostrado más efectivos en los últimos años, al igual que en el caso del reconocimiento de objetos en general, son los métodos basados en redes neuronales profundas. La mayor parte de estos enfoques utilizan diferentes arquitecturas de CNN, como las que hemos enumerado en secciones anteriores para la clasificación de imágenes

y detección de objetos. Entre las propuestas específicas para la clasificación de gestos, existen enfoques basados en CNN como el presentado por [Lin et al. \(2014a\)](#), que incorpora un paso previo de calibración. Las CNN se han aplicado para reconocer el lenguaje de signos tanto mediante fotogramas individuales ([Bheda and Radpour, 2017](#)) como en secuencias de fotogramas ([Pigou et al., 2015](#)), necesarias para la detección de gestos dinámicos. En [Molchanov et al. \(2015\)](#), se propone una CNN que toma como entrada tanto la intensidad de los píxeles como el mapa de profundidad obtenido a partir de secuencias de vídeo, con el objetivo de reconocer gestos dinámicos destinados a una interfaz sin contacto para automóviles. También podemos encontrar otros enfoques de aprendizaje profundo basados en la segmentación de las manos a partir de profundidad, como es el caso de SegNet, una arquitectura profunda convolucional de tipo *encoder-decoder* propuesta para detectar las puntas de los dedos ([Nguyen et al., 2019](#)). El reconocimiento de gestos dinámicos en secuencias de vídeo también se ha abordado mediante el uso de arquitecturas de tipo Bidirectional Long Short-Term Memory (BiLSTM) ([Xie et al., 2017](#)) y Temporal Segment Networks (TSN) ([Benitez-Garcia et al., 2021](#)) con el objetivo de capturar la información temporal.

En el presente trabajo, dado que buscamos reconocer de forma general cualquier tipo de gesto de la mano utilizando la cámara que incorporan la mayor parte de teléfonos inteligentes actuales, nos centraremos en métodos basados en redes neuronales profundas que tomen como entrada una imagen RGB. Además del reconocimiento de gestos, nuestro propósito es integrar en la red funciones para identificar objetos o describir la escena. Para hacer esto de manera eficiente, seguimos un enfoque similar a [Köpüklü et al. \(2019\)](#), en el que se implementa una arquitectura de red de dos etapas para construir un detector de gestos en tiempo real: la primera etapa está dedicada a la detección del gesto y la segunda a la clasificación, que solo se ejecuta en el caso en que previamente se haya detectado la presencia de un gesto en la imagen. En nuestro caso, en el Capítulo 6 proponemos una arquitectura *multi-head* con un *backbone* dedicado a la clasificación de gestos, y una cabeza especializada para cada gesto, que solo se ejecutará si se detecta el gesto al que corresponde. A diferencia del trabajo de [Köpüklü et al. \(2019\)](#), nuestra propuesta clasifica el gesto en el primer paso, y se añade un segundo paso con el objetivo de realizar las acciones especializadas que desencadenará la detección de cada uno de estos gestos, los cuales, además, podrán ser tanto estáticos como dinámicos.

3.3. Image captioning

Los métodos de *captioning* tienen como objetivo generar una descripción textual de una imagen, para lo cual es necesario combinar técnicas de procesamiento del lenguaje natural con visión por computador. Este es un tema de gran interés en el campo de la IA, ya que se trata de un problema muy complejo al que solo se ha podido empezar a dar solución recientemente gracias a la combinación de técnicas de DL. Normalmente,

los modelos de descripción de imágenes que utilizan enfoques de aprendizaje profundo se componen de dos partes: un módulo de extracción de características de la imagen y otro módulo para generar la descripción textual a partir de las características anteriores.

Gracias a la combinación de los numerosos avances en el campo del reconocimiento de objetos y en el de traducción automática de textos, se han creado modelos capaces de describir con precisión el contenido de una imagen (You et al., 2016; Tanti et al., 2018), de un vídeo (Lee and Kim, 2018) e incluso de responder preguntas a partir de la información de un vídeo (Kim et al., 2020). La mayoría de estos modelos utilizan un marco codificador-decodificador. En este tipo de arquitectura la imagen de entrada alimenta primero a un codificador, que generalmente es una CNN, la cual extrae una serie de características y las codifica en un descriptor llamado vector de contexto. Este vector se pasa como entrada al decodificador, que lo procesa para generar una descripción de la imagen dada (Huang et al., 2019b). También podemos encontrar enfoques que utilizan capas de atención para mejorar los resultados. En este caso, en lugar de usar el vector de contexto como entrada, toman la entrada de la última capa de convolución del codificador CNN para preservar así los detalles espaciales de las características (Huang et al., 2019c).

La descripción de imágenes tiene una amplia gama de escenarios de aplicación, puede ser útil para la indexación automática de imágenes, para automatizar tareas de etiquetado y clasificación, en redes sociales, entre otras muchas. Por ejemplo, en Shuster et al. (2019) se presenta un trabajo para la personalización de contenidos que se basa en analizar las descripciones de las imágenes que resulten más atractivas al usuario. Otra de las aplicaciones en las que el *captioning* resulta de gran utilidad es para la ayuda a personas con discapacidad visual, no solo para generar automáticamente descripciones de las imágenes, sino para otro tipo de tareas como transcribir el texto que se muestra en ella (Shirke et al., 2018).

CAPÍTULO 4

CLASIFICACIÓN Y LOCALIZACIÓN DE GESTOS

Resumen

La investigación que se desarrolla en este capítulo introduce una metodología que nos proporciona una base para alcanzar los objetivos de la presente tesis. Se trata de un método para localizar y reconocer gestos de la mano a partir de imágenes basado en *Deep Learning*. Inicialmente se definen los gestos a detectar y se crea un conjunto de datos con imágenes correspondientes a estos. Este conjunto se utiliza para entrenar y evaluar la metodología propuesta, usando redes neuronales con diferentes topologías para clasificar y localizar los gestos, para lo cual nos basaremos además en *Grad-CAM* para el cálculo de los mapas de calor.

Contenidos

4.1	Introducción	45
4.2	Método	46
4.3	Datasets	49
4.4	Experimentación	51
4.5	Conclusiones	57

4.1. Introducción

Tal como se indicó en la Sección 1.1, uno de nuestros principales objetivos es el desarrollo de métodos de detección de gestos que puedan ser ejecutados en tiempo real en dispositivos móviles, de modo que resulten útiles para la elaboración aplicaciones de ayuda a personas con discapacidad visual.

En este capítulo abordaremos por lo tanto el desarrollo de métodos eficientes para el reconocimiento de gestos, así como el objetivo de validar dicha metodología con datos reales. Para alcanzar estos objetivos generales planteamos una serie de hitos: en primer lugar la clasificación de los gestos, los cuales deberán ser posteriormente localizados dentro del espacio de coordenadas de la imagen y, por último, y con el objetivo de que el método pueda funcionar en tiempo real en dispositivos móviles, debemos buscar aquellas soluciones que maximicen tanto la precisión como la eficiencia.

Tal como se ha visto en el Capítulo 3, actualmente los enfoques más prometedores para abordar problemas de reconocimiento y localización de objetos en imágenes son aquellos basados en *Deep Learning* (DL) y, en concreto, en arquitecturas de red tipo CNN. Por ello, a lo largo de este capítulo exploraremos y evaluaremos diferentes modelos CNN aplicados a la tarea de reconocimiento de gestos de la mano.

Debido a que dichos modelos deben ser entrenados y evaluados con datos del dominio del problema, en este caso la detección de gestos, crearemos nuestro propio conjunto de datos con imágenes de este tipo, incluyendo cuatro clases de gestos estáticos: “apuntar con un dedo”, “con dos dedos”, “lupa” y “otros gestos” distintos a estos (ver Figura 4.1), así como una clase adicional que no contiene “ningún gesto” y que nos permitirá validar la robustez tanto de la clasificación como de la detección.



Figura 4.1.: Descripción gráfica de los gestos propuestos: apuntar con un dedo, con dos dedos, lupa y el pulgar hacia arriba, considerado como “otro gesto”.

Para llevar a cabo la localización de los gestos en la imagen nos basaremos en el enfoque *Gradient-weighted Class Activation Mapping* (Grad-CAM) (Selvaraju et al., 2016). Este método calcula un mapa que nos indica cuánto contribuye cada píxel en las predicciones realizadas por la red, que en este caso nos indicarán la clasificación de los gestos. Al procesar cada imagen el método analiza los gradientes de la clase objetivo en la capa convolucional final, generando un mapa que marca las zonas de la

imagen que han sido más relevantes para la predicción realizada. Con ello tendremos una localización aproximada de la región de la imagen en la que estará localizado el gesto que fue clasificado por la red.

4.2. Método

En la Figura 4.2 se muestra un esquema general de las distintas partes de la metodología propuesta. El enfoque seguido para el reconocimiento y localización de gestos se basa en una arquitectura CNN entrenada para clasificación y en el cálculo de *heat maps* o mapas de calor (Selvaraju et al., 2016). Como se puede ver en este mismo esquema, durante la etapa de entrenamiento solo se ajustan los pesos de la CNN, la cual se inicializa usando *transfer learning* (que explicaremos en la Sección 4.2.1) y partiendo de estos pesos se entrena con nuestro conjunto de datos. En la etapa de inferencia se utiliza la CNN ya entrenada para predecir la categoría de las imágenes y, en caso de detectar un gesto, se aplica la técnica de Grad-CAM para calcular el área de activación de la predicción actual, obteniendo de esta forma la posición del gesto. A continuación, veremos en detalle cada uno de estos pasos.

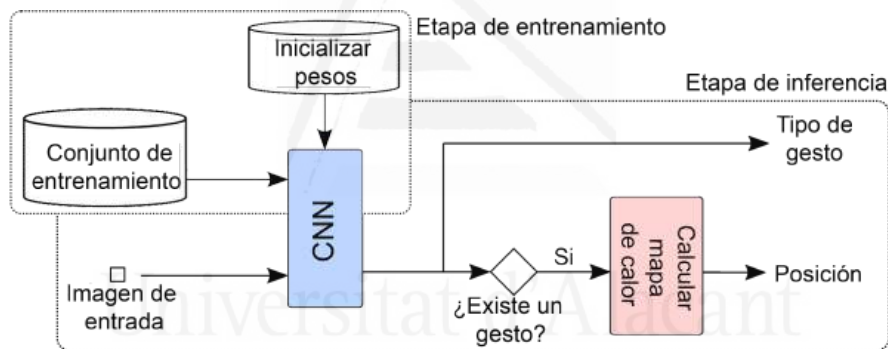


Figura 4.2.: Esquema gráfico de la metodología propuesta para la detección y localización de gestos.

Para la arquitectura CNN del esquema propuesto compararemos tanto la eficiencia como la eficacia de siete topologías diferentes del estado del arte, incluyendo:

- Dos modelos clásicos: VGG-16/19 (Simonyan and Zisserman, 2014).
- Tres modelos de red más recientes que mejoran los resultados de las redes clásicas en este tipo de tareas: ResNet (He et al., 2016), Inception V3 (Szegedy et al., 2015) y Xception (Chollet, 2016).
- Dos arquitecturas ligeras (eficientes) destinadas a obtener buenos resultados con

un número reducido de parámetros: MobileNet ([Howard et al., 2017](#)) y SqueezeNet ([Iandola et al., 2016](#)).

Se puede encontrar más información sobre estas redes en el Apéndice D o consultando las referencias correspondientes.

Además de la precisión de clasificación o detección de estas topologías, otro aspecto importante que tendremos en cuenta para nuestra propuesta es su eficiencia, ya que el objetivo es utilizar este método en un dispositivo móvil. Para esto consideraremos el número de parámetros de la arquitectura, puesto que cuantos más tenga, más lento será su funcionamiento y mayor será el espacio de almacenamiento requerido. Sin embargo, es importante llegar a un compromiso entre la eficiencia y la eficacia de la red, ya que al reducir el número de parámetros también se ve afectada la precisión. En la sección de experimentación analizaremos en detalle los resultados obtenidos en función de estos dos criterios.

La Tabla 4.1 muestra el número de parámetros de las redes consideradas en orden descendente. Como puede verse, las redes con menos parámetros son MobileNet y SqueezeNet, las dos que habíamos destacado como “ligeras”.

Tabla 4.1.: Número de parámetros de cada red en orden descendente.

Modelo	#Parámetros	Modelo	#Parámetros
ResNet	29.887.365	VGG 16	17.868.613
Inception v3	28.102.437	MobileNet	7.431.365
Xception	27.161.133	SqueezeNet	3.876.421
VGG 19	23.178.309		

Como hemos indicado, una vez detectado un gesto, utilizamos Grad-CAM ([Selvaraju et al., 2016](#)) para el cálculo de su posición en la imagen. Esta técnica habitualmente se emplea para producir explicaciones visuales de las predicciones realizadas por las redes CNN, ya que permite marcar sobre la imagen de entrada las zonas en las que se ha fijado la red para realizar una determinada predicción. La forma de marcar esto es mediante los mapas de calor o *heat maps*, en los cuales se representan mediante valores más altos las zonas que resultaron más relevantes para la predicción.

En nuestro caso, para calcular estos mapas de calor usamos el enfoque Grad-CAM, que es una generalización del método *Class Activation Mapping* (CAM) propuesto por [Zhou et al. \(2016\)](#). Gracias a este método no es necesario utilizar ninguna técnica de localización adicional, sino que a partir del propio resultado de la CNN se calcula el gradiente con respecto a la función de pérdida o *loss* del modelo y, en base a esto, se toman las secciones del gradiente que contribuyeron más a la predicción. Por último, este resultado se escala para ajustarlo al tamaño de la imagen original y se busca la zona de mayor activación, asumiendo esta como la posición del gesto detectado.

4.2.1. Detalles sobre el entrenamiento

Para entrenar las topologías CNN consideradas con el número de clases de nuestro conjunto de datos fue necesario modificar sus últimas capas. En concreto se reemplazaron todas las capas tipo *fully connected* de las redes originales por las tres capas que se indican en la Tabla 4.2. Para unir estas nuevas capas con el resultado de las convoluciones de la arquitecturas base (a las que denominaremos *backbones*¹), en lugar de utilizar la operación *flatten* se agregó una capa de tipo *Global Average Pooling*, la cual, además de reducir considerablemente la cantidad de parámetros necesarios para esa capa, también nos permitió aplicar posteriormente la técnica de Grad-CAM.

Tabla 4.2.: Descripción de las capas adicionales.

#	Capa	Tamaño de salida
1	Global Average Pooling	1×2048
	Fully-connected	
	Activación ReLU	
	Dropout 0.2	
2	Fully-connected	1×1024
	Activación ReLU	
	Dropout 0.2	
	Fully-connected	
3	Activación SoftMax	$1 \times \# \text{clases}$

El entrenamiento de las distintas redes se realizó mediante *backpropagation* estándar usando Descenso por Gradiente Estocástico (Bottou, 2010) para el ajuste de los pesos y considerando además el método de Zeiler (2012a) para ajustar la tasa de aprendizaje (o *learning rate*). Como función de pérdida, para calcular el error entre la salida de CNN y el resultado esperado, se utilizó *categorical crossentropy* (entropía categórica cruzada).

Además, para esta etapa consideraremos dos estrategias: entrenar la red comenzando desde cero, es decir, inicializando los pesos de manera aleatoria (a lo que nos referiremos por “no inicializado”) y entrenar la red partiendo de los pesos aprendidos usando otro conjunto de datos, utilizando un enfoque de *transfer learning* o de transferencia de conocimiento. Para esto se realizan los siguientes pasos:

1. Se inicializa la red con los pesos obtenidos del entrenamiento con otro *dataset*. En concreto nosotros nos basaremos en los aprendidos usando el conjunto de datos utilizado para ILSVRC, que en realidad es un subconjunto de 1.000 clases de ImageNet (Deng et al., 2009), una base de datos de propósito genérico para la detección de objetos.

¹Este término se utiliza habitualmente para referirse al uso de una arquitectura de red del estado del arte como extractor de características. Además esta arquitectura se suele utilizar solo parcialmente, eliminando sus capas finales, bien para obtener directamente las características obtenidas en una de sus capas o para añadirle otra serie de capas y adaptarla a otra tarea.

2. Partiendo de estos pesos se realiza el entrenamiento con nuestro conjunto de datos (que describiremos en la siguiente sección). A este proceso también se le denomina *fine-tuning*, ya que se trata de un proceso mucho menos costoso que un entrenamiento desde cero, puesto que se parte de una buena inicialización y, por tanto, solo hay que realizar un “afinado” de los pesos para nuestro conjunto de datos.

4.3. Datasets

En este apartado se presentan los conjuntos de datos utilizados para el entrenamiento y evaluación de la metodología propuesta, empezando por ImageNet, el cual se ha empleado únicamente para inicializar los pesos de los modelos, y continuando con el conjunto de datos de gestos específico de esta tarea que hemos recopilado y que utilizaremos en los experimentos que se llevarán a cabo en la siguiente sección.

4.3.1. ImageNet

ImageNet ([Deng et al., 2009](#)) es una base de datos con un total de 14 millones de imágenes de todo tipo clasificadas en 22.000 categorías. Está disponible de manera pública y se utiliza de manera habitual en multitud de tareas de investigación, como en clasificación o segmentación de imágenes o para el reconocimiento de objetos. ImageNet se ha vuelto especialmente relevante en el campo del Deep Learning por su uso en el reto *ImageNet Large Scale Visual Recognition Challenge* (ILSVRC) ([Russakovsky et al., 2015](#)), el cual consiste en clasificar en 1.000 categorías un subconjunto de 1,2 millones de imágenes de este *dataset*. Además, al contar con tantas imágenes correctamente etiquetadas, también resulta de gran utilidad para la inicialización de modelos en tareas en las que se cuenta con conjuntos de datos reducidos, ayudando en la mayoría de los casos a reducir el tiempo de entrenamiento y a obtener mejores resultados ([Gershgorn, 2017](#); [Krizhevsky et al., 2012](#)).

4.3.2. Conjunto de datos de gestos

Para entrenar y evaluar el método propuesto hemos recopilado manualmente un conjunto de datos de gestos realizados con las manos (ver Figura 4.4). Para esto utilizamos diferentes modelos de teléfonos móviles con los cuales capturamos las siguientes cinco categorías de imágenes: (i) apuntar con un dedo o (ii) con dos dedos, (iii) hacer la forma de una lupa, (iv) otros gestos (como manos abiertas o cerradas) y (v) “ningún gesto” (imágenes en las que no aparecían manos).

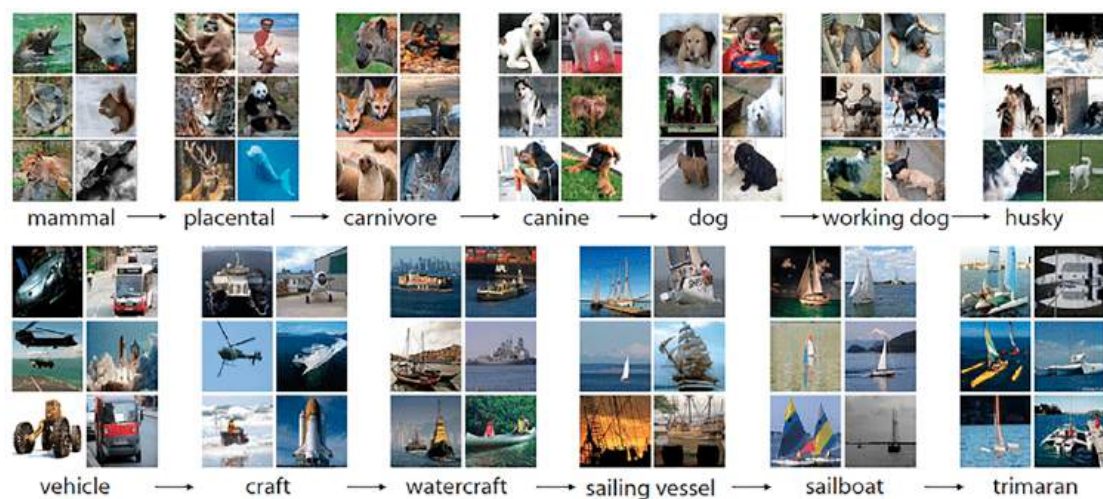


Figura 4.3.: Ejemplos de la jerarquía de ImageNet para las ramas de mamíferos y vehículos. Fuente (Deng, 2012).

El conjunto de datos recopilado contiene un total de 20 mil imágenes, para las cuales intentamos equilibrar el número de muestras de cada clase (ver Tabla 4.3). Para el entrenamiento se utilizó un conjunto formado por el 70 % de las imágenes, dejando el 30 % restante para la evaluación de los modelos. La separación de los datos de entrenamiento y validación se realizó teniendo en cuenta el fondo de la imagen, seleccionando para la validación imágenes con fondos que no se habían visto durante el entrenamiento. Además, también consideramos cuatro tamaños de imágenes comúnmente utilizados por este tipo de redes: $128 \times 128\text{px}$, $160 \times 160\text{px}$, $192 \times 192\text{px}$ y $224 \times 224\text{px}$.

Tabla 4.3.: Descripción del conjunto de datos utilizado para la evaluación del método, incluyendo el nombre de la clase, el número de muestras de cada clase y una breve descripción de las mismas.

Gesto	# muestras	Descripción
Un dedo	4.000	Señalar con un dedo
Dos dedos	3.960	Señalar con dos dedos
Lupa	4.000	Los dedos pulgar e índice forman un círculo
Otro	4.000	Otros gestos distintos a los anteriores
Ninguno	4.000	Imágenes sin manos



Figura 4.4.: Imágenes de ejemplo de nuestro conjunto de datos de gestos que muestran la variabilidad de fondos incluidos.

4.4. Experimentación

Esta sección presenta los resultados obtenidos mediante el enfoque propuesto y los conjuntos de datos descritos en la sección anterior. En primer lugar se describen las métricas que utilizaremos para evaluar cuantitativamente los resultados. Seguidamente analizaremos las distintas partes del método, incluyendo experimentos con diferentes configuraciones de red, inicialización de parámetros, y con y sin aumento de datos. Por último evaluaremos la técnica propuesta para el proceso de localización de los gestos.

4.4.1. Métricas de evaluación

Las distintas arquitecturas CNN comparadas se evaluaron utilizando la métrica F_1 , la cual se puede expresar en función de la precisión y el *recall* (exhaustividad) obtenido. Tomando una de las clases como positivas y el resto como negativas definimos:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \quad (4.1)$$

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad (4.2)$$

$$F_1 = 2 \times \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (4.3)$$

donde TP (*True Positives*) denota el número de ejemplos de la clase positiva etiquetados correctamente, FN (*False Negatives*) el número de ejemplos de la clase positiva que se han etiquetado erróneamente como otra clase, y FP (*False Positives*) el número de ejemplos de otras clases etiquetados de manera incorrecta como clase positiva.

Cuando tenemos un problema de clasificación con varias clases, como es nuestro caso, existen dos formas de combinar los resultados obtenidos para cada clase en una única métrica: a nivel micro y a nivel macro.

La métrica macro F_1 se calcula simplemente como el promedio de las F_1 de las diferentes clases. Por otro lado, micro F_1 agrega las contribuciones de TP, FN y FP de las diferentes clases para el cálculo de la precisión y el *recall*. Es decir, en este caso, consideraríamos como TP, FN y FP la suma de los TP, FN y FP obtenidos para todas las clases.

En una configuración de clasificación de varias clases es preferible el uso de la métrica micro si se sospecha que pueda haber un desequilibrio de muestras por clase (es decir, si tenemos muchos más ejemplos de una clase que de las otras). En nuestro caso se ha utilizado la forma macro, ya que hemos construido el conjunto de datos evitando que haya desequilibrio de clases.

La localización detectada se midió utilizando la métrica *Intersection over Union* (IoU), comparando el cuadro delimitador (*bounding box*) en el que el método predice que se localiza el objeto (*op*) con el cuadro delimitador del etiquetado (*ground truth* o *gt*), en el que conocemos que debe situarse dicho objeto. En caso de obtenerse más de una predicción nos quedaremos con la que tenga una mayor superposición con el GT, estableciendo el resto como FP. Las predicciones se considerarán TP si la relación entre el área de superposición del cuadro delimitador predicho (B_{op}) y el cuadro delimitador del GT (B_{gt}) excede un cierto umbral (λ) de acuerdo con la siguiente ecuación:

$$\frac{area(B_{op} \cap B_{gt})}{area(B_{op} \cup B_{gt})} > \lambda \quad (4.4)$$

donde $area(B_{op} \cap B_{gt})$ representa el área de la intersección entre el cuadro delimitador de la predicción y el del GT, y $area(B_{op} \cup B_{gt})$ representa el área de su unión. Por convención, usamos un umbral de $\lambda = 0.5$ para establecer un candidato como TP.

4.4.2. Resultados de clasificación

En primer lugar llevamos a cabo un experimento para determinar el mejor tamaño de imagen de entrada a la vez que analizamos el proceso de inicialización de las redes. Para esto, escalaremos espacialmente las imágenes entre 128×128 px y 224×224 px y compararemos los resultados según el tipo de inicialización: sin inicialización (con pesos aleatorios) y usando los pesos pre-entrenados aprendidos usando el conjunto de datos ImageNet (Deng et al., 2009).

La Tabla 4.4 muestra los resultados de este experimento en términos de la métrica $macro - F_1$ (%) comparando las siete CNN indicadas en la Sección 4.2. Para la red ResNet solo fue posible obtener resultados para el tamaño de entrada de 224×224 px, ya que se trata de una red con muchas capas de *pooling* (quedándose sin datos para tamaños inferiores). Como puede verse, los mejores resultados se obtienen en general utilizando las redes inicializadas y con tamaños grandes para la imagen de entrada. Entre los modelos considerados destacan las redes Xception, Inception v3 y MobileNet, utilizando inicialización y un tamaño de entrada de 224×224 px. Además de estas, la red SqueezeNet, con un tamaño de entrada de 192×192 px, también obtiene resultados competitivos (98,36 %).

Tabla 4.4.: Comparación del resultado (en términos de $macro - F_1$) obtenido por las distintas CNN propuestas para la tarea de clasificación usando diferentes tamaños de entrada y pesos de inicialización.

Modelo	128 × 128		160 × 160		192 × 192		224 × 224	
	No Inic.	Inic.	No Inic.	Inic.	No Inic.	Inic.	No Inic.	Inic.
VGG16	54,33	78,17	46,66	83,36	52,10	85,47	40,09	32,22
VGG19	44,24	33,17	36,12	51,74	30,15	26,17	42,77	64,04
ResNet	-	-	-	-	-	-	67,91	93,35
Xception	68,44	98,90	70,93	99,41	74,29	99,21	75,55	99,52
InceptionV3	72,67	95,28	70,65	97,03	78,91	98,11	76,26	99,04
MobileNet	54,63	97,32	54,82	98,70	59,96	98,98	64,11	99,29
SqueezeNet	28,05	92,31	31,07	93,73	34,26	98,36	36,13	97,93
Promedio	53,73	82,53	51,71	89,38	54,95	84,38	57,55	81,87

Con el objetivo de mejorar los resultados obtenidos en el experimento anterior (por tamaño de entrada y modelo de red), a continuación vamos a analizar la mejora obtenida al aplicar la técnica de aumentado de datos (Krizhevsky et al., 2012; Chatfield et al., 2014a). Esta técnica consiste en generar artificialmente variaciones en las imágenes utilizadas durante el proceso de entrenamiento. En concreto aplicamos aleatoriamente diferentes tipos de transformaciones, incluyendo zoom (hasta $\pm 0,2$ veces el tamaño de la imagen), rotaciones (en el rango $[-0,2^\circ, 0,2^\circ]$) y *flips* (reflexión) horizontales.

La Tabla 4.5 muestra los resultados obtenidos para la clasificación de gestos al aplicar esta técnica. Como puede verse, en general el proceso de aumentado de datos ayuda a mejorar los resultados previos. En algunos casos solo mejora ligeramente, al no haber mucho margen, por ejemplo, la red Xception mejora 0,25 puntos (de 99,52 a 99,77). Sin embargo, en otros casos el resultado se incrementa mucho más, como para la red VGG19, que aumenta hasta 13 puntos.

Tabla 4.5.: Comparación de resultados en términos de $macro-F_1$ (%) al utilizar técnicas de aumentado de datos. El mejor resultado por fila se marca en negrita.

Modelo	Sin inicialización			Con inicialización		
	Mejor tamaño de entrada	Sin aumentado	Con aumentado	Mejor tamaño de entrada	Sin aumentado	Con aumentado
VGG16	128 × 128	54,33	68,57	192 × 192	85,47	76,79
VGG19	128 × 128	44,24	54,52	224 × 224	64,04	77,76
ResNet	224 × 224	67,91	78,89	224 × 224	93,35	93,59
Xception	224 × 224	75,55	84,43	224 × 224	99,52	99,77
InceptionV3	192 × 192	78,91	86,10	224 × 224	99,04	98,76
MobileNet	224 × 224	64,11	82,58	224 × 224	99,29	99,45
SqueezeNet	224 × 224	36,13	47,34	192 × 192	98,36	94,32

4.4.3. Evaluación de la eficiencia

Uno de los retos que nos planteamos inicialmente es el de encontrar una CNN eficiente con la que optimizar el tiempo de ejecución del método propuesto en dispositivos móviles. Desde este punto de vista, nuestra tarea puede verse como un *Problema de Optimización Multiobjetivo* (*Multi-objective Optimization Problem* o MOP), en el que hay que optimizar dos criterios: la precisión y la eficiencia. La métrica F_1 nos permite comparar la precisión de las redes, sin embargo, para evaluar la eficiencia nos basaremos el número de parámetros de la red (ver Tabla 4.1) ponderados por el tamaño de entrada, ya que este también influye en el rendimiento.

La forma habitual de evaluar este tipo de problemas es mediante el concepto de “no dominancia”. Se dice que una solución domina a otra si, y solo si, es mejor o igual en cada función objetivo y, al menos, estrictamente mejor en una de ellas. Por tanto, las mejores soluciones (puede haber más de una) son aquellas que no están dominadas. Las estrategias dentro de este conjunto definen lo que se conoce como frente de Pareto, y pueden

ser consideradas las mejores sin tener que definir ningún orden entre ellas (Miettinen, 1999).

La Figura 4.5 muestra los resultados de este análisis MOP, donde el eje vertical representa la métrica F_1 y el eje horizontal el número de parámetros en porcentaje, considerando como máximo la red con más parámetros (ResNet) ponderada por el tamaño de imagen más grande (224×224 px). En este gráfico, el método de entrenamiento se representa mediante la forma y el color de los puntos, resaltando además los elementos “no dominados”. Las redes que optimizan ambos objetivos son SqueezeNet con tamaños de entrada de 128×128 px y 192×192 px, y MobileNet y Xception, con un tamaño de entrada de 224×224 px y en todos los casos aplicando aumentado de datos.

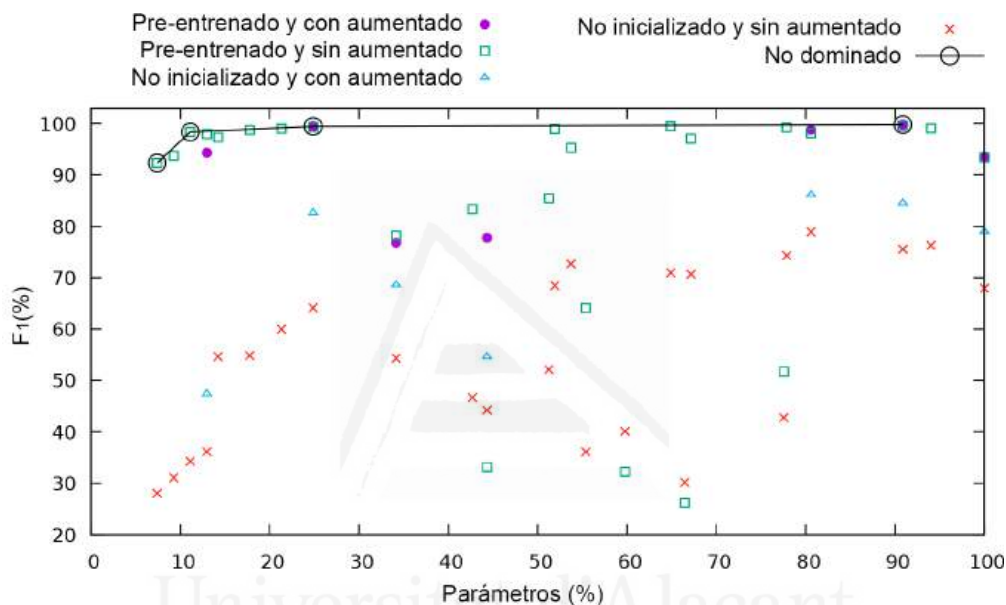


Figura 4.5.: Análisis de F_1 vs. eficiencia como un problema MOP. Los elementos no dominados se destacan mediante círculos.

4.4.4. Evaluación de la posición

Según la metodología propuesta (ver Figura 4.2), una vez clasificada la imagen, en caso de detectar uno de los gestos buscados, procedemos a calcular su posición usando el enfoque Grad-CAM (Selvaraju et al., 2016). Como se ha explicado, mediante esta técnica obtenemos un mapa de calor que representa las regiones que más han influido en la predicción. En la Figura 4.6 se incluyen varios ejemplos de los mapas obtenidos usando las mejores topologías de red determinadas en la sección anterior (Xception y MobileNet, ambas con aumento de datos y con tamaño de entrada de 224×224 px). Como se puede ver, las zonas marcadas pertenecen a la región de la imagen donde aparece

el gesto, mostrando además una buena precisión en la posición detectada.

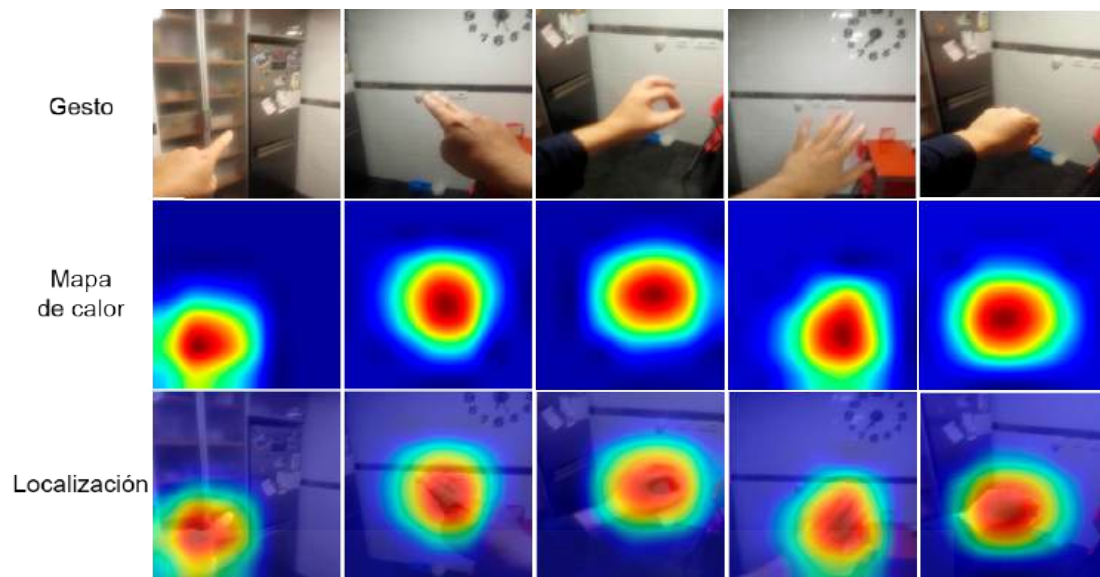


Figura 4.6.: Mapas de calor calculados usando Grad-CAM (Selvaraju et al., 2016) en los que se destacan las regiones detectadas. Las clases localizadas de izquierda a derecha son: “un dedo”, “dos dedos”, “lupa” y dos ejemplos de “otros gestos”.

Para evaluar la precisión de la posición calculada mediante este método usaremos la métrica IoU descrita en la Sección 4.4.1. Para ello, calculamos el IoU entre el cuadro delimitador de la predicción realizada (tomando para esto la zona de máxima activación del mapa de calor, esto es, la que aparece en rojo) y el cuadro delimitador etiquetado en el GT. La Tabla 4.6 muestra los resultados de este experimento, donde la columna IoU representa el promedio obtenido para esta métrica y la columna Acc (*Accuracy*) el porcentaje de detecciones correctas (TP) usando un umbral $\lambda = 0,5$ (que es el valor comúnmente utilizado con esta métrica). Como puede observarse, el método propuesto obtiene muy buen resultado con todos los gestos, obteniendo una precisión media del 98 %. Además, si analizamos el 2 % de errores cometidos, vemos que la desviación en la posición detectada no es muy grande, ya que si disminuimos el umbral λ solo en un 4 %, entonces el algoritmo detecta correctamente el 100 % de los casos.

Tabla 4.6.: Evaluación (en términos de IoU) de la localización obtenida.

Categoría	Un dedo	Dos dedos	Lupa	Otros gestos	Media
IoU	0,8234	0,8343	0,8254	0,8605	0,8359
Acc (%)	96	98	98	100	98

4.5. Conclusiones

En este capítulo se ha presentado una primera propuesta metodológica para abordar el problema principal de esta tesis doctoral: la clasificación y localización de gestos de las manos en imágenes. En concreto, se han realizado las siguientes aportaciones:

- Para evaluar la efectividad del método propuesto hemos elaborado manualmente un conjunto de datos de gestos con 20 mil imágenes, lo cual perseguía el objetivo que nos planteamos inicialmente de validar nuestra propuesta con datos reales. Para esto grabamos cinco categorías de gestos sobre diferentes fondos usando varios modelos de teléfonos móviles actuales.
- Se han evaluado diferentes arquitecturas CNN del estado del arte para la tarea de clasificación de los gestos contemplados en el conjunto de datos anterior.
- Se ha aplicado y evaluado sobre dichas redes el enfoque Grad-CAM para la estimación de la localización de los gestos. Esta aproximación tiene la ventaja de no añadir carga de computación, ya que simplemente se necesita post-procesar el resultado ya obtenido por la red.
- Se ha buscado optimizar tanto la precisión como la eficiencia de las soluciones obtenidas, planteando esta búsqueda como un MOP, siguiendo de esta forma nuestro objetivo inicial de estudiar el desarrollo de métodos eficientes para el reconocimiento de gestos.

Respecto a la evaluación de la clasificación, los mejores resultados (teniendo en cuenta la métrica $macro - F_1$) se obtuvieron mediante la topología Xception (99,77 %), seguida de MobileNet (99,45 %) y de Inception v3 (99,04 %). Respecto a la localización, estas redes obtuvieron una media del 98 % de detecciones correctas (utilizando la métrica IoU).

En cuanto a la optimización conjunta de precisión y eficiencia, los resultados “no dominados” (aquellos que optimizan ambos objetivos) fueron obtenidos por SqueezeNet con tamaños de entrada 128×128 px y 192×192 px, y MobileNet y Xception con tamaño de entrada de 224×224 px, y usando en todos los casos la técnica de aumentado de datos.

Aunque se han obtenido buenos resultados de localización, hemos de destacar que el método Grad-CAM no proporciona una alta precisión, ya que las zonas marcadas mediante esta técnica suelen ser grandes. Esto no es un problema si se quiere detectar toda la mano, ya que la precisión proporcionada sería suficiente. Sin embargo, para objetos más pequeños, como es el caso de la detección de la punta de los dedos (para por ejemplo saber dónde se está señalando), necesitamos un método de localización más preciso.

Esto será abordado en el próximo capítulo, en el que propondremos un método de localización de objetos que, además que ser eficiente y de obtener buenos resultados con objetos pequeños, también requiere pocas imágenes para su entrenamiento. Esto último ha sido otro de los problemas detectados en este capítulo, ya que, debido al alto coste que supone (en horas de trabajo) etiquetar grandes conjuntos de datos, hemos recurrido a la grabación de vídeos y al etiquetado de *frames* casi consecutivos. Este otro problema los abordaremos en el Capítulo 6, en el cual, además de elaborar un conjunto de datos con más variedad de imágenes, escenas y tipos de gestos, también abordaremos la tarea final de integrar toda la metodología, de modo que una vez clasificado un gesto no solo se detecte su posición sino que se realicen las acciones correspondientes al mismo (como podría ser describir la escena o el objeto apuntado).



Universitat d'Alacant
Universidad de Alicante

CAPÍTULO 5

SELECCIÓN DE FILTROS PARA LA DETECCIÓN DE OBJETOS

Resumen

El presente capítulo propone una mejora sobre el método de detección presentado en el capítulo previo. En concreto, se introduce una novedosa metodología basada en la selección de filtros para la localización precisa de objetos pequeños. El método parte de una red neuronal convolucional entrenada para la clasificación categórica y la analiza para seleccionar el subconjunto de los filtros que maximice la precisión de detección de la clase objetivo. Posteriormente dichos filtros permitirán utilizar la misma red categórica tanto para la tarea de clasificación como para la de localización de los objetos de interés.

Contenidos

5.1	Introducción	60
5.2	Método	61
5.3	Datasets	66
5.4	Experimentos	69
5.5	Conclusiones	80

5.1. Introducción

En este capítulo presentamos un método de aprendizaje débilmente supervisado para la localización eficiente y precisa de objetos de tamaño reducido. En el capítulo anterior se propuso una aproximación inicial para abordar los objetivos de clasificación y localización de gestos, concluyendo que, aunque presentaba buenos resultados al estimar la posición de la mano, no proporcionaba suficiente precisión para detectar objetos pequeños, como podría ser la punta de los dedos.

En el presente capítulo se propone una mejora sobre el método de localización previo que también se enfoca en crear un modelo eficiente basado en el aprovechamiento de una CNN entrenada para la clasificación. En concreto, a partir de esta red, se analizan los filtros aprendidos y se seleccionan aquellos que permitan detectar con mayor precisión los objetivos. Una vez realizado este proceso, el método calcula el gradiente entre las activaciones de cada uno de estos filtros y la imagen de entrada y, a partir de la combinación de dichos gradientes, proporciona el resultado final con la localización.

Para entrenar y evaluar métodos de localización de este tipo es fundamental contar con un conjunto de datos con cientos (sino miles) de ejemplos en los que los objetos a detectar estén etiquetados mediante cuadros delimitadores. Etiquetar grandes conjuntos de datos con la posición de todos los elementos conlleva una gran cantidad de trabajo, siendo una tarea que además es propensa a errores. Es por ello que, siguiendo el objetivo planteado inicialmente de buscar soluciones que requieran pocos datos etiquetados para el entrenamiento, en el presente capítulo estudiaremos el uso de una metodología basada en un enfoque débilmente supervisado que nos ayude a reducir este requisito de etiquetado.

Aunque nuestro objetivo final es la localización de gestos de la mano, en realidad, la técnica propuesta puede aplicarse a cualquier conjunto de datos. De este modo, y con el propósito de validar el método antes de aplicarlo a la tarea de localización de gestos, en este capítulo se ha optado por basar las pruebas iniciales en el conjunto de datos MASATI (*MARitime SATellite Imagery dataset*) (Gallego et al., 2018b). Este *dataset* contiene miles de imágenes de satélite con la localización precisa de barcos, los cuales representan únicamente unos pocos píxeles de la imagen. Se trata por tanto de un problema de localización de objetos pequeños especialmente complejo con el que poner a prueba el método.

Tras estudiar la propuesta para el cálculo de la localización con este *dataset*, en el siguiente capítulo se aplicará a un nuevo conjunto de datos de gestos. De este modo, será posible realizar la detección precisa tanto de las manos como de la punta de los dedos. Además, este método también se integrará con el resto de funciones, añadiendo componentes para el procesamiento de otros gestos y para la realización de las acciones correspondientes.

En la siguiente sección se detalla el método propuesto, posteriormente describiremos el *dataset* utilizado en este capítulo para la evaluación, y, por último, se analizará la eficacia de la técnica propuesta comparándola con otra serie de aproximaciones del estado del arte.

5.2. Método

Los métodos de localización basados en aprendizaje débilmente supervisado nos permiten detectar las regiones de la imagen de entrada que resultaron más relevantes para clasificar una determinada clase, como sería el caso de la técnica Grad-CAM estudiada en el capítulo anterior. Sin embargo, los filtros aprendidos por una CNN pueden considerar más elementos aparte del objeto buscado, algunos de los cuales no tienen porque ser adecuados para su localización. Esto puede suceder en los casos en los que un mismo objeto se clasifica en distintas categorías. Por ejemplo, en nuestro caso, al clasificar los gestos con uno o más dedos extendidos o, en el caso del conjunto de datos MASATI, para la detección de barcos cuando además aparece (o no) una parte de costa en la imagen, ya que el método se fijará también en esta zona de tierra para distinguir entre categorías.

En la Figura 5.1 se incluyen algunos ejemplos que ilustran este problema para el caso del conjunto de datos MASATI. La primera fila muestra las imágenes de entrada, en la que los barcos a localizar se marcan mediante cuadros delimitadores. En la segunda y tercera fila se pueden ver los mapas de prominencia (*saliency maps*) obtenidos al aplicar las técnicas de Grad-CAM (Selvaraju et al., 2016) y de retropropagación (*backpropagation* o BP) (Simonyan et al., 2013), respectivamente. En las columnas (a) y (e) se aprecia claramente cómo la atención de la red se centra en otras localizaciones distintas a la del barco. También se aprecia cómo la precisión de la localización puede ser muy pobre en los casos en los que se obtienen valores de activación altos para una zona amplia de la imagen (ver Figuras 5.1 (b), (c) y (d)).

Esto ocurre porque, como se revisó en el Capítulo 2, las CNN aprenden una jerarquía de características en las diferentes capas convolucionales: desde las características de bajo nivel (como bordes, esquinas, etc.) hasta características de alto nivel en las últimas capas convolucionales (que suelen ser las empleadas para calcular los mapas de calor o de prominencia). Sin embargo, según la argumentación previa, estos filtros podrían activarse ante diferentes elementos de la imagen para realizar la clasificación. Esto quiere decir que para la clasificación se activan algunos filtros que no necesariamente contienen el objeto buscado, sino que también pueden contener otros elementos de la imagen que ayuden a realizar la clasificación.

La Figura 5.2 muestra un subconjunto aleatorio de los mapas de características obtenidos en diferentes filtros de una CNN al procesar una imagen de la base de datos MASATI

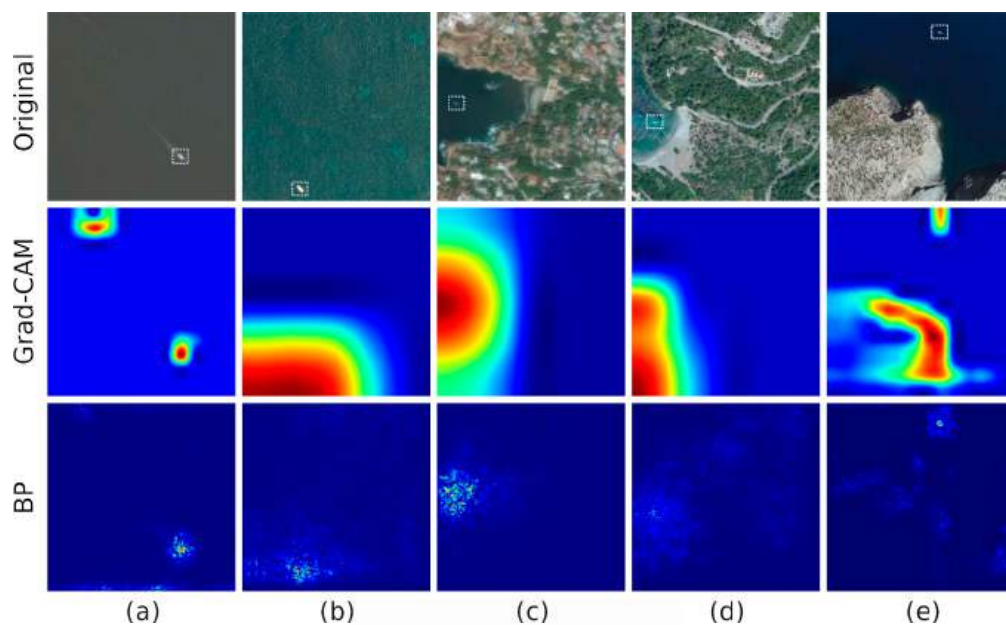


Figura 5.1.: Ejemplos de mapas de prominencia (*saliency maps*) obtenidos mediante Grad-CAM y retropropagación (BP). La primera fila muestra las imágenes de entrada en las que los barcos están marcados con un cuadro delimitador.

en la que aparece un barco en una zona de costa. Como puede verse, la mayoría de los filtros no presentan activaciones que marquen la localización del barco, y, entre los filtros que sí que lo detectan, puede verse como también se fijan en otros elementos de la imagen.

5.2.1. Esquema del método propuesto

En base a esta motivación, el objetivo de la solución propuesta en este capítulo es utilizar solo aquellos filtros de la CNN que presenten valores de activación altos para el elemento buscado. La Figura 5.3 muestra el esquema del método, donde, igual que en el capítulo anterior, el primer paso es el entrenamiento de una CNN para la clasificación de las distintas clases. Una vez que se han aprendido los pesos de dicha red, se realiza un nuevo proceso de *selección de filtros* (*Filter Selection* o FS) para determinar el subconjunto de filtros (de entre todos los filtros de la red) que maximizan la precisión en cuanto a la ubicación del objeto buscado. Finalmente, en la etapa de inferencia, las nuevas imágenes se clasifican mediante la CNN entrenada, y, si la clase predicha se corresponde con la clase objetivo, se utilizan los filtros seleccionados para calcular la posición del objeto buscado.

Los pasos 1 (Entrenar CNN) y 2 (Ajustar FS) del esquema de la Figura 5.3 se realizan

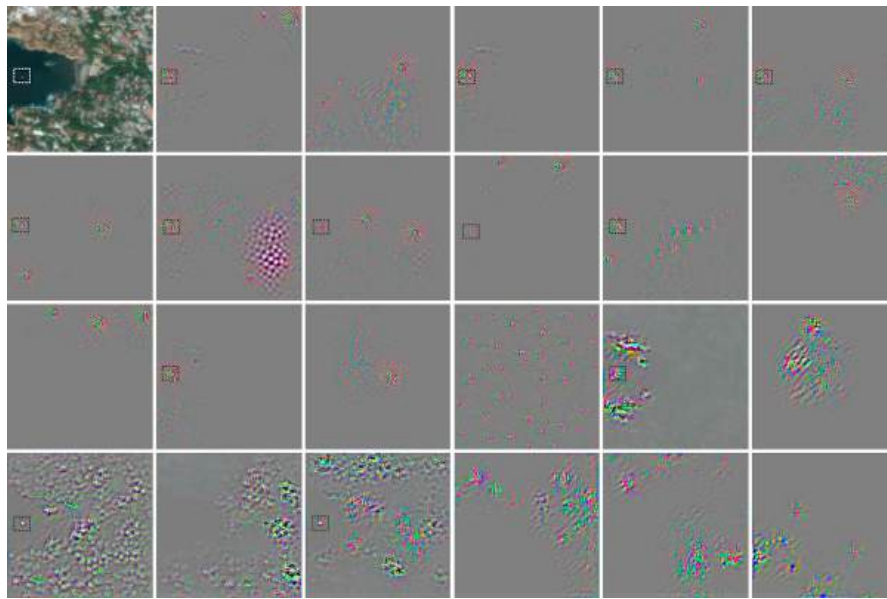


Figura 5.2.: Ejemplo de las activaciones obtenidas al clasificar una imagen de costa en la que aparece un barco. La figura superior izquierda es la muestra usada como entrada y las otras imágenes son las activaciones obtenidas para un subconjunto aleatorio de filtros de la última capa convolucional del modelo VGG-16 (Simonyan and Zisserman, 2014). El barco está marcado con un cuadro delimitador solo si la activación lo detectó correctamente.

durante la etapa de entrenamiento del método, mientras que el paso 3 se refiere a la etapa de inferencia. En las siguientes secciones se detallan cada uno de los pasos en los que se divide el método propuesto.

5.2.2. Paso 1: Entrenar la CNN

En este primer paso se entrena la CNN para la tarea de clasificación, para esto se sigue el mismo proceso explicado en la Sección 4.2.1 del capítulo anterior: 1) se modifican las últimas capas de la red para adaptarlas a nuestra tarea, 2) se inicializa la red con los pesos obtenidos con el conjunto de datos ILSVRC y 3) se realiza un proceso de ajuste fino (*fine-tuning*) con nuestro conjunto de datos.

En la experimentación que realizaremos en este capítulo evaluaremos el uso de la metodología propuesta en dos topologías CNN ampliamente conocidas, como son VGG-16 y VGG-19 (Simonyan and Zisserman, 2014). En el Apéndice D se puede consultar más información sobre estas redes.

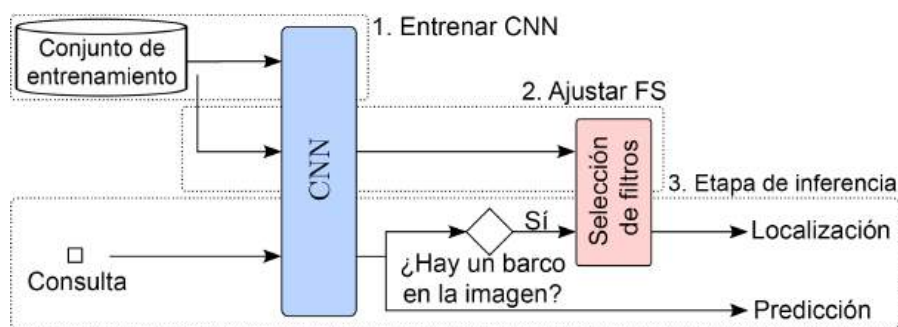


Figura 5.3.: Esquema del método propuesto.

5.2.3. Paso 2: Selección de filtros

Una vez entrenada la CNN procedemos a aplicar el algoritmo de selección de filtros. Este proceso consiste en seleccionar los filtros de la red que proporcionan con mayor precisión la localización de una determinada clase objetivo. Para esto analizaremos los filtros $\mathcal{F}$ de la red y nos quedamos con el subconjunto $\mathcal{F}^c \subseteq \mathcal{F}$ que detecta la clase c con una precisión mayor a un determinado umbral α .

Para obtener el subconjunto $\mathcal{F}^c$, primero calculamos los resultados de localización de cada filtro $f \in \mathcal{F}$ de manera individual. Para ello estimamos el conjunto de predicciones $P_{l,f}^{(i)}$ para una imagen de entrada i y un filtro f de la capa l de la siguiente manera:

$$P_{l,f}^{(i)} = \text{Blobs}((\tilde{G}_{l,f}^{(i)} > \beta) \oplus s) \quad (5.1)$$

donde el conjunto $\tilde{G}_{l,f}^{(i)}$ representa los gradientes normalizados en el rango $[0, 1]$ obtenidos por el filtro f de la capa l (ver Ecuación 5.2). De estos gradientes solo se seleccionan los valores que estén por encima de un umbral β , obteniendo así una matriz binaria del mismo tamaño $\mathbb{R}^{(w \times h)} \rightarrow [0, 1]^{(w \times h)}$, donde w y h son el ancho y el alto de la imagen de entrada, respectivamente. Posteriormente aplicamos una operación morfológica de dilatación (denotada por $\oplus$) usando un elemento estructurante s . Esta dilatación está destinada a cerrar pequeños huecos y aumentar el tamaño de las detecciones después de aplicar el umbral. Finalmente, la función Blobs calcula los grupos de píxeles conectados (o *blobs*), devolviendo una lista de cuadros delimitadores marcando los *blobs* detectados.

Los gradientes $G_{l,f}^{(i)}$ del filtro f de la capa l con respecto a una imagen de entrada i se obtienen realizando una única pasada de retropropagación. Para esto se calcula la derivada parcial del mapa de activación $A_{l,f}^{(i)}$ obtenido para el filtro f con respecto al espacio de la imagen de entrada I y evaluado en la imagen $I^{(i)}$. Los gradientes obtenidos

se escalan en el rango $[0, 1]$ usando la función r , lo cual nos devuelve el conjunto $\tilde{G}$ de gradientes normalizados:

$$\tilde{G}_{l,f}^{(i)} = r \left(\frac{\partial A_{l,f}^{(i)}}{\partial I} \Big|_{I^{(i)}} \right) \quad (5.2)$$

Una vez obtenido el conjunto de predicciones $P_{l,f}$ para cada una de las imágenes de entrada de una determinada clase c (imágenes pertenecientes al conjunto I^c), se procede a calcular el subconjunto de filtros $\mathcal{F}^c$ que se usará para predecir la localización de esa clase en la etapa de inferencia. Para hacer esto calculamos la Intersección sobre Unión (IoU) entre el conjunto de predicciones $P_{l,f}$ y la localización real etiquetada en el *ground-truth* (GT), y seleccionamos los filtros cuyo IoU promedio es mayor que un umbral α para ser utilizados para la localización de la clase c . Formalmente, el subconjunto de filtros $\mathcal{F}^c$ se calcula de la siguiente manera:

$$\mathcal{F}^c = \left\{ f \in \mathcal{F} \mid \frac{1}{|I^c|} \sum_{i=1}^{|I^c|} IoU(P_{l,f}^{(i)}, B_c^{(i)}) > \alpha \right\} \quad (5.3)$$

donde $B_c^{(i)}$ son las localizaciones etiquetadas (GT) para la imagen i y la clase c , y $|I^c|$ representa la cardinalidad del conjunto I^c con las imágenes de la clase c .

Para calcular el IoU de las predicciones obtenidas para una imagen de entrada i , debemos tener en cuenta que tanto los conjuntos de predicciones $P_{l,f}^{(i)}$ como el GT $B_c^{(i)}$ pueden contener varios cuadros delimitadores. Cada cuadro delimitador predicho del conjunto $P_{l,f}^{(i)}$ se asignará al cuadro del GT ($B_c^{(i)}$) con el que tenga un valor de IoU mayor. Se considerará que una detección es positiva si la relación de superposición de área entre el cuadro delimitador predicho y el cuadro delimitador asignado del GT excede un cierto umbral λ (normalmente se considera buena predicción si la área es mayor o igual que $\lambda = 50\%$), siguiendo la Ecuación 5.4:

$$IoU(P_{l,f}^{(i)}, B_c^{(i)}) = \frac{\text{area}(P_{l,f}^{(i)} \cap B_c^{(i)})}{\text{area}(P_{l,f}^{(i)} \cup B_c^{(i)})} \quad (5.4)$$

donde $\text{area}(P_{l,f}^{(i)} \cap B_c^{(i)})$ denota la intersección entre el cuadro delimitador predicho y su cuadro asignado del GT, y $\text{area}(P_{l,f}^{(i)} \cup B_c^{(i)})$ denota la unión entre ambos.

Una vez realizada esta etapa, el subconjunto de filtros $\mathcal{F}_c$ seleccionados para cada clase objetivo c se almacena para ser utilizado en la etapa de inferencia, la cual se detalla a continuación. La influencia de los diferentes parámetros de configuración para el método propuesto se evalúa en la Sección 5.4.2.

5.2.4. Paso 3: etapa de inferencia

Una vez completados los pasos 1 y 2 (correspondientes a la etapa de entrenamiento), ya podemos utilizar el método propuesto para calcular la localización de los objetos buscados. En esta etapa de inferencia (ver Figura 5.3), se procesa una imagen de entrada a mediante el modelo CNN entrenado y, si la predicción para cualquiera de las clases objetivo es positiva (por ejemplo, si se detecta un gesto de la mano o, en el caso de MASATI, si se detecta un barco), se utilizan los filtros seleccionados $\mathcal{F}_c$ para esa clase c para obtener la localización del objeto buscado.

Esto se hace siguiendo los mismos pasos que en la Ecuación 5.1, pero realizando la suma de los gradientes obtenidos para el subconjunto filtros $\mathcal{F}_c$. La siguiente función $FS(i, c)$ nos permite calcular la localización de los objetos de una clase dada c para una imagen de entrada i usando el subconjunto de filtros precalculado $\mathcal{F}_c$, esto es:

$$FS(i, c) = \text{Blobs} \left(\left(\left(\frac{1}{|\mathcal{F}_c|} \sum_{f=1}^{|\mathcal{F}_c|} \tilde{G}_f^{(i)} \right) > \beta \right) \oplus s \right) \quad (5.5)$$

Como puede verse, la Ecuación 5.5 es similar a la Ecuación 5.1. Sin embargo, la Ecuación 5.1 calcula la predicción para un solo filtro, mientras que la Ecuación 5.5 realiza la combinación del conjunto de filtros seleccionados $\mathcal{F}_c$.

Conviene observar que durante la etapa de inferencia el enfoque propuesto realiza la clasificación y localización simultáneamente, ya que se basa en las activaciones de los filtros obtenidas al clasificar la imagen, es decir, no es necesario realizar ningún paso adicional de la imagen a través de la red ni utilizar capas adicionales u otro modelo con el fin de calcular la localización.

5.3. Datasets

De la misma forma que en el Capítulo 4, en este estudio también usaremos el conjunto de datos ImageNet (descrito en la Sección 4.3) para inicializar los pesos de las

arquitecturas de red. Posteriormente, para entrenar y evaluar el modelo propuesto, nos basaremos en el *dataset* MASATI que se describe a continuación.

5.3.1. MASATI v1

El conjunto de datos *MAritime SATellite Imagery* (MASATI) fue creado por un grupo de investigadores de la Universidad de Alicante (Gallego et al., 2018b). Contiene 6.212 imágenes de satélite obtenidas de mapas de *Microsoft Bing* clasificadas en siete categorías: “tierra”, “costa”, “mar”, “barco”, “múltiple”, “costa y barco” y “detalle”. Las clases “barco” y “costa & barco” contienen imágenes en las que aparece un solo barco dentro de la imagen, en el segundo caso incluyendo además parte de costa. La clase “multi” describe imágenes con dos o más instancias de barcos, en algunos casos incluyendo costa también. Los barcos de estas clases tienen longitudes de entre 4 y 10 píxeles, con un área de entre 6 y 154 píxeles. La clase “detalle” contiene imágenes con un solo barco con una longitud de entre 20 y 100 píxeles. La función de esta última clase es la de ayudar a mejorar la precisión de los modelos durante el proceso del entrenamiento.

Las imágenes fueron capturadas en RGB y con diferentes tamaños, ya que esto dependía de la región de interés a registrar. En general, el tamaño de imagen promedio tiene una resolución de alrededor de 512×512 píxeles. El conjunto de datos se elaboró con imágenes de satélite obtenidas entre marzo y septiembre de 2016, e incluye diferentes regiones de Europa, África, Asia, el mar Mediterráneo y los océanos Atlántico y Pacífico.

5.3.2. MASATI v2

Para evaluar el método propuesto en este capítulo para la detección precisa de objetos se realizó una extensión del conjunto de datos MASATI. En concreto se añadieron 1.177 muestras para equilibrar el número de prototipos de las diferentes clases, obteniendo un total de 7.389 imágenes. En esta nueva versión también se amplió el etiquetado realizado, para, además de clasificar las imágenes, marcar mediante cuadros delimitadores la localización exacta de los barcos. La nueva versión de este conjunto de datos está disponible públicamente para la comunidad científica en <http://www.iuii.ua.es/datasets/masati>.

La Tabla 5.1 muestra la distribución de muestras por clase. Además, en la Figura 5.4 se pueden ver algunos ejemplos de cada una de estas clases. Como se observa, este *dataset* incluye imágenes muy variadas, en las que las categorías “barco”, “barco & costa” y “multi” contienen ejemplos de barcos muy pequeños y difíciles de detectar. La clase que supone un mayor reto es “multi”, ya que estas imágenes contienen muchos ejemplos de barcos por imagen (en total tiene 1.966 barcos etiquetados, con un promedio de 7 barcos

por imagen, aunque algunas imágenes pueden contener hasta 82 barcos).

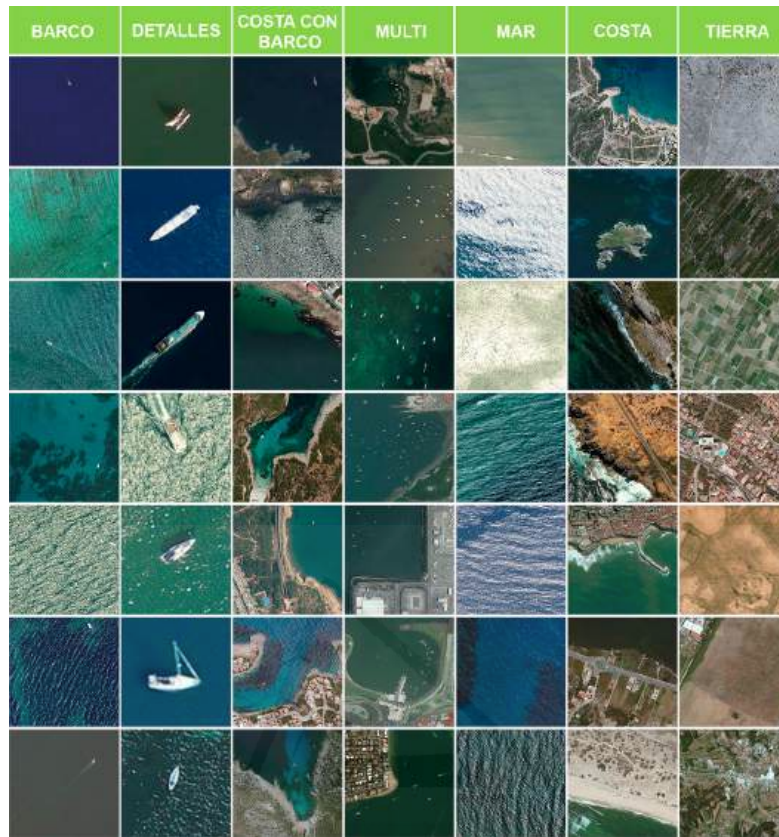


Figura 5.4.: Ejemplos de imágenes de diferentes clases del conjunto de datos MASATI v2. Las cuatro primeras columnas muestran clases con barcos y las tres últimas las categorías en las que no aparecen barcos.

Para la evaluación del método propuesto se crearon además dos conjuntos dentro de este *dataset* que agrupaban las clases según su nivel de dificultad: uno denominado “conjunto complejo” que incluye todas las clases y otro “conjunto simple” que no incluye las muestras de la clase “multi”. Esto nos permitió realizar una mejor evaluación del método propuesto, analizando primero la precisión de la detección para una sola instancia (con el conjunto simple) y luego analizando su comportamiento cuando aparecían múltiples objetos de la misma clase.

Cada uno de estos dos conjuntos se dividió utilizando el 80 % de las muestras para el entrenamiento y el 20 % restante para la evaluación. Se trata de particiones no superpuestas (es decir, el conjunto de prueba no contiene ninguna de las muestras vistas durante el entrenamiento) y se mantuvo el mismo porcentaje de muestras de cada clase en cada partición. Destacar también que se utilizaron las mismas particiones para realizar los experimentos tanto con el método propuesto como con todos los enfoques

Tabla 5.1.: Distribución de las clases del conjunto de datos MASATI v2.

Clase	# muestras	Descripción
Barco (<i>Ship</i>)	1.027	Mar con barco (sin costa)
Detalle (<i>Detail</i>)	1.789	Detalles de barco
Costa & barco (<i>Coast & ship</i>)	1.037	Costa con barco
Multi	304	Múltiples barcos (con y sin costa)
Mar (<i>Sea</i>)	1.022	Mar (sin barcos)
Costa (<i>Coast</i>)	1.132	Costa (sin barcos)
Tierra (<i>Land</i>)	1.078	Tierra (sin mar)

alternativos comparados.

5.4. Experimentos

En esta sección mostramos la experimentación llevada a cabo para las diferentes partes del método propuesto utilizando el conjunto de datos descrito en la Sección 5.3. En primer lugar vamos a evaluar los resultados obtenidos en el paso inicial de clasificación, seguidamente analizaremos el proceso de selección de filtros evaluando diferentes configuraciones de parámetros y, por último, compararemos los resultados obtenidos mediante el enfoque propuesto con otros métodos del estado del arte.

5.4.1. Evaluación de la clasificación

El primer paso del método propuesto consiste en entrenar la CNN para la clasificación del *dataset* considerado. Para esto se sigue el mismo procedimiento explicado en el capítulo anterior, en primer lugar inicializamos la red con ImageNet y seguidamente se realiza fine-tuning con el *dataset*, que en este caso será MASATI v2. Para esta experimentación el entrenamiento se realizó durante un máximo de 500 épocas con un *mini-batch* de 32 muestras y realizando una parada temprana (*early stopping*) cuando la pérdida no disminuía durante 10 épocas.

Tal y como se indicó en la Sección 5.2.2, nos vamos a centrar en las topologías VGG-16 y VGG-19, ya que, además de obtener buenos resultados para la tarea inicial de clasificación, también representan un *baseline* con el que compararse en detección, pues se utilizan frecuentemente como *backbones* para métodos de localización del estado del arte, como SSD (Liu et al., 2016), Faster RCNN (Ren et al., 2015) y CAM (Zhou et al., 2016), entre otros.

La Tabla 5.2 muestra los resultados promedio (usando las métricas Precision, Recall y F_1 definidas en la Sección 4.4.1) para los dos conjuntos considerados (simple y complejo). Como se puede observar, ambas redes obtienen excelentes resultados, cercanos al 100 %, al discriminar entre las diferentes clases del conjunto simple. También se obtienen buenos resultados en el caso del conjunto complejo, aunque son algo inferiores debido a la complejidad de las categorías adicionales. La red VGG-16 obtiene mejores resultados que VGG-19 para el conjunto simple, sin embargo, este comportamiento se invierte para el conjunto complejo. Esta diferencia puede estar motivada por la complejidad de la red y la cantidad de parámetros a aprender, ya que puede realizar un sobreajuste para datos simples, pero ser necesaria para datos más complejos. No obstante, estas diferencias son muy pequeñas.

Tabla 5.2.: Resultados obtenidos con las dos arquitecturas CNN evaluadas para los dos conjuntos de datos considerados (simple y complejo).

Modelo	Dataset	Precision	Recall	F_1
VGG-16	Simple	99,31	99,57	99,44
	Complejo	97,43	96,12	96,77
VGG-19	Simple	99,11	99,02	99,06
	Complejo	97,42	97,39	97,40

5.4.2. Resultados de la selección de filtros

En esta sección evaluamos el proceso de selección de filtros analizando la influencia de los diferentes hiperparámetros. Para simplificar este análisis nos centraremos en el conjunto simple, dado que los resultados obtenidos para el conjunto complejo y las tendencias observadas para los diferentes hiperparámetros fueron bastante similares. Al final de esta sección se reportan también los resultados obtenidos para el conjunto complejo.

Las redes categóricas entrenadas en la sección anterior para el conjunto simple se emplean ahora para analizar la precisión de localización de los barcos mediante el método propuesto. Notar que este conjunto contiene barcos en las clases “barco” y “costa & barco”, por lo que las muestras de ambas se combinan para analizar la precisión de la detección. La clase “multi” pertenece al conjunto complejo y “detalle” se utilizó solo durante el entrenamiento de la red (con el fin de proporcionar más ejemplos de barcos a diferentes escalas). Además, los barcos de esta clase están centrados y ocupan casi toda la imagen, por lo que encontrar su localización resulta muy sencillo.

Los resultados también se evalúan utilizando la métrica F_1 (ver Ecuación 4.3), pero en este caso medimos los objetos (barcos en este caso) cuya localización se detectó correctamente. Esto se hace calculando el cuadro delimitador predicho (P), y comparándolo

con el cuadro delimitador del *ground-truth* (B) con el que tiene un IoU más alto (usando la Ecuación 5.4). Se considera que un cuadro delimitador predicho P está correctamente localizado si $\text{IoU}(P, B) \geq \lambda$. En este caso, establecemos $\lambda = 0,5$ (un umbral comúnmente utilizado en este tipo de tareas, como en PASCAL VOC), y calculamos la métrica F_1 considerando como TP cuando un cuadro P se superpone con un cuadro B con un $\text{IoU} \geq \lambda$, como FP cuando P no se superpone con ningún B , y como FN aquellos casos en los que no se detecta un objeto de B (no hay ningún P que se superponga con él). Es importante señalar que si se obtienen múltiples detecciones para el mismo objeto, solo la primera se cuenta como positiva mientras que el resto se contarán como negativas.

En primer lugar evaluaremos la influencia del tamaño del conjunto de entrenamiento utilizado para seleccionar los filtros. Esto lo haremos mediante la realización de un experimento en el que utilizaremos un conjunto de entrenamiento incremental, empezando con una sola imagen de entrenamiento e incrementando progresivamente las imágenes hasta llegar a 100 (se detuvo el experimento en este tamaño ya que se alcanzó un punto en el que los resultados dejaron de mejorar). Para evaluar la influencia del tamaño de este conjunto se congelaron el resto de parámetros del método, seleccionando la penúltima capa de convolución de cada red, un tamaño $|\mathcal{F}^c| = 4$, $\beta = 0.8$ y un elemento estructurante s de tamaño 7×7 para la dilatación. La Figura 5.5a ilustra el resultado de este experimento. A partir de estos resultados podemos concluir que para ambas arquitecturas (VGG-16 y VGG-19) es suficiente entrenar con entre 14 y 20 imágenes etiquetadas con la posición de los objetos para obtener la mejor precisión. Por lo tanto, finalmente establecemos el tamaño del conjunto de entrenamiento en solo 14 imágenes etiquetadas.

En segundo lugar vamos a evaluar la influencia de la capa l seleccionada en la CNN para extraer los filtros y calcular la localización (ver Ecuación 5.3). La Figura 5.5b muestra los resultados de la experimentación con este parámetro. Este experimento se hizo calculando el resultado obtenido para todas las capas de los dos modelos de red considerados, estableciendo para esto el resto de parámetros del método a los valores anteriormente mencionados. Dado que la red VGG-16 tiene un total de 18 capas y VGG-19 tiene 21, en dicha figura representamos el resultado en función de la profundidad de la capa, donde el 100 % de la profundidad significa la última capa de la red. Como se puede observar, los resultados obtenidos usando la primera parte de ambas redes (hasta un 40 % de profundidad) no fueron buenos. Sin embargo, como era de esperar, se obtuvieron mejores resultados de localización mediante las capas finales (a partir del 70 % de profundidad), ya que contienen características de mayor nivel y por tanto más información para esta detección. Finalmente se seleccionó la capa “*block5_conv2*” para VGG-16 y “*block4_conv3*” para VGG-19 (se puede consultar la arquitectura de red completa en [Simonyan and Zisserman \(2014\)](#)).

Otra variable importante a analizar es el número de filtros seleccionados para obtener la localización, es decir, el tamaño del conjunto $|\mathcal{F}^c|$ en la Ecuación 5.5, el cual puede ser ajustado modificando el valor umbral α (recordemos que dicho umbral se utiliza para

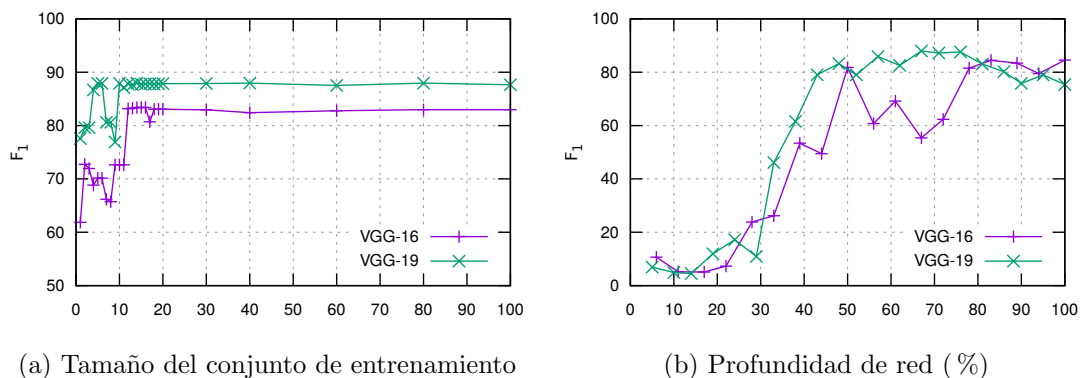


Figura 5.5.: Resultados de localización (F_1 %) obtenidos al variar (a) el tamaño del conjunto de entrenamiento y (b) la capa de la red de la que se seleccionan los filtros (expresada en porcentaje con respecto al 100 % de la profundidad total de la red).

determinar qué filtros se seleccionan). Para este experimento, fijaremos la mejor capa determinada previamente (que, en ambos casos, contiene 512 filtros), un conjunto de entrenamiento con 14 imágenes, $\beta = 0,8$ y un elemento estructurante de 7×7 . Como se puede observar en la Figura 5.6a, para los dos modelos de red se obtienen los mejores resultados al combinar entre 3 y 5 filtros para calcular la localización, siendo 4 la mejor opción en ambos casos. Notar que estos filtros se seleccionaron estableciendo el umbral α en 0,458 para VGG-16 y 0,454 para VGG-19.

Las Figuras 5.6b y 5.6c muestran un histograma del IoU promedio obtenido para cada uno de los filtros en la capa seleccionada de las redes VGG-16 y VGG-19, respectivamente. La coordenada vertical de este gráfico se truncó a un valor máximo de 30 filtros para facilitar su visualización. La primera columna, correspondiente al rango $[0, 0.005]$ de IoU, contiene el 33 % de los filtros de VGG-16 y 6 % de los de VGG-19. Al analizar los resultados de VGG-16, se observa que el 63,87 % de los filtros tiene un IoU inferior a 0,2 y que solo el 13,48 % supera el 0,4, siendo el valor máximo (de 0,4603) obtenido por un único filtro. En el caso de VGG-19, hay menor porcentaje (53,52 %) de filtros que no superan el 0,2 y más filtros que superan el 0,4 (25,00 %), con un valor máximo muy similar de 0,4604.

Otro parámetro a analizar es el tamaño del umbral β (consultar las Ecuaciones 5.1 y 5.5), utilizado para binarizar los gradientes de los mapas de activación obtenidos. De igual manera que en los anteriores experimentos, configuramos el resto de los parámetros a los mejores valores encontrados hasta el momento y variamos solo el parámetro β en el rango $[0, 1]$. En la Figura 5.7a se puede ver cómo se obtienen mejores resultados con valores más altos para este umbral, es decir, al seleccionar solo aquellos píxeles con las activaciones más altas. El valor específico seleccionado para VGG-16 fue $\beta = 0,94$,

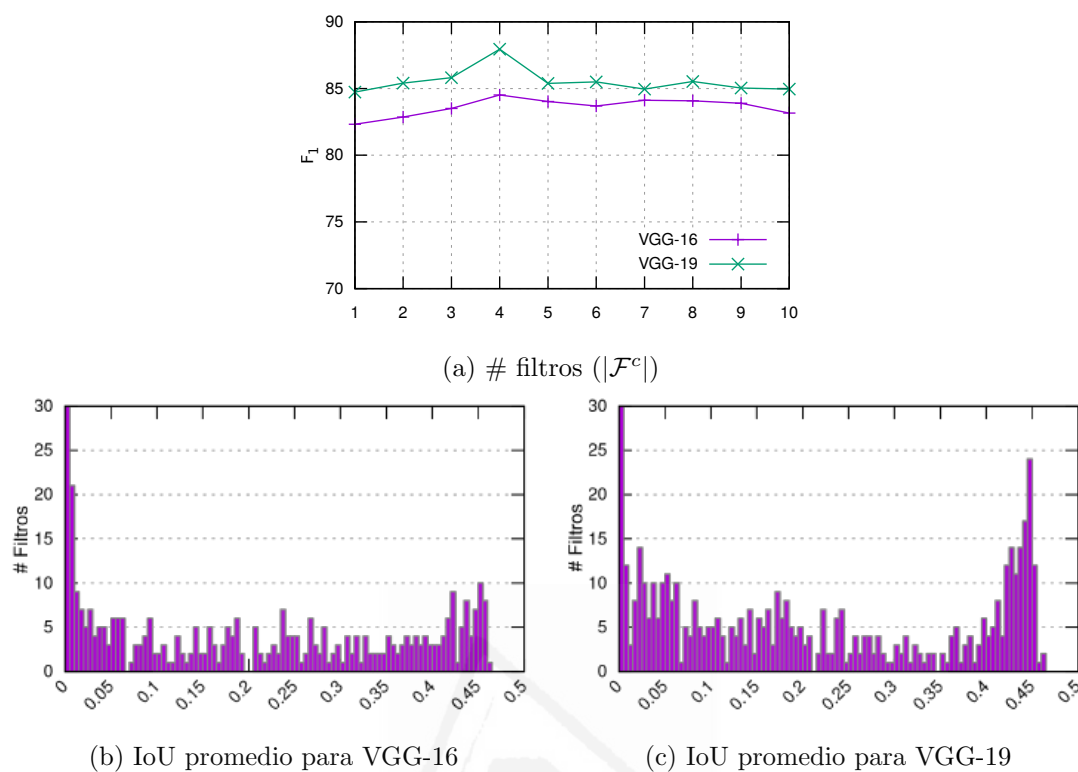


Figura 5.6.: (a) Resultados de localización (F_1 %) obtenidos al variar el número de filtros en el conjunto $|\mathcal{F}^c|$. Las figuras (b) y (c) muestran el histograma con el IoU promedio obtenido por cada uno de los filtros en la capa seleccionada de las redes VGG-16 y VGG-19, respectivamente.

mientras que para VGG-19 fue $\beta = 0,82$.

Finalmente, también analizamos la influencia del tamaño del elemento estructurante s (ver Ecuaciones 5.1 y 5.5) que se utiliza para la dilatación del resultado obtenido por los filtros de activación antes de calcular el cuadro delimitador con la posición de los objetos detectados. La influencia de este parámetro se evaluó variando el tamaño de s entre 3×3 y 13×13 , y estableciendo los parámetros restantes a los mejores encontrados en los experimentos anteriores. Como muestra la Figura 5.7b, el resultado de este experimento permanece bastante estable al variar el parámetro, y mejora solo ligeramente para el tamaño de kernel 7×7 , razón por la cual finalmente seleccionamos este tamaño.

La Tabla 5.3 resume los mejores hiperparámetros encontrados después de realizar la experimentación, tanto para el conjunto simple como para el complejo. Como se puede observar, los resultados obtenidos con cada una de las redes para los dos conjuntos son muy similares. En ambos casos se obtuvo la misma cantidad de imágenes de entrenamiento, la misma capa para extraer los filtros y el mismo tamaño de kernel. Las variaciones

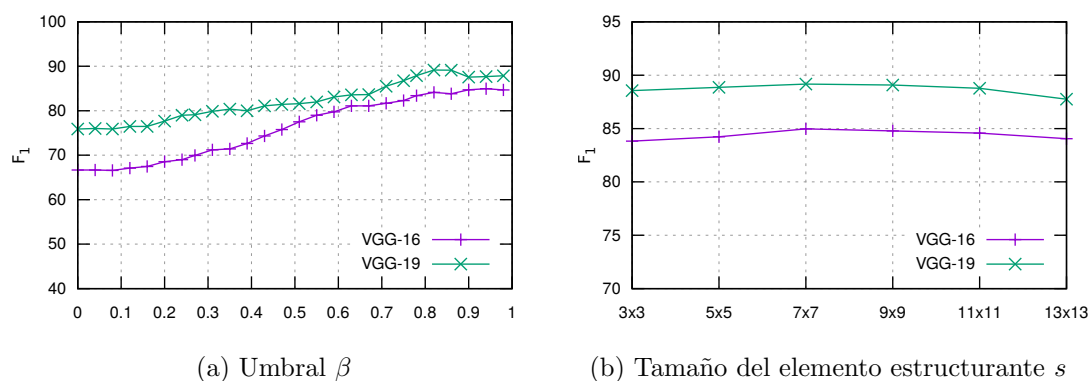


Figura 5.7.: Resultados de localización (F_1 %) al variar (a) el umbral β y (b) el tamaño del elemento estructural s .

ocurren solo con respecto al número de filtros seleccionados y el umbral β . En el caso del conjunto complejo, parece beneficioso combinar más filtros para obtener una detección más precisa. Con respecto al umbral β , un valor alto permite seleccionar solo las detecciones más probables, por lo que cuando se pretende localizar un número reducido de objetivos (como en el conjunto simple) es mejor incrementar este valor. Siguiendo este mismo criterio, en el caso del conjunto complejo es mejor reducir ligeramente este umbral cuando se intentan detectar muchos más objetivos.

Tabla 5.3.: Configuración del método de selección de filtros para cada una de las redes con los conjuntos simple y complejo. El tamaño del conjunto de entrenamiento no es un parámetro del algoritmo pero se evaluó para analizar su influencia en los resultados. Junto al parámetro α también se indica el número de filtros seleccionados entre paréntesis.

Parametro	VGG-16		VGG-19	
	Conj. Simple	Conj. Complejo	Conj. Simple	Conj. Complejo
Tamaño entr.:	14	14	14	14
Capa l :	<i>block5_conv2</i>	<i>block5_conv2</i>	<i>block4_conv3</i>	<i>block4_conv3</i>
Umbral α :	0,458 (4)	0,457 (5)	0,454 (4)	0,453 (5)
Umbral β :	0,94	0,64	0,82	0,61
Kernel s :	7×7	7×7	7×7	7×7

La Figura 5.8 presenta un ejemplo de los filtros seleccionados para una imagen de entrada junto con el proceso de combinar el resultado hasta obtener la predicción final. Este ejemplo se realiza para una imagen de costa con un solo barco (ubicado en la parte superior izquierda) para la que la mayoría de los métodos comparados (como se verá en la siguiente sección) cometen errores. La primera fila de esta figura incluye la imagen de entrada y la salida obtenida, la segunda fila muestra el gradiente obtenido para los cuatro filtros seleccionados, y la última fila presenta el resultado de ir combinando de

forma incremental los filtros anteriores. Un valor de activación más alto se indica con rojo oscuro. Como se puede ver, al utilizar solo el filtro número 35, la predicción realizada es errónea (detecta un segundo barco junto a la costa), pero gracias a la combinación de los cuatro filtros, el algoritmo suprime este error y consigue detectar correctamente la posición del barco.

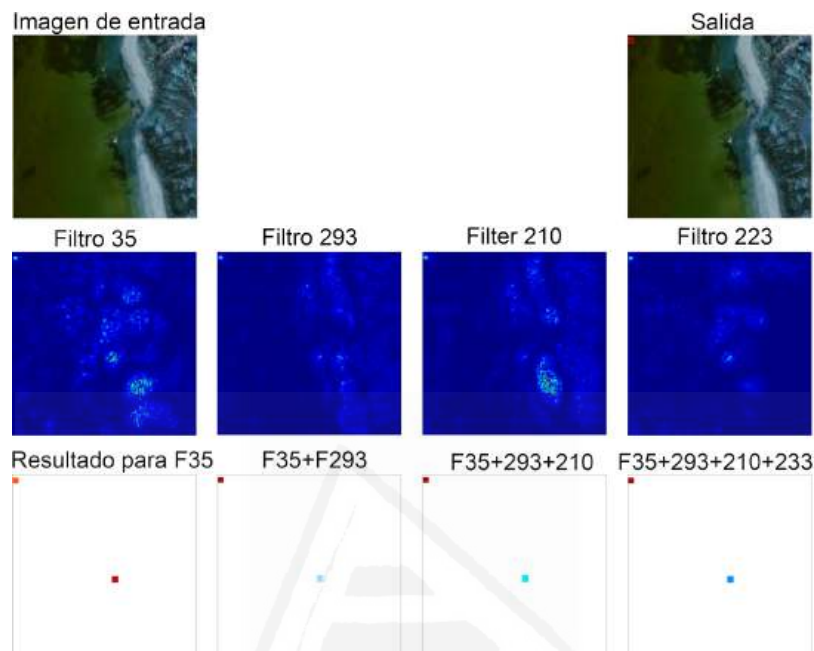


Figura 5.8.: Ejemplo del proceso realizado para calcular la localización de un barco. La primera fila muestra las imágenes de entrada y el resultado de salida, marcando mediante un cuadro delimitador el barco detectado en la parte superior izquierda. La segunda fila muestra los gradientes obtenidos por los cuatro filtros seleccionados para la imagen de entrada. La tercera fila muestra el proceso de combinar incrementalmente el resultado de los filtros. Los colores oscuros indican un valor de activación más alto.

5.4.3. Comparación con otros métodos

Habiendo analizado los diferentes parámetros del método propuesto y determinada la mejor configuración (ver Tabla 5.3), ahora compararemos los resultados obtenidos para los conjuntos simple y complejo con otros métodos alternativos del estado del arte. En particular, compararemos nuestro enfoque con los siguientes métodos (descritos en el Capítulo 3):

- Prominencia visual (*visual saliency*) mediante Retropropagación (BP) (Simonyan et al., 2013) usando los modelos VGG-16 y VGG-19.

- SelAE (Gallego et al., 2018a): Este enfoque utiliza una arquitectura de tipo *Selectional Auto-Encoder* (SAE) especializada en la segmentación de derrames de petróleo. Devuelve una distribución de probabilidad a la que se aplica un umbral para seleccionar los píxeles a segmentar.
- Modelos Faster R-CNN (FRCNN) (Ren et al., 2015) y RetinaNet (Lin et al., 2018), los cuales han mostrado obtener resultados competitivos para la detección de barcos en imágenes SAR Wang et al. (2019). Ambos modelos utilizan una red ResNet50 como *backbone* inicializada con los pesos obtenidos para ILSVRC y aplican aumento de datos durante el entrenamiento. Además, con el objetivo de mejorar la precisión de los métodos para objetos pequeños, se ajustó el tamaño de los anclajes (*anchors*) al tamaño promedio de los cuadros delimitadores de nuestro conjunto de datos.
- Modelos YOLO v2 (Redmon and Farhadi, 2017), YOLO v3 (Redmon and Farhadi, 2018) y YOLT (Etten, 2018). En este caso también inicializamos con los pesos obtenidos para ILSVRC, los entrenamos usando aumento de datos y ajustamos el tamaño de los anclajes. Para la arquitectura YOLT se estableció el parámetro “*min retain prob*” a 0,35, ya que Etten (2018) indica que para valores entre 0,3 y 0,4 se obtienen mejores resultados.

Para esta comparación, además de las métricas utilizadas anteriormente a nivel de detección de objetos (Precision, Recall y F_1), mostraremos también el valor promedio del IoU junto con la Precisión Media (*Average Precision* o AP), puesto que estas métricas son ampliamente utilizadas para evaluar métodos de detección de objetos. En esta comparativa nos basaremos en la métrica AP que se utiliza más recientemente en el resto PASCAL VOC, interpolando todos los puntos en lugar de utilizar un conjunto fijo de valores de *recall* equidistantes (Everingham et al., 2015). Esta métrica calcula el valor medio en el intervalo de *recall* $[0, 1]$, lo cual es equivalente al área bajo la curva (AUC) para la curva *Precision-Recall* (PRC).

La Tabla 5.4 muestra los resultados de la comparación realizada para el conjunto simple con los métodos del estado del arte anteriormente mencionados. Para cada métrica, los dos mejores resultados están marcados en negrita. En general, el método propuesto aparece entre los dos mejores en todas las métricas, a excepción del IoU promedio, aunque esto solo indica que la precisión del área detectada es ligeramente menor, siendo necesario analizar el resto de métricas para contabilizar el número de detecciones correctas. Según la métrica F_1 , los mejores resultados se obtienen con VGG-19+FS (nuestra propuesta) y con YOLO v3, siendo el método propuesto un 1,19 % mejor que YOLO v3. Es importante tener en cuenta que el enfoque propuesto ha sido entrenado usando solo 14 imágenes etiquetadas con la localización, mientras que YOLO v3 se entrenó con el conjunto de datos etiquetado completo. Según la métrica AP, el método propuesto obtiene los mejores resultados para los dos modelos de red con los que se aplica.

Tabla 5.4.: Comparación de los resultados obtenidos para el conjunto simple utilizando el método propuesto (VGG-16/19 + FS) y otros métodos del estado del arte. Estos resultados se calcularon empleando un umbral $\lambda = 0,5$ en la métrica de IoU para considerar una detección como correcta. Los dos mejores resultados por métrica se marcan en negrita.

Método	Precision	Recall	F_1	Avg(IoU)	AP
VGG-16 + BP	69,16	81,92	75,00	62,10	81,54
VGG-19 + BP	64,78	75,00	69,52	59,09	74,79
FRCNN	80,79	63,08	70,84	70,91	52,85
RetinaNet	70,61	61,92	65,98	64,50	51,89
YOLO v2	75,77	66,15	70,64	67,03	57,52
YOLO v3	88,67	87,31	87,98	82,60	85,99
SelAE	68,35	93,85	79,09	63,20	93,06
YOLT	74,44	77,31	75,85	86,98	67,35
VGG-16 + FS	77,12	94,62	84,97	69,91	94,36
VGG-19 + FS	84,01	95,00	89,17	75,50	94,98

Respecto a los resultados obtenidos para el conjunto complejo (ver Tabla 5.5), el método propuesto también obtuvo resultados competitivos. Los mejores resultados según la métrica F_1 se obtuvieron con VGG-19+FS seguido de YOLT y de VGG-16+FS. Los métodos YOLO v2 y v3 no funcionaron tan bien, ya que no están orientados para la detección de múltiples objetivos pequeños, como podría ser el caso de otros enfoques como YOLT o SelAE. Estos últimos fueron los mejores según la métrica AP, aunque el enfoque propuesto obtuvo el tercer mejor resultado muy cercano a estos. Destacar de nuevo que estos métodos se entrenaron con el conjunto completo, etiquetado con la ubicación de todos los barcos, mientras que el método propuesto usó solo 14 imágenes etiquetadas para su entrenamiento.

Seguidamente vamos a estudiar más en profundidad el comportamiento de estos métodos y sus capacidades de generalización. Para esto analizaremos los resultados al procesar imágenes con un número de objetivos diferentes a los vistos durante el entrenamiento y también al considerar un conjunto de datos de entrenamiento reducido. Los resultados de este experimento se muestran en la Tabla 5.6, centrándonos en este caso en las métricas F_1 y AP.

Las primeras columnas de esta tabla muestran los resultados obtenidos al evaluar los diferentes métodos con el conjunto complejo pero utilizando los modelos entrenados con el conjunto simple. En este caso, los métodos que mejor generalizan son SelAE y VGG-19+FS, teniendo este último un valor F_1 solo un 1,6 % inferior al primero. Recordemos que SelAE se entrenó usando todas las imágenes y aplicando aumento de datos, lo

Tabla 5.5.: Comparación de los resultados obtenidos para el conjunto complejo utilizando el método propuesto (VGG-16/19 + FS) y otros métodos del estado del arte. Estos resultados se calcularon empleando un umbral $\lambda = 0,5$ en la métrica de IoU para considerar una detección como correcta. Los dos mejores resultados por métrica se marcan en negrita.

Método	Precision	Recall	F_1	Avg(IoU)	AP
VGG-16 + BP	78,03	72,14	74,97	69,60	71,12
VGG-19 + BP	82,58	66,67	73,78	60,44	66,03
FRCNN	44,92	65,63	53,33	70,59	44,37
RetinaNet	77,55	59,38	67,26	56,38	53,79
YOLO v2	76,72	46,35	57,79	42,79	39,45
YOLO v3	96,36	55,21	70,20	50,29	54,27
SelAE	69,82	92,19	79,46	81,08	85,90
YOLT	77,46	84,11	80,65	95,22	74,67
VGG-16 + FS	79,89	78,65	79,27	70,45	71,52
VGG-19 + FS	93,53	75,26	83,41	67,76	74,18

que significa que esto puede ayudarle a generalizar mejor. En estos resultados también se ve cómo algunos métodos, como YOLO y BP, son muy dependientes del conjunto de entrenamiento, ya que no generalizan bien cuando se procesan imágenes con un mayor número de objetivos. Aunque los objetos y el tipo de imágenes sean los mismos, su resultado empeora hasta un 50 % en el caso de YOLO v3 y un 46 % en el caso de VGG-16+BP.

En las columnas centrales y últimas de esta tabla se evalúan las capacidades de aprendizaje y generalización de todos los métodos. Para esto se realiza un entrenamiento considerando un conjunto reducido de datos, utilizando el mismo número de imágenes que en el método propuesto, es decir, solo 14. Por lo tanto, esto nos permite evaluar todos los métodos por igual y analizar cómo se comportan cuando no se dispone de una gran cantidad de datos de entrenamiento. Como se puede observar, los resultados obtenidos empeoran considerablemente para todos los métodos comparados, disminuyendo entre un 30 % y casi un 70 % en algunos casos. En el caso del conjunto simple (columnas centrales), el método del estado del arte que mejor funciona con pocos datos es YOLO v3 seguido de RetinaNet, y en el caso del conjunto complejo (columnas de la derecha) es YOLT, que obtiene un resultado bastante estable en ambos casos. Sin embargo, si comparamos estos resultados con los del método propuesto, existe una diferencia muy notable, lo que demuestra su capacidad de generalización.

Por último, en la Figura 5.9 se compara de forma visual los resultados obtenidos por los diferentes métodos. Cada columna corresponde a un método y cada fila a una

Tabla 5.6.: Evaluación de las capacidades de generalización de los diferentes métodos analizados. En las primeras columnas comparamos los resultados obtenidos al entrenar con el conjunto simple y evaluar usando el conjunto complejo (con más objetivos). Las columnas centrales y las últimas columnas muestran los resultados obtenidos cuando se entrena con una cantidad reducida de datos pero se evalúa en el conjunto de prueba completo. Los dos mejores resultados por métrica y columna se marcan en negrita. En todos los casos se usa un umbral de $\lambda = 0,5$ en la métrica de IoU para considerar una detección correcta.

Método	<i>Entrenar con el conjunto simple y hacer el test con el conjunto complejo</i>		<i>Entrenar con 14 imágenes del conjunto simple y hacer el test con el conjunto simple</i>		<i>Entrenar con 14 imágenes del conjunto complejo y hacer el test con conjunto complejo.</i>	
	F_1	AP	F_1	AP	F_1	AP
VGG-16 + BP	28,72	16,62	9,99	7,65	3,51	3,89
VGG-19 + BP	27,77	16,02	7,38	7,34	4,75	2,80
FRCNN	59,08	41,61	34,22	20,09	14,36	5,00
RetinaNet	60,58	43,84	51,32	42,78	29,19	16,67
YOLO v2	34,81	20,92	40,60	37,55	16,28	8,30
YOLO v3	37,45	23,05	64,49	50,28	7,50	3,87
SelAE	72,18	56,77	10,90	24,93	32,76	44,29
YOLT	47,91	30,45	48,17	32,26	43,59	27,08
VGG-16 + FS	64,27	44,46	84,97	94,36	79,27	64,52
VGG-19 + FS	70,58	54,49	89,17	94,98	83,41	71,18

imagen de ejemplo diferente. En estos resultados se marcan los cuadros delimitadores de las detecciones obtenidas usando los siguientes colores: verde para TP y rojo para FP. Además se ha agregado un círculo en la esquina inferior izquierda para indicar si la detección fue exitosa (en color verde), si la detección falló (en rojo), o si se detectaron algunos objetivos pero también se obtuvieron falsos positivos (en azul). Como se observa, los métodos más fiables son SelAE, YOLO v3 y VGG-19+FS, ya que detectaron todos los objetivos y arrojaron solo algunos FP. Algunas imágenes que pueden parecer simples, como las que se muestran en las filas 4 y 5, resultan problemáticas para los métodos BP, RetinaNet y YOLT, principalmente debido al pequeño tamaño de los objetos a detectar. Las dos últimas filas muestran ejemplos de la clase “multi” y, como se puede ver, los métodos SelAE, YOLT y VGG-19+FS también obtienen los mejores resultados.

5.5. Conclusiones

En este capítulo se han realizado las siguientes aportaciones:

- Se ha propuesto un método débilmente supervisado para la localización precisa de objetos pequeños en la imagen.
- Se ha creado y publicado una nueva versión del conjunto de datos MASATI para evaluar la precisión de la localización en un problema complejo, como es el caso de la detección de objetos pequeños.
- Se ha realizado un análisis y ajuste de los diferentes parámetros del método utilizando el nuevo conjunto de datos MASATI v2.
- Se ha comparado el método propuesto con hasta 8 enfoques diferentes del estado del arte, mostrando resultados competitivos de la metodología propuesta en todos los casos.

El método propuesto pertenece al grupo de métodos débilmente supervisados, ya que solo necesita una pequeña parte del conjunto de entrenamiento (14 muestras en este caso) etiquetadas con la información de la tarea a realizar. El resto del *dataset* es suficiente con que esté etiquetado a nivel categórico, ya que el método primero entrena una CNN para la tarea de clasificación y posteriormente aprovecha los mapas de características aprendidos, seleccionando solo aquellos que resultan adecuados para la tarea de detección. Esto, además, hace que el método resulte muy eficiente, pues se basa en los mismos filtros de la red categórica para estimar la localización, sin necesidad de añadir parámetros ni capas adicionales. Todo esto se encuentra alineado con dos de los objetivos que nos planteamos inicialmente: el desarrollo de métodos eficientes de reconocimiento de gestos y explorar soluciones que requieran pocos datos etiquetados para su entrenamiento.

En el próximo capítulo abordaremos la aplicación de este método al reconocimiento de gestos, donde nos interesa localizar de forma precisa tanto la mano como la punta de los dedos. Por ejemplo, en el gesto de “apuntar” a un objeto, necesitamos determinar la localización de la punta del dedo para identificar el objeto señalado, o, en el gesto de “arrastrar”, para centrar la imagen en la posición indicada. Por este motivo, se estudiará cómo integrar de forma eficiente este método con el resto de funciones necesarias para la aplicación propuesta en esta tesis.

CAPÍTULO 6

SISTEMA PARA EL PROCESAMIENTO EFICIENTE DE GESTOS

Resumen

En este capítulo se propone una arquitectura tipo *multi-head* que aborda el objetivo final de esta tesis, el desarrollo de un sistema interactivo controlado por gestos que sirva de ayuda a personas con discapacidad visual. Esta arquitectura integra módulos basados en los estudios de los capítulos previos, y presenta una solución eficiente, capaz de ejecutarse en dispositivos móviles, que permite controlar una aplicación mediante gestos de tipo estático y dinámico. Cada gesto desencadena una acción diferente, como el reconocimiento de objetos, la descripción de la escena o el escalado de la imagen, quedando todas estas funcionalidades integradas en la misma arquitectura de red neuronal.

Contenidos

6.1	Introducción	83
6.2	Interfaz de la aplicación	84
6.3	Datasets	86
6.4	Método	90
6.5	Experimentación	98
6.6	Conclusiones	111

6.1. Introducción

Nuestro principal objetivo, como se ha mencionado capítulos anteriores, es desarrollar un método de reconocimiento de gestos sobre el cual construir un sistema interactivo de bajo coste que pueda ser ejecutado de forma eficiente en dispositivos móviles. Este método podría ser integrado en aplicaciones como *Supervision for Cardboard* (ver Figura 6.1a) para servir como ayuda a personas con dificultades visuales. En nuestra propuesta, el reconocimiento de gestos se realiza desde una visión egocéntrica (Tekin et al., 2019), ya que el método está destinado a aplicaciones basadas en cámaras RGB ubicadas en el punto de vista del usuario (ver Figura 6.1b). En este sentido, los gestos podrían usarse para desencadenar acciones como identificar el objeto que el usuario está señalando, describir la escena cuando el usuario hace un gesto estático de “lupa” o acercar y alejar la imagen mediante un gesto dinámico de “pellizco”. Para ello, en este capítulo proponemos una arquitectura de red neuronal que integrará tanto los elementos estudiados en capítulos anteriores, para la clasificación y localización de gestos, como las funciones asociadas a cada acción, para así proporcionar una solución completa y eficiente que permita construir un sistema interactivo con las características mencionadas.

El sistema propuesto se basa en una arquitectura de red neuronal tipo *multi-head* que integra funciones de reconocimiento de gestos dinámicos y estáticos, así como funciones de reconocimiento de objetos y de descripción de escenas. Inicialmente, el método detecta y clasifica los gestos y, posteriormente, en función del gesto detectado, realiza una segunda etapa para llevar a cabo la acción correspondiente. Esta arquitectura *multi-head* optimiza los recursos necesarios para realizar distintas tareas sobre una imagen de entrada, aprovechando para esto la información obtenida de un *backbone* común inicial.

Siguiendo una aproximación similar a la propuesta en el Capítulo 4, el sistema primero realiza la clasificación de los gestos y, en función de estos, activa o no la acción correspondiente, lo cual resulta en una solución mucho más eficiente y efectiva que el uso de diferentes redes separadas. De este modo, cada una de las cabezas de la red se dedica a la función asociada a un gesto dado y solo se ejecuta si este gesto es detectado por el *backbone*. En una de estas cabezas se integra el método de selección de filtros propuesto en el capítulo anterior (Capítulo 5) para llevar a cabo la localización de la punta de los dedos extendidos para las clases “Apuntar” y “Arrastrar”. Además, en este capítulo también se introducirán mejoras en este método para acelerar el proceso de localización.

Para entrenar y evaluar el sistema se ha elaborado un nuevo conjunto de datos con unas 44 mil imágenes etiquetadas a varios niveles (categoría de gestos, cuadros delimitadores de gestos y punta de dedos, y descripciones de objetos y escenas). Se ha buscado generar un conjunto de datos variado, incluyendo imágenes de diferentes tipos de gestos, con diferentes fondos de interiores y exteriores, condiciones de iluminación, etc., capturando las imágenes con cámaras de varios teléfonos móviles. Además, se ha creado un conjunto de datos de gestos sintéticos para pre-entrenar el sistema y mejorar así los resultados. La

comparación realizada con métodos del estado del arte muestra resultados competitivos para todas las acciones que realiza el sistema, como son la clasificación y localización de gestos, o la generación de descripciones de objetos y escenas. También se ha comprobado que el sistema se puede ejecutar casi en tiempo real en varios modelos de dispositivo móvil.

A continuación presentamos en detalle el sistema propuesto describiendo la interfaz, los conjuntos de datos generados, el método y la experimentación.

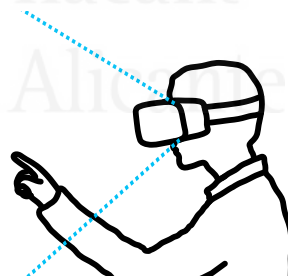
6.2. Interfaz de la aplicación

Como se desarrolló en la introducción y en la revisión del estado del arte, una de las áreas de mejora de las aplicaciones destinadas a ayudar a las personas con discapacidad visual, como *Aerial Obstacle Detection* (Sáez et al., 2015) o *Supervision for Cardboard* (Supervision, 2016), es la interfaz. Su usabilidad podría mejorar de forma notable si se pudiesen manejar de una forma natural e intuitiva, mientras los usuarios se mueven o realizan otras tareas, sin tener que pulsar sobre la pantalla táctil, botones, etc.

Para ello, extendiendo la propuesta inicial que se realizó en el Capítulo 4, definiremos una interfaz que nos permita interactuar con la aplicación mediante el uso de diferentes gestos de la mano. Con este objetivo planteamos un sistema que detecte los gestos desde un punto de vista egocéntrico: los usuarios pueden instalar sus teléfonos móviles en unas Gafas de Realidad Virtual (VRG) o en un dispositivo *Google Cardboard* de bajo coste (ver Figura 6.1a), viendo de este modo el entorno directamente a través de la pantalla del móvil e interactuando con la aplicación a través de gestos (ver Figura 6.1b).



(a) *Google Cardboard*. Fuente de la imagen (Supervision, 2016).



(b) Esquema para ilustrar la interfaz de usuario del sistema propuesto.

Figura 6.1.: *Google Cardboard* y esquema de la interfaz de usuario propuesta.

Para controlar el sistema se propone un conjunto de gestos que extiende el que se

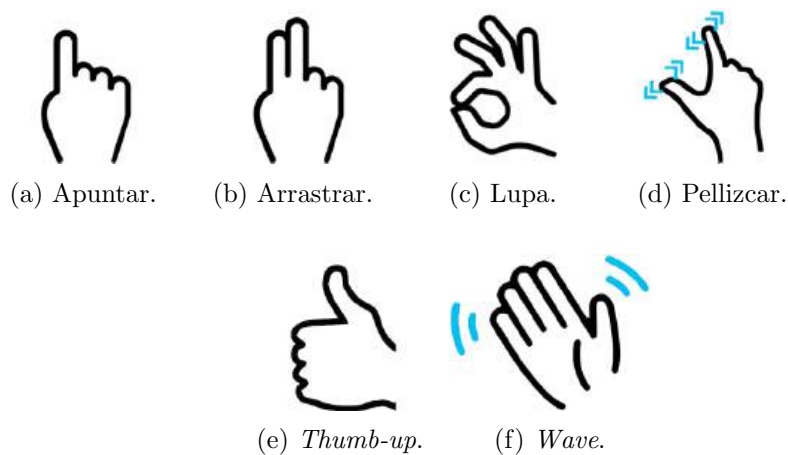


Figura 6.2.: Descripción gráfica de los gestos propuestos. “Apuntar”, “Arrastrar”, “Lupa” y “Pellizcar” son los gestos que se utilizan para interactuar con la interfaz. “*Thumb-up*” y “*Wave*” serán considerados como “Otros gestos” por el sistema. “Apuntar”, “Arrastrar”, “Lupa” y “*Thumb-up*” son ejemplos de gestos estáticos, mientras que “Pellizcar” y “*Wave*” son gestos dinámicos.

planteó en el Capítulo 4: a los gestos estáticos “Apuntar”, “Arrastrar” y “Lupa”, se añade un nuevo gesto dinámico “Pellizcar” (ver Figuras 6.2a–6.2d), orientado este último a ajustar el nivel de zoom. Los cuatro gestos propuestos se describen con mayor detalle a continuación:

- **Apuntar:** Gesto estático formado al extender el dedo índice y flexionar los dedos restantes hacia la palma. Este gesto nos permitirá señalar los objetos de los que queremos obtener una descripción.
- **Arrastrar:** Gesto estático formado al extender tanto el índice como el dedo corazón y flexionando los dedos restantes hacia la palma. Este gesto nos permitirá congelar la imagen y al mismo tiempo realizar un movimiento panorámico de la escena siguiendo las puntas de nuestros dedos. Es útil en combinación con el gesto de *zoom*.
- **Lupa:** Gesto estático formado al unir el pulgar y el índice formando un círculo y tener el resto de los dedos extendidos. Este gesto tiene como objetivo obtener una descripción de la escena y de los objetos que contenga (tarea también denominada como *image captioning*).
- **Pellizcar:** Gesto dinámico formado al mover el pulgar y el índice uno hacia el otro, acercándolos o alejándolos, para realizar una operación de zoom (reducir o ampliar el zoom sobre la imagen, respectivamente). Es equivalente al gesto de pellizcar que se usa habitualmente en las pantallas táctiles.

Este pequeño conjunto de gestos se ha seleccionado para permitir una interacción intuitiva y fácil con el sistema. Sin embargo, nuestra propuesta está diseñada de manera genérica (como se verá en la Sección 6.4), por lo que sería fácil agregar nuevos gestos para expandir su funcionalidad, si fuera necesario.

Es importante señalar que, además de los gestos propuestos, el sistema tiene que identificar si hay o no un gesto presente en la imagen, y tiene que diferenciar estos gestos de cualquier otro gesto posible, tanto estático como dinámico (como los gestos del pulgar hacia arriba o del saludo que se muestran en las Figuras 6.2e–6.2f).

6.3. Datasets

Para entrenar y evaluar tanto la metodología propuesta como los modelos del estado del arte con los que nos compararemos, se elaboraron tres conjuntos de datos¹: uno de imagen real, uno sintético y otro basado en Flickr8k para la tarea de *captioning*. A continuación los describimos en detalle.

6.3.1. Dataset de imagen real

El conjunto de datos presentado en el Capítulo 4, pese a contar con un gran número de muestras, tenía poca variabilidad, ya que se trataba de fotogramas casi consecutivos extraídos de breves secuencias de vídeo. Por lo tanto, para esta propuesta final, para solucionar este problema y poder generar modelos mucho más robustos, se decidió crear un nuevo *dataset* seleccionando una parte del conjunto anterior (aproximadamente un 45 %) con las muestras más variadas y añadiendo otras nuevas para completarlo. Las nuevas muestras se añadieron tanto para las cinco clases previamente contempladas (“Apuntar”, “Arrastrar”, “Lupa”, “Otro” y “Ninguno”), como para la nueva clase “Pellizco”.

Al igual que para el conjunto de datos anterior, partimos de vídeos grabados para cada gesto utilizando cámaras de diferentes teléfonos. Sin embargo, para generar una mayor variabilidad, además de extraer los fotogramas mucho más espaciados, se incluyeron escenas de interiores y exteriores (en el anterior solo habían interiores), con mayor variabilidad de fondos y bajo diferentes condiciones de iluminación. También cabe mencionar que para el nuevo gesto “Pellizco”, al tratarse de un gesto dinámico, se extrajeron un promedio de 10 fotogramas consecutivos para las secuencias de apertura y cierre de los dedos, para así entrenar y evaluar métodos que identifiquen tanto el movimiento de los dedos como su posición.

¹Estos conjuntos de datos están disponibles gratuitamente para la comunidad científica en la siguiente dirección: <https://www.dlsi.ua.es/~jgallego/datasets/gestures>

En total se extrajeron 12,064 fotogramas de los vídeos originales, tratando de equilibrar el número de muestras seleccionadas para cada clase. En la Tabla 6.1 se resume la información sobre este *dataset* (y de los otros dos que veremos a continuación), incluyendo las clases consideradas, el número de muestras disponibles por clase, así como una breve descripción de su contenido. Además, en la Figura 6.3 se pueden ver algunos ejemplos de estas imágenes.

La resolución original utilizada para grabar los vídeos e imágenes fue de $1,920 \times 1,080$ píxeles. Sin embargo, tras realizar una serie de experimentos iniciales sobre el rendimiento y precisión (presentados en el Capítulo 4), y también motivados por las restricciones de algunos de los métodos evaluados, se decidió escalar las imágenes a una resolución de 224×224 píxeles.

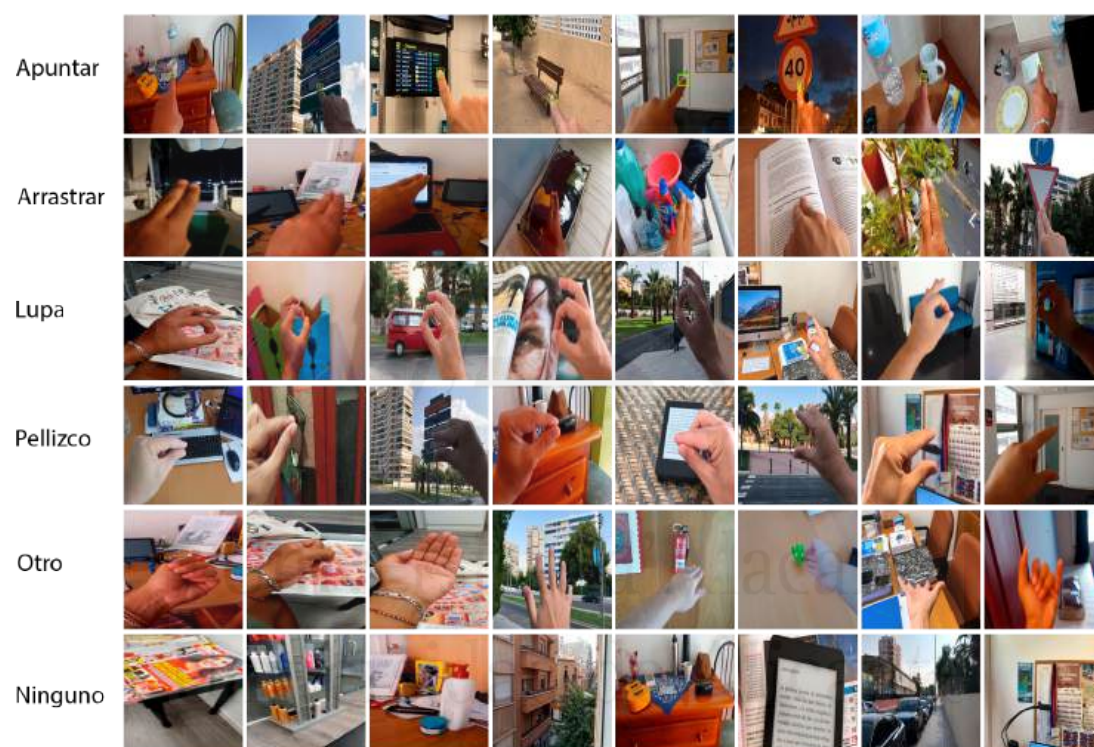


Figura 6.3.: Ejemplos del conjunto de datos con imágenes reales. Las primeras cuatro filas muestran los diferentes gestos propuestos para interactuar con la interfaz. Las dos últimas filas incluyen algunos ejemplos de las clases “Otro” y “Ninguno”.

Las imágenes de este conjunto se etiquetaron usando la herramienta LabelImg descrita en la Apéndice B, indicando tanto la categoría como la posición del gesto. En cuanto a la posición, las coordenadas se anotaron usando cuadros delimitadores para:

- La posición de la mano dentro de la imagen (para todas las clases menos la clase “Ninguno”).
- La posición de las puntas de los dedos.
- Las coordenadas y etiquetas de los objetos a los que apuntamos (para las clases “Apuntar” y “Lupa”).

Tabla 6.1.: Resumen de los conjuntos de datos utilizados, incluyendo el nombre de la clase, el número de muestras por clase y una breve descripción del contenido de cada clase.

Gesto	# Sintético	# Real	# Captioning	Descripción
Apuntar	3,200	2,000	2,000	Imágenes de gestos de señalar con un dedo.
Arrastrar	3,200	2,000	–	Imágenes que incluyen gestos de señalar con dos dedos.
Lupa	3,200	2,000	2,000	Imágenes que incluyen gestos de lupa.
Pellizco	3,200	2,064	–	Secuencias de gestos dinámicos de pellizco.
Otro	3,200	2,000	–	Imágenes de gestos distintos a los cuatro definidos.
Ninguno	3,200	2,000	8,000	Imágenes en las que no aparece ninguna mano.
Total	19,200	12,064	12,000	Gran total: 43,264

6.3.2. Dataset de imagen sintética

Con el objetivo de mejorar los resultados obtenidos se creó un conjunto de datos sintético con el que pre-entrenar los modelos generados. Para esto nos basamos en la herramienta LibHand (una librería de código abierto para generar poses de manos humanas, ver Apéndice B) y la modificamos para habilitar la definición de gestos a través de ficheros de configuración con los rangos de movimiento tolerados para las articulaciones de los dedos². De este modo, nuestra versión modificada de LibHand es capaz de generar variaciones de gestos cambiando de manera aleatoria los ángulos de cada articulación dentro de los rangos definidos. Además, también se modificó para que los gestos se generasen sobre fondo transparente y se superpusiesen sobre imágenes de fondo reales seleccionadas también de manera aleatoria.

²La versión modificada de esta herramienta se puede descargar desde la siguiente dirección: <https://github.com/malozano/libhand>

Mediante esta herramienta generamos un conjunto de datos con un total de 13.200 imágenes a una resolución de 224×224 píxeles (ver Tabla 6.1). Además de variar la forma de los gestos y su posición, también se introdujeron variaciones en el color de la piel y se aplicó desenfoque (de manera aleatoria) para simular el efecto de movimiento. Para los fondos usamos imágenes seleccionadas aleatoriamente de los conjuntos de datos Flickr (Hodosh et al., 2013) y *Visual Object Classes Challenge 2012* (VOC2012)³. En la Figura 6.4 se pueden ver algunos ejemplos de los gestos generados.



Figura 6.4.: Ejemplos de las imágenes incluidas en el conjunto de datos sintético.

Otra modificación importante realizada sobre esta herramienta fue la opción de generar también automáticamente el etiquetado de las imágenes. En este etiquetado se anota tanto la categoría del gesto como la posición de la mano y de la punta de los dedos. En este caso no se etiquetan los objetos señalados, ya que este conjunto de datos se utiliza únicamente para mejorar la precisión de la detección de los gestos y no la del reconocimiento de objetos.

6.3.3. Flickr8k ampliado

Como se explicó en la sección anterior, el gesto de “Lupa” desencadena la acción de mostrar una descripción de la escena que se esté visualizando. Para entrenar y evaluar sistemas capaces de generar estas descripciones, fue necesario utilizar un conjunto de datos adicional que incluyese imágenes con sus correspondientes descripciones. Concretamente, usamos el conjunto de datos Flickr8k (Hodosh et al., 2013), que contiene 8.000 imágenes seleccionadas manualmente del sitio web Flickr con cinco descripciones para

³<http://host.robots.ox.ac.uk/pascal/VOC/voc2012/index.html>

cada una (en la Figura 6.5 se pueden ver algunos ejemplos de este *dataset*). Además se agregó un subconjunto de 4.000 imágenes de nuestro *dataset* de imágenes reales, incluyendo también 5 descripciones por imagen (esto permitió que el sistema propuesto se adaptara a nuestro tipo de entrada, es decir, imágenes de gestos tomadas con teléfonos móviles). Este subconjunto incluye imágenes de los gestos de “Lupa” y “Apuntar” (2000 para cada uno). Aunque la función de *captioning* se propuso inicialmente para el gesto de “Lupa”, se decidió incluir también el gesto de “Apuntar” para aumentar la variabilidad, y poder utilizar la cabeza de *captioning* también para estos gestos (lo cual podría resultar apropiado para algunas aplicaciones).

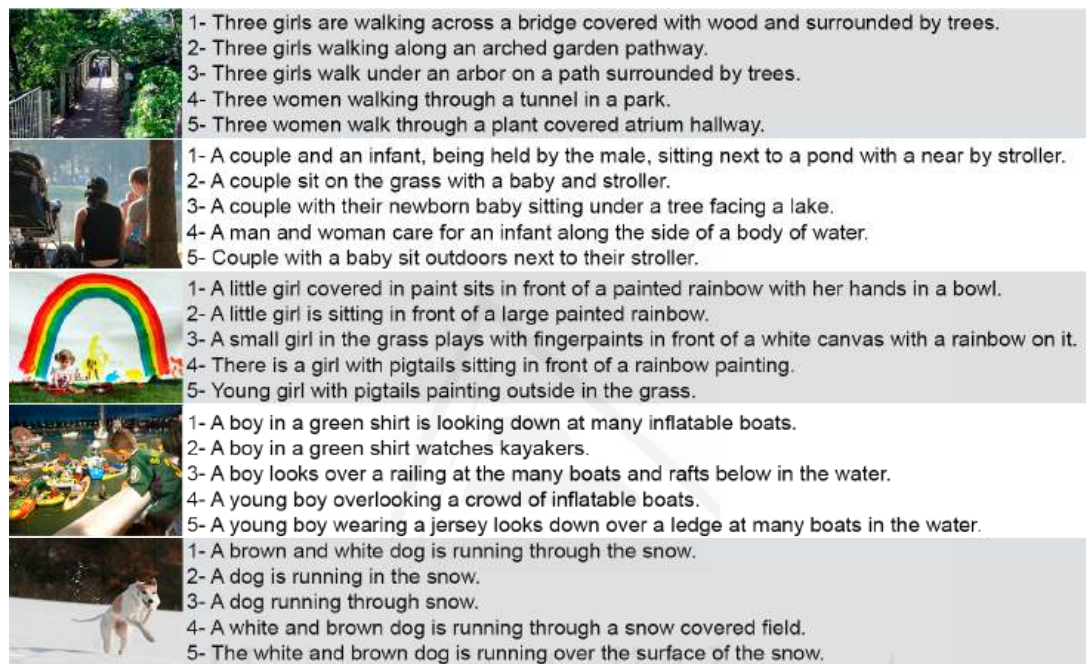


Figura 6.5.: Ejemplos del *dataset* Flickr8k donde se muestra cada imagen anotada con cinco descripciones diferentes.

6.4. Método

El sistema propuesto recibe como entrada una secuencia de fotogramas de vídeo capturada en tiempo real con la cámara de un teléfono móvil y procesa cada uno de estos fotogramas para, en primer lugar, clasificar los gestos que aparezcan en la imagen, y, a partir de dicha clasificación, realizar las acciones pertinentes en función del gesto detectado. Para ello se propone el uso de una arquitectura dividida en dos etapas (como la que se ilustra en la Figura 6.6) en la que para cada imagen se sigue el siguiente proceso:

- **Primera etapa:** Un *backbone* inicial procesa la imagen para extraer un conjunto

de características representativas.

- **Segunda etapa:** En función de las características extraídas en la primera etapa, el gesto se clasifica y se realiza la acción pertinente mediante un módulo o cabeza de red especializada en dicho gesto.

Esta arquitectura tiene como principal ventaja poder realizar múltiples tareas con un reducido número de parámetros y, por lo tanto, con unos menores requisitos *hardware*. Para esto, se aprovechan las mismas características iniciales y, dependiendo del gesto detectado, solo se activa una de las cabezas especializadas de la segunda etapa. Las características comunes extraídas por el *backbone* son inicialmente clasificadas (utilizando la cabeza de clasificación que se puede ver en la Figura 6.6), obteniendo como resultado un vector tipo *one-hot* L -dimensional, siendo L el número de posibles gestos a detectar. El resto de cabezas especializadas se activan o desactivan mediante el uso de una capa tipo *switch* que consulta el valor de la posición correspondiente del vector *one-hot* obtenido.

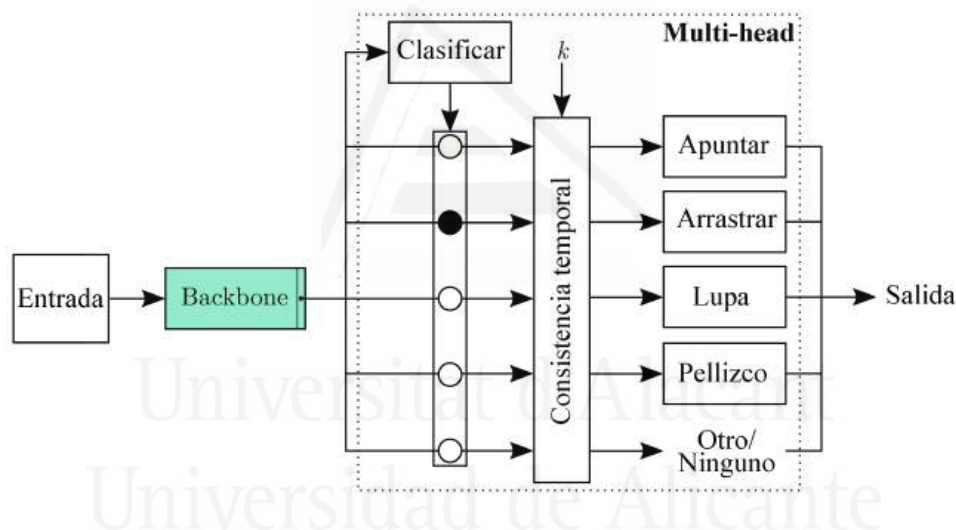


Figura 6.6.: Esquema de la red *multi-head* propuesta.

Para mejorar la estabilidad del sistema se establece un margen de tiempo de k fotogramas para dar la detección de un gesto como válida. Es decir, el sistema procesa los fotogramas del vídeo de entrada y devuelve una respuesta para cada uno, sin embargo, para dar una respuesta consistente para fotogramas consecutivos, deben transcurrir al menos k fotogramas con la misma respuesta para considerar válida la predicción. En las pruebas realizadas bastó con establecer este margen de tiempo en 2 fotogramas, como veremos en el apartado de experimentación.

Como se puede ver en el esquema de la Figura 6.6, esta consistencia temporal se

realiza para todos los posibles gestos, incluyendo las categorías “Otro” y “Ninguno”. Estas dos categorías se unen y al detectarse no se realiza ningún procesamiento adicional, simplemente no devuelven respuesta. De esta forma, cuando se deja de detectar un gesto durante k fotogramas, también se dejará de dar una respuesta.

Las siguientes subsecciones describen en detalle las diferentes partes de este enfoque.

6.4.1. Backbone y clasificación

El primer paso del método es procesar cada uno de los fotogramas de entrada usando un *backbone* con el objetivo de obtener un vector de características común que represente a la imagen procesada. Estas características son posteriormente usadas tanto para realizar la clasificación del gesto como por la cabeza especializada correspondiente. Dado que esta parte es la más importante, pues de su resultado va a depender la eficiencia y eficacia del método propuesto, se compararon un total de 18 aproximaciones diferentes, incluyendo arquitecturas de red como MobileNet (Howard et al., 2017), EfficientNet (Tan and Le, 2019), Xception (Chollet, 2016) y SqueezeNet (Iandola et al., 2016) (en el Apéndice D se puede consultar una descripción de las mismas). En la Sección 6.5.1 se analizarán los resultados obtenidos por cada arquitectura, sin embargo adelantamos que la aproximación que obtuvo los mejores resultados (considerando el compromiso entre precisión y tiempo de ejecución) fue la red Darknet-53 (Redmon and Farhadi, 2018).

Darknet-53 es el *backbone* utilizado por YOLO v3. Se compone de 53 capas convolucionales con conexiones residuales o atajos entre las capas. Esta red es más eficiente y obtiene mejores resultados que sus versiones anteriores y que otros *backbones* similares. Para incluirla en nuestra propuesta se conectó su última capa convolucional con una operación de *Global Average Pooling* (GAP), en la que obtenemos el vector común de características, y, a partir de esta, se conectó con la cabeza de clasificación y con una capa especial tipo *switch*.

Para la cabeza de clasificación solo fue necesario añadir una capa densa de L neuronas con activación tipo SoftMax, para hacer *fine-tuning* a las L clases de nuestro *dataset*, siendo esto una práctica común en las tareas de aprendizaje por transferencia (Chatfield et al., 2014b). Como resultado de esta cabeza se obtiene un vector *one-hot* L -dimensional, donde se marca a 1 el gesto detectado y a 0 el resto. Este resultado se conecta con la capa *switch* para activar únicamente el procesamiento de la cabeza correspondiente al gesto detectado. Seguidamente, y como se puede ver en el esquema de la Figura 6.6, se añade el bloque de consistencia temporal y se conecta con el resto de cabezas especializadas.

La metodología propuesta tiene la ventaja adicional de que permite añadir nuevos gestos fácilmente. Para esto simplemente habría que añadir una nueva cabeza que realice las acciones oportunas, modificar la cabeza de clasificación para añadir una categoría

más y reentrenar el *backbone* para que detecte el nuevo gesto.

6.4.2. Gestos de señalar y arrastrar

Las acciones correspondientes a estos dos gestos comparten una primera parte común: la detección de la punta de los dedos extendidos. Para este proceso se compararon hasta 8 posibles aproximaciones, incluyendo métodos de detección de objetos como YOLO (Redmon and Farhadi, 2018), RetinaNet (Lin et al., 2017) y Faster R-CNN (Ren et al., 2015) (como se verá en la Sección 6.5.2). Igual que antes, adelantamos que finalmente se determinó que el que mejor resultados obtenía, tanto por precisión como por eficiencia, fue el método de *Filter Selection* (FS) descrito en el capítulo anterior.

6.4.2.1. Selección de filtros

Como ya vimos, FS es un método de detección de objetos débilmente supervisado basado en la selección del conjunto de filtros de una CNN categórica que maximizan la precisión de detección de las clases consideradas. Es decir, este método no añade capas extra, sino que aprovecha los filtros ya aprendidos en la red categórica para estimar la localización de los objetos de cada clase, siendo por lo tanto también más eficiente al no requerir parámetros adicionales.

En este capítulo nos basaremos también en esta solución pero introduciendo una modificación para acelerar el proceso de detección. Además, no lo aplicaremos sobre VGG-16/19 sino sobre red categórica descrita en la sección anterior (Darknet-53 + cabeza de clasificación), pre-entrenada mediante ILSVRC (Krizhevsky et al., 2012) y ajustada para la clasificación de las clases de nuestro *dataset* (este proceso de entrenamiento se explicará con detalle en la Sección 6.4.5).

Siguiendo el método propuesto de FS, una vez entrenada esta red categórica se realiza la selección del subconjunto de filtros $\mathcal{F}^c$ para cada clase c , tal como se describió en la Sección 5.2.3, siguiendo la Ecuación 5.3. Pero en este caso, introducimos una mejora en la forma de calcular el conjunto de predicciones $P_{l,f}^{(i)}$, modificando la Ecuación 5.1 por la siguiente:

$$P_{l,f}^{(i)} = \text{Blobs}((r(A_{l,f}^{(i)}) > \beta) \oplus s) \quad (6.1)$$

donde $A_{l,f}^{(i)}$ representa el mapa de activación (también conocido como mapa de características) obtenido para el filtro f de la capa l y la imagen de entrada i . En nuestra

nueva aproximación, a diferencia del método original, no se realiza una pasada de *back-propagation*, sino que se utilizan directamente los mapas de activación, acelerando significativamente por tanto todo el proceso. Las activaciones obtenidas se reescalan en el rango $[0, 1]$ usando la función r . Sobre este resultado, al igual que en el método original, se aplica el umbral β para obtener una matriz binaria, sobre ella se aplica una operación de dilatación morfológica, y finalmente se devuelve la lista de recuadros que delimitan los *blobs* detectados.

Una vez finalizada esta etapa, el subconjunto de filtros $\mathcal{F}_c$ para cada clase objetivo c se almacena para ser utilizado en la etapa de inferencia. La influencia de los diferentes parámetros de este método se evaluará en Sección 6.5.2.

6.4.2.2. Detección de la punta del dedo

En la etapa de inferencia, las imágenes de entrada son procesadas por la red categórica (el *backbone* de la Figura 6.6 + la cabeza de clasificación) y, en el caso de detectarse el gesto de “Apuntar” o de “Arrastrar”, los mapas de activación de la red se utilizan para obtener su localización. Definimos la función $FS(i, c)$ para calcular la localización de los objetivos utilizando el subconjunto de filtros seleccionados $\mathcal{F}^c$ de forma similar a como se describió en la Sección 5.2.4, pero realizando en este caso la suma de las activaciones en lugar de los gradientes:

$$FS(i, c, l) = Blobs\left(\left(\left(\frac{1}{|\mathcal{F}^c|} \sum_{f \in \mathcal{F}^c} r(A_{l,f}^{(i)})\right) > \beta\right) \oplus s\right) \quad (6.2)$$

Para el gesto de “Arrastrar” el sistema solo necesita conocer la posición de la punta de los dedos extendidos, ya que esto es suficiente para calcular el desplazamiento entre fotogramas consecutivos. Sin embargo, para el gesto de “Apuntar”, también es necesario identificar y describir el objeto más cercano a la punta del dedo. Dependiendo de la aplicación final, esto podría ser implementado mediante el método de generación de descripción que se detalla en el siguiente apartado o aplicando FS a las categorías de los objetos que queramos que nuestro sistema identifique. Esto permitiría, por lo tanto, que el sistema indicara la clase del objeto señalado consultando las anotaciones del conjunto de datos ILSVRC (Krizhevsky et al., 2012).

6.4.3. Gesto de lupa

Como se indicó en la Sección 6.2, el gesto de lupa permite obtener una descripción textual de una escena. Para esta acción se añade una cabeza especializada basada en el modelo de generación de *captioning* propuesto por Tanti et al. (2018), denominado como *merge-model*. Esta arquitectura multimodal (ver Figura 6.7) realiza una fusión tardía de la información (las características obtenidas de la imagen y el texto de la descripción). Como indican los autores, los resultados sugieren que las modalidades visuales y lingüísticas para la generación de *captioning* no necesitan ser codificadas conjuntamente por la RNN, ya que eso produce modelos grandes que requieren mucha memoria y pocas ventajas tangibles en el rendimiento; más bien, la integración multimodal debería retrasarse a una etapa posterior. Por tanto, la fusión tardía nos permite utilizar arquitecturas especializadas para cada modalidad, como, en nuestro caso, el *backbone* para procesar las imágenes y una RNN para el texto.

En nuestra implementación, las características obtenidas por el *backbone* para la imagen de entrada se procesan en primer lugar mediante una capa Fully Connected (FC) con 256 neuronas. La salida de esta capa se combina con la obtenida de la parte recurrente utilizada para la información lingüística, formada por una capa de *embedding* y una capa LSTM con 256 neuronas. A partir de las características combinadas se añaden dos capas FC también con 256 neuronas cada una. En todas las capas se utiliza la función de activación ReLU, a excepción de la última capa que utiliza una función de activación Softmax para determinar la siguiente palabra que predice la arquitectura. Durante la inferencia, se pasa primero el *token* de inicio “*startseq*”, generando así una palabra, y a partir de ahí el modelo se llama de forma recursiva con las palabras generadas hasta el momento como entrada, hasta que se obtiene el *token* de finalización “*endseq*” o se alcanza la longitud máxima de descripción.

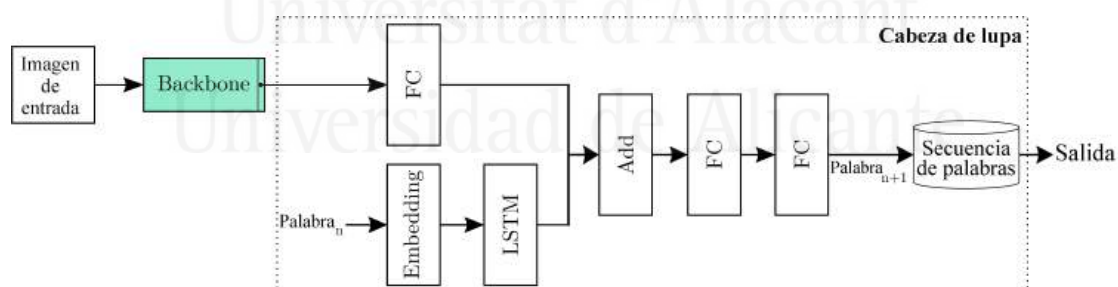


Figura 6.7.: Modelo de *captioning* usado para la cabeza que procesa el gesto de lupa.

Por último, es importante destacar que se realiza un paso de post-procesado sobre las frases generadas, ya que en ocasiones la red genera frases que empiezan por “*A hand/finger is pointing to...*” o “*A hand and...*”. Por este motivo se ha definido un conjunto de reglas básicas que modifican la frase en estos casos para quitar estos textos y, de este modo, generar una frase que solo se refiera a la escena.

6.4.4. Gesto de pellizco

El propósito de este gesto es controlar el nivel de *zoom*. Para ello, cuando este gesto se detecta la imagen se congela y se nos permite aumentar y disminuir el nivel de *zoom* con el movimiento de los dedos. Para controlar esta acción se ha añadido una cabeza especializada (ver Figura 6.8), la cual recibe dos entradas: una con las características extraídas por el *backbone* correspondiente al fotograma t actual, y otra con las correspondientes al fotograma $t - d$. Es decir, no se compara con el fotograma inmediatamente anterior, sino con el obtenido d fotogramas antes. Este histórico es gestionado internamente por la propia cabeza, para lo cual es suficiente con almacenar un *buffer* de d elementos. En nuestro caso, después de una serie de experimentos preliminares, establecemos $d = 5$, para tener una diferencia notable entre los fotogramas que faciliten la detección del movimiento y así mejorar el resultado.

La arquitectura de esta cabeza es muy sencilla (ver Figura 6.8), lo cual a su vez hace que sea un modelo tanto eficiente como eficaz. Las características de las dos entradas (fotograma actual y fotograma $t - d$) se concatenan y se procesan por una capa de convolución (con 64 filtros de tamaño 3×3), y seguidamente se añade una capa de Batch Normalization y otra de Max Pooling para reducir la dimensionalidad. Este resultado se conecta con una capa FC (con 32 neuronas y función de activación ReLU) a través de una operación de *flatten* y, por último, con otra capa FC con activación Softmax para determinar si se está realizando *zoom-in*, *zoom-out* o no se está realizando *zoom*.

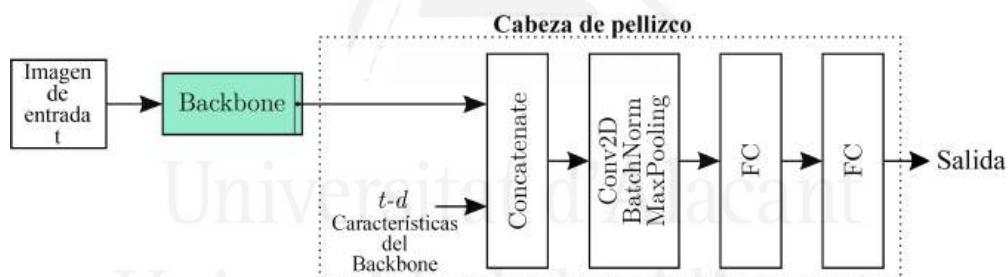


Figura 6.8.: Esquema de la arquitectura propuesta para la cabeza que procesa el gesto de pellizco.

6.4.5. Etapa de entrenamiento

En lugar de entrenar toda la arquitectura en una etapa usando una función de pérdida combinada, se propone un proceso de entrenamiento en dos fases. En la primera fase se entrena el *backbone* para la tarea de clasificación, mientras que en la segunda fase se congelan los pesos obtenidos para el *backbone* y la cabeza de clasificación, y se entrena el resto de cabezas. Este proceso, como se verá en la experimentación, ha demostrado obtener buenos resultados y, además, permitir añadir nuevas cabezas de manera sencilla,

ya que solo sería necesario modificar y reentrenar la cabeza de clasificación (congelando el *backbone*) y después entrenar la nueva cabeza añadida.

Como hemos indicado, en la primera etapa se entrena el *backbone* junto con la cabeza de clasificación. Para esto, se parte de los pesos pre-entrenados con el conjunto de datos ILSVRC (Krizhevsky et al., 2012) y, a partir de esta inicialización, se aplica un proceso de ajuste fino (*fine-tune*) utilizando los conjuntos de datos descritos en la Sección 6.3. Primero se emplea el conjunto de datos sintéticos, después de lo cual, y en función de los pesos obtenidos, se realiza un segundo entrenamiento utilizando el conjunto de datos con imágenes reales.

Para calcular el error se utilizó la función de pérdida categórica de *entropía cruzada* entre cada activación de salida y su activación esperada. El ajuste de los parámetros de la red se realizó mediante retropropagación, usando descenso por gradiente estocástico (Bottou, 2010) y ajustando la tasa de aprendizaje mediante el enfoque adaptativo propuesto por Zeiler (2012b). El entrenamiento se realizó durante un máximo de 200 épocas para cada conjunto de datos (el sintético y el real), con un tamaño de *mini-batch* de 32 muestras y aplicando la *parada temprana* cuando la pérdida no disminuyese durante 10 épocas.

También se empleó aumentado de datos para incrementar artificialmente el tamaño del conjunto de entrenamiento mediante la aplicación aleatoria de diferentes tipos de transformaciones a las imágenes de entrenamiento originales. Esta técnica generalmente mejora el rendimiento y ayuda a reducir el sobreajuste (ver Sección 2.3.3.1). En nuestro caso, para cada imagen del conjunto de entrenamiento se generaron 10 imágenes aumentadas. Las transformaciones aplicadas se seleccionaron aleatoriamente del siguiente conjunto de posibles transformaciones: *flips* horizontales, desplazamientos horizontales y verticales ($[-10, 10]$ % del tamaño de la imagen), *zoom* ($[-10, 10]$ % del tamaño de la imagen original) y rotaciones (en el rango $[-5^\circ, 5^\circ]$).

Para el entrenamiento del resto de cabezas de la arquitectura propuesta se congelaron los pesos del *backbone* y de la cabeza de clasificación, y se ajustaron únicamente los pesos de las capas de cada uno de estos módulos. Para las cabezas de “Apuntar” y “Arrastrar” se aplicó el proceso de FS descrito en la Sección 6.4.2 a partir de los filtros aprendidos por el *backbone*. Para la cabeza encargada del gesto de “Lupa” se realizó el proceso de *fine-tuning* (durante 200 épocas con un *batch-size* de 32) sobre los pesos de la cabeza utilizando el conjunto de datos para *captioning* (Flickr8k ampliado) presentado en la Sección 6.3.

Para la cabeza de “Pellizco” se disponía del etiquetado de las secuencias de vídeo indicando si se estaba haciendo *zoom-in* o *zoom-out*. A partir de esto se extrajeron los fotogramas (como se describe en la Sección 6.3) y se entrenó incluyendo aumentado de datos (también durante 200 épocas con *batch-size* de 32). En este caso se aplicó la

misma transformación a los dos fotogramas comparados, considerando gestos de apertura y cierre a distintas velocidades (variando en ± 1 fotogramas el valor de d) e incluyendo gestos estáticos (usando el mismo fotograma o comparando con el fotograma anterior o posterior).

Los diferentes conjuntos de datos se dividieron en dos particiones, una para entrenamiento con el 80 % de las muestras y otra para *test* con las muestras restantes. Además, una pequeña parte del subconjunto de entrenamiento (10 %) se usó como partición de validación para ajustar los hiperparámetros y detener el entrenamiento cuando no hubiera mejora. Estas particiones se crearon manteniendo la misma proporción de muestras por clase. Destacar también que estas mismas particiones se emplearon para entrenar y evaluar todos los enfoques considerados para garantizar una comparación justa entre ellos.

6.5. Experimentación

En este apartado se evalúan las diferentes partes del método propuesto, comenzando por el funcionamiento del *backbone* y la cabeza de clasificación, y continuando con el análisis de los resultados obtenidos por cada una de las cabezas especializadas. En todos los casos, los resultados se comparan con los de otros métodos del estado del arte. Para esto, es importante tener en cuenta que todos los modelos fueron entrenados y evaluados bajo las mismas condiciones, inicializándose con los pesos pre-entrenados obtenidos con el *dataset* ILSVRC, entrenando durante el mismo número de épocas, con los mismos *datasets* descritos en la Sección 6.3, y manteniendo las mismas particiones de entrenamiento y *test*.

6.5.1. Evaluación de la clasificación de gestos

Para evaluar el desempeño de los métodos de clasificación de gestos se eligieron tres métricas de evaluación ampliamente utilizadas para este tipo de tareas: *Precision*, *Recall* y F_1 (previamente definidas en la Sección 4.4.1). Recordamos que, dado que los experimentos se realizan para un problema multiclase, reportamos los resultados en términos de macro- F_1 para una evaluación global, que se calcula como el promedio de los F_1 obtenidos para cada clase.

Para esta primera parte del método (dedicada a la clasificación del gesto inicial), consideramos tres enfoques diferentes:

- Arquitecturas CNN categóricas para clasificar el gesto en la imagen de entrada.

- Redes de detección de objetos para detectar y clasificar el gesto.
- Enfoque de seguimiento de manos basado en la librería MediaPipe (Zhang et al., 2020b), que detecta tanto la mano como la posición de los dedos.

6.5.1.1. Arquitecturas CNN categóricas

Para el primer tipo de aproximación se consideraron 13 topologías CNN representativas del estado del arte (ver Tabla 6.2), cuya descripción se encuentra en el Apéndice D. Estas topologías se usaron como *backbones*, eliminando sus capas finales y reemplazándolas con la cabeza de clasificación descrita en la Sección 6.4.1.

La Tabla 6.2 muestra los resultados obtenidos para la tarea de clasificación utilizando el conjunto de datos generado sintéticamente. Hay que tener en cuenta que todas las redes se inicializaron con los pesos obtenidos con el conjunto de datos ILSVRC y luego se entrenaron y evaluaron para este conjunto de datos sintético. Como puede verse, se consiguen buenos resultados en todos los casos, siendo los mejores los de Darknet-53 seguido del modelo Xception, y los peores los obtenidos por las arquitecturas VGG.

Tabla 6.2.: Comparación de los resultados obtenidos para la clasificación del *dataset* sintético con las 13 topologías CNN del estado del arte consideradas. Los mejores resultados obtenidos por métrica están marcados en negrita, mientras que los segundos mejores están subrayados.

Modelo	Precision	Recall	F_1
SqueezeNet (Iandola et al., 2016)	87,25	86,13	86,69
ResNet-50 (He et al., 2016)	95,74	94,75	95,24
VGG 16 (Simonyan and Zisserman, 2014)	85,12	67,37	75,21
VGG 19 (Simonyan and Zisserman, 2014)	84,77	65,40	73,84
Inception v3 (Szegedy et al., 2015)	96,60	96,25	96,42
MobileNet v1 (Howard et al., 2017)	97,85	97,83	97,84
MobileNet v2 (Sandler et al., 2018)	97,24	97,12	97,18
MobileNet v3 (Howard et al., 2019)	94,83	92,99	93,90
EfficientNet-B0 (Tan and Le, 2019)	97,16	97,05	97,10
EfficientNet-B1 (Tan and Le, 2019)	97,64	97,41	97,52
DenseNet121 (Huang et al., 2017)	96,51	96,03	96,27
Xception (Chollet, 2016)	98,33	98,30	98,31
Darknet-53 (Redmon and Farhadi, 2018)	99,76	99,75	99,75

Los pesos aprendidos con el conjunto de datos sintético se utilizaron para inicializar las redes antes de entrenar con el conjunto de datos de imagen real. La Tabla 6.3 muestra los resultados obtenidos tras este segundo paso del proceso de entrenamiento. Esta tabla también incluye una comparación con el resultado que se obtendría si no se aplicase esta inicialización, es decir, inicializando con ILSVRC y luego entrenando directamente con

el conjunto de datos real. Como se puede observar, los mejores resultados se obtienen nuevamente con la arquitectura Darknet-53 seguida de Xception. Se observa que esta inicialización ayuda a mejorar los resultados en más de un 3 % de media y, si analizamos estos resultados individualmente, hasta casi un 10 % para el caso de EfficientNet-B0 y un 5,5 % para Darknet-53.

Tabla 6.3.: Resultados obtenidos para la clasificación del *dataset* de imagen real después de inicializar las 13 topologías CNN del estado del arte utilizando los pesos aprendidos con el *dataset* sintético. Esta tabla también muestra el resultado obtenido al no aplicar esta inicialización, es decir, comenzando solo con los pesos pre-entrenados con ILSVRC. Los mejores resultados obtenidos por métrica están marcados en negrita, los segundos mejores están subrayados.

Modelo	Sin inicialización			Inicialización sintética		
	Precision	Recall	F_1	Precision	Recall	F_1
SqueezeNet	69,13	67,08	68,09	71,23	67,95	69,55
ResNet-50	80,26	78,98	79,61	81,78	80,07	80,92
VGG16	76,72	73,58	75,12	78,45	76,29	77,35
VGG19	79,19	78,33	78,76	81,20	79,90	80,54
Inception v3	75,68	73,33	74,49	78,62	75,50	77,03
MobileNet v1	82,72	82,75	82,73	83,96	83,04	83,50
MobileNet v2	83,44	82,42	82,93	84,68	86,60	84,12
MobileNet v3	84,13	79,86	81,94	85,24	83,27	84,24
EfficientNet-B0	67,29	61,09	64,04	78,50	69,86	73,93
EfficientNet-B1	75,15	69,50	72,21	78,59	75,14	76,86
DenseNet121	82,88	79,29	81,05	85,90	81,43	83,61
Xception	87,01	86,25	86,63	91,42	90,21	90,81
Darknet-53	89,80	87,64	88,71	95,31	93,14	94,21
Promedio	79,49	76,93	78,18	82,68	79,95	81,28

6.5.1.2. Redes de detección de objetos

El segundo enfoque evaluado para la clasificación del gesto inicial es el uso de redes de detección de objetos, para lo cual se compararon cuatro alternativas: Faster R-CNN (FRCNN) (Ren et al., 2015), RetinaNet (Lin et al., 2017), YOLO v3 (Redmon and Farhadi, 2018) (descritas en la Sección 3.1.2.1), y SelAE (Gallego et al., 2018a) (descrita en el capítulo anterior, en la Sección 5.4.3).

La Tabla 6.4 muestra los resultados obtenidos por estos métodos, comparándolos con los dos mejores resultados obtenidos previamente por las redes de clasificación (ver Tabla 6.3). Los métodos de detección de objetos devuelven la posición de los gestos en la imagen (el etiquetado utilizado para este proceso se describe en la Sección 6.3). Dado que estas redes pueden devolver múltiples predicciones, se seleccionó el cuadro delimitador predicho con la confianza más alta. Para evaluar el resultado obtenido es necesario diferenciar si el *ground truth* contiene un gesto o no. Cuando lo hace, la predicción se considera TP si, además de coincidir la clase, su IoU con el *ground truth* es superior a

0,5, FP cuando es inferior a 0,5 y FN en el caso de no devolver ninguna predicción. Lo contrario aplica para el gesto “Ninguno” (es decir, los casos en los que el *ground truth* no contiene ningún gesto), se considera TP cuando no hay predicción y FP cuando la hay.

La Tabla 6.4 muestra que, entre los enfoques de detección de objetos, YOLO v3 obtiene el mejor resultado, seguido de SelAE y RetinaNet. Sin embargo, si los comparamos con los métodos entrenados específicamente para la clasificación categórica, Darknet-53 (el *backbone* utilizado por el propio YOLO v3) sigue obteniendo la mejor puntuación.

Tabla 6.4.: Resumen de los resultados obtenidos por los diferentes enfoques considerados para la tarea de clasificación de gestos. El mejor resultado para cada métrica está marcado en negrita y el segundo mejor resultado está subrayado.

Aproximación	Modelo	Precision	Recall	F_1
Categórico	Xception	91,42	90,21	90,81
	Darknet-53	95,31	93,14	94,21
Object detection	FRCNN	78,71	70,34	74,29
	RetinaNet	86,25	83,29	84,74
	SelAE	88,17	86,21	87,18
	YOLO v3	<u>94,30</u>	<u>93,13</u>	<u>93,71</u>
Seguimiento de manos	MediaPipe	76,81	77,94	77,37

6.5.1.3. Enfoque de seguimiento de la mano

El resultado mediante este tercer enfoque se ha incluido también en la Tabla 6.4, el cual realiza la detección de gestos mediante el método de seguimiento de manos proporcionado por MediaPipe Hands (Zhang et al., 2020b). Este método, a partir de una única imagen RGB, detecta 21 puntos clave (*keypoints*) en 3D que se corresponden con las articulaciones de los dedos de una mano. Para clasificar los gestos en función de esta información consideramos diferentes enfoques, como aplicar kNN o SVM con los 21 puntos clave directamente o acumulando los ángulos articulares de cada dedo. Este último (utilizando la suma de ángulos y SVM) fue el que mejores resultados obtuvo, por lo que es el que finalmente se incluye en la comparativa. Sin embargo, como se puede observar en la tabla, los resultados de este método no son competitivos si los comparamos con los del resto de aproximaciones (a excepción de FRCNN). Estos peores resultados se deben a que, en muchos casos, MediaPipe no detecta correctamente la mano. Estos resultados también coinciden con un trabajo reciente (Amaliya et al., 2021b) en el que se compara el rendimiento de este método con otros enfoques para el reconocimiento de la lengua de signos.

En la Figura 6.9 se pueden ver algunos ejemplos de las detecciones realizadas por este método. La primera fila muestra las detecciones correctas y la segunda fila los casos en

los que no se ha podido detectar la mano. Como se observa, los fallos se producen cuando la mano está parcialmente ocluida (es decir, solo se ven los dedos o parte de la mano), lo cual es bastante común en la aplicación propuesta debido a la posición egocéntrica de la cámara.



Figura 6.9.: Ejemplos de las detecciones obtenidas con el método MediaPipe. La primera fila muestra detecciones correctas y la segunda fila los casos en los que no ha podido detectar la mano.

6.5.1.4. Problema de optimización multiobjetivo

Otro aspecto importante a considerar es la eficiencia del método seleccionado. Dado que cuantos más parámetros tienen los modelos, menor es el rendimiento y mayor es el espacio de almacenamiento requerido, es necesario llegar a un compromiso entre la eficiencia de la red y su precisión. Sin embargo, estos criterios son con bastante frecuencia opuestos, ya que la mejora de uno de ellos suele conllevar el empeoramiento del otro. Desde este punto de vista, de la misma forma que se hizo en la Sección 4.4.2, esta tarea puede verse como un Problema de Optimización Multiobjetivo (MOP) en el que se pretende optimizar dos funciones simultáneamente.

Asumiendo este escenario MOP, la Figura 6.10 muestra los resultados obtenidos por los diferentes métodos evaluados en la sección anterior, en la que cada punto es un valor bidimensional definido por su F_1 y el número de parámetros de la topología correspondiente. Como puede verse, los mejores resultados para ambos criterios (es decir, los elementos *no dominados*) los obtienen MobileNet v2 (con el menor número de parámetros), Darknet-53 (con el mejor F_1) y SeIAE (como solución intermedia). Sin embargo, y como se argumentó anteriormente, es fundamental obtener una alta precisión en el primer paso del método propuesto, por lo que seleccionamos DarkNet-53 como *backbo-*

ne ya que obtiene un F_1 que es un 7,03 % superior al siguiente resultado no dominado (SeIAE). Además, esta arquitectura ha mostrado un buen desempeño en dispositivos móviles actuales. En base a pruebas realizadas en un Samsung A51 y Huawei P30 lite se obtuvo una tasa de procesamiento promedio de entre 3 y 4 FPS.

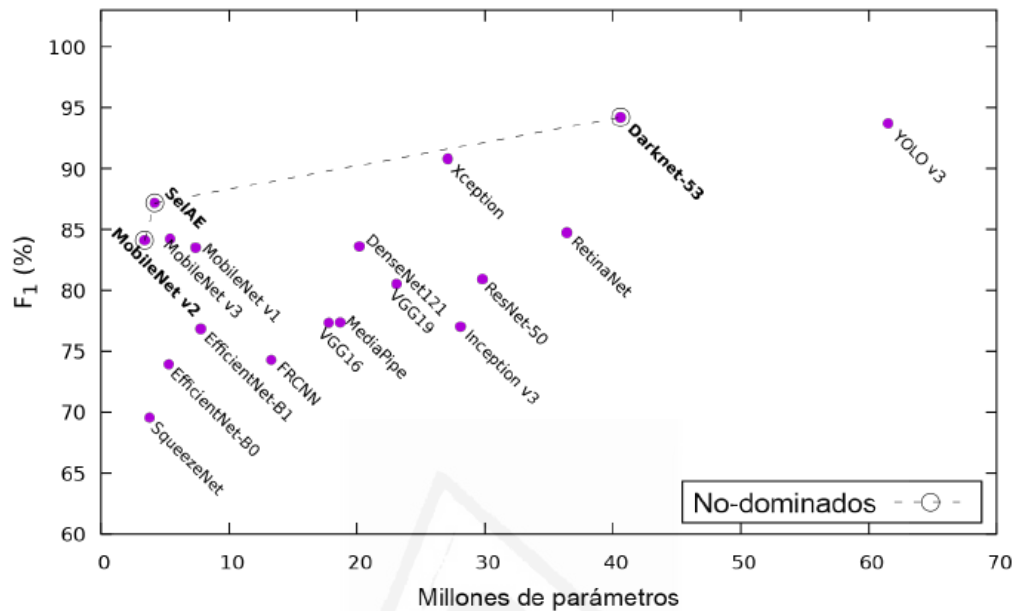


Figura 6.10.: Análisis de F_1 y número de parámetros como Problema de Optimización Multiobjetivo (MOP). Los elementos no dominados están resaltados.

6.5.1.5. Consistencia temporal

Otra parte importante a evaluar de la arquitectura propuesta es el módulo de consistencia temporal (ver Figura 6.6) y el efecto del valor seleccionado para el parámetro k . Este parámetro permite controlar el número de fotogramas que deben transcurrir con la misma respuesta para que esta sea considerada válida. La Figura 6.11 muestra una gráfica en la que se estudia este valor en el rango $[1, 4]$ para la tarea de clasificación utilizando el *backbone* Darknet-53. Como se puede observar, el resultado mejora cuando se establece en 2 o 3 fotogramas, ya que esto evita que se produzcan fotogramas con errores aislados. Sin embargo, si aumentamos este valor el resultado empieza a empeorar, ya que tiene que esperar muchos fotogramas para cambiar la respuesta y en todas las transiciones entre gestos comete errores. Por lo tanto, decidimos establecer el valor de k en 2, ya que como máximo genera un fotograma erróneo entre transiciones y a cambio mejora el resultado de F_1 de 94,21 % a 97,03 %.

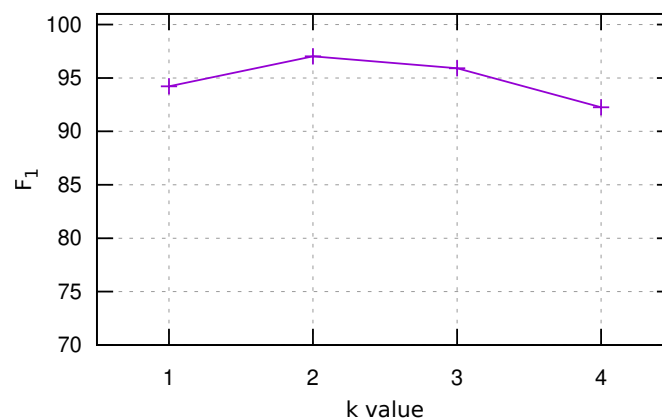


Figura 6.11.: Evaluación del parámetro k en la consistencia temporal para la tarea de clasificación utilizando el *backbone* Darknet-53.

6.5.2. Evaluación de gestos de señalar y arrastrar

Una vez establecida la configuración principal de la arquitectura y de la clasificación inicial de los gestos, se procede a analizar los resultados obtenidos por cada una de las cabezas especializadas. En esta sección nos centraremos en las cabezas utilizadas para los gestos de “Apuntar” y “Arrastrar”, comenzando con el análisis y ajuste de la metodología propuesta para estas acciones (ver Sección 6.4.2) y luego comparándola con otras arquitecturas. En todos los casos, se utilizó el conjunto de datos con el etiquetado de la punta del dedo para los gestos de “Apuntar” y “Arrastrar”.

Los resultados también se evalúan utilizando la métrica F_1 (Ecuación 4.3), pero en este caso medimos los objetos (las puntas de los dedos de los dos gestos considerados) cuya ubicación se detectó correctamente. Esto se hace calculando el cuadro delimitador de los objetos predichos (P) y comparándolo con el cuadro delimitador del *ground truth* (B) con el que tiene un IoU más alto (usando la Ecuación 5.4). Se considera que un cuadro delimitador predicho P se detecta correctamente si $\text{IoU}(P, B) \geq \lambda$. Establecemos $\lambda = 0.5$ (un valor de umbral comúnmente utilizado en este tipo de tareas), y calculamos la métrica F_1 , considerando que las detecciones correctas son TP (es decir, las que tienen un IoU mayor que λ), las detecciones incorrectas son FP (es decir, en las que P no se superpone con ningún B con un IoU mayor que λ), y aquellos casos en los que no se detecta un objeto del *ground truth* son FN. Hay que tener en cuenta que si se predicen múltiples detecciones del mismo objeto, solo la primera se cuenta como positiva, mientras que el resto se contarán como negativas.

6.5.2.1. Análisis y ajuste de los parámetros del método propuesto

Primero vamos a evaluar los parámetros de la metodología propuesta basada en selección de filtros (ver Sección 6.4.2), comenzando con la capa seleccionada (variable l de la Ecuación 5.3). Para ello fijamos el resto de parámetros a una configuración inicial ($|\mathcal{F}^c| = 3$, $\beta = 0,6$ y un elemento estructurante s cuadrado de tamaño 5×5) y variaremos solo la capa l seleccionada. Para este análisis consideraremos tres arquitecturas de red representativas de las evaluadas anteriormente (Darknet-53, DenseNet121 y MobileNet v3), aunque, como veremos más adelante, este ajuste se realizó para todas las redes.

La Figura 6.12a muestra la influencia de la capa seleccionada para predecir la localización. Como cada red tiene un número diferente de capas, en esta figura representamos el resultado en función de la profundidad de la capa, donde el 100 % de la profundidad representa la última capa de la red. Como puede verse, los resultados en la primera parte (más o menos hasta el 50 % de profundidad) no fueron buenos. Sin embargo, como era de esperar, se obtuvieron mejores resultados de localización en la última parte de las redes (a partir del 50 % de profundidad), a partir de la cual se aprenden características de mayor nivel. Por lo tanto, finalmente se seleccionó la capa “conv2d_49” para Darknet-53 (Redmon and Farhadi, 2018), “conv5_block6_2_conv” para DenseNet121 (Huang et al., 2017) y “re_lu_35” para MobileNet v3 (Howard et al., 2019) (las arquitecturas de red completas se pueden consultar en los artículos correspondientes).

Otra variable importante a analizar es el número de filtros seleccionados para obtener la localización, es decir, el tamaño del conjunto $|\mathcal{F}^c|$ en la Ecuación 6.2, que puede ser ajustado modificando el valor de umbral α . Para este experimento usamos las mejores capas previamente seleccionadas, $\beta = 0.6$ y $s = 5 \times 5$. La Figura 6.12b muestra los resultados obtenidos al variar el tamaño de este conjunto. Como se puede observar, se obtiene un máximo al utilizar entre 3 y 6 filtros, y los mejores resultados se obtienen con 4 filtros para Darknet-53 y DenseNet121, y con 5 filtros para MobileNet v3.

Otro parámetro a analizar es el valor del umbral β (ver Ecuaciones 6.1 y 6.2). Como antes, configuramos el resto de los valores de los parámetros a los mejores valores encontrados hasta el momento y variamos solo este parámetro en el rango $[0, 1]$. Como se puede ver en la Figura 6.12c, se obtienen mejores resultados con valores más altos para este umbral, es decir, al seleccionar solo aquellos píxeles con las activaciones más altas. Los valores específicos seleccionados para cada red son: $\beta = 0,92$ para Darknet-53, $\beta = 0,84$ para DenseNet121 y $\beta = 0,88$ para MobileNet v3.

Finalmente, también analizamos la influencia del tamaño del elemento estructurante s (ver Ecuaciones 6.1 y 6.2) que se utiliza para la dilatación del resultado obtenido de la activación de los filtros antes de calcular el cuadro delimitador con la posición de los objetos detectados. La influencia de este parámetro se evaluó variando el tamaño de s entre 3×3 y 13×13 , y fijando los parámetros restantes a los mejores valores

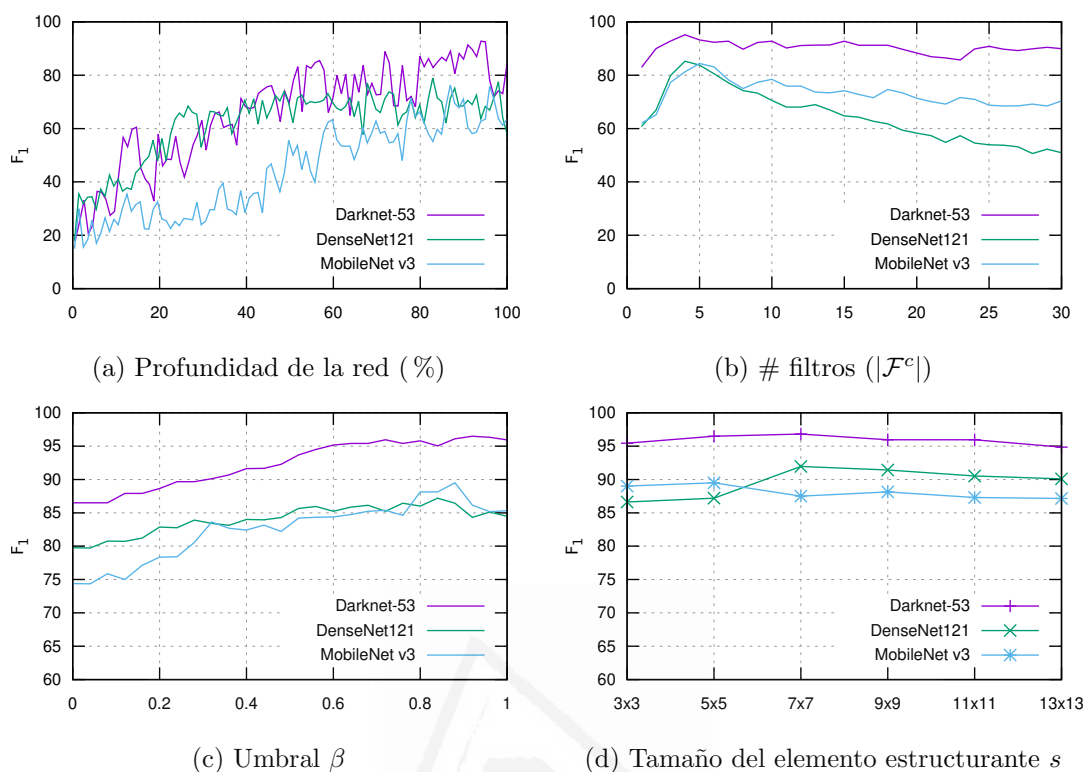


Figura 6.12.: Resultados de localización (F_1 %) obtenidos al variar (a) la capa de la red de la que se seleccionan los filtros (en porcentaje con respecto al 100 % de la profundidad total de la red), (b) el número de filtros en el conjunto $|\mathcal{F}^c|$, (c) el umbral β , y (d) el tamaño del elemento estructurante s .

encontrados en los experimentos anteriores. La Figura 6.12d muestra el resultado de este análisis. Como puede verse, el resultado se mantiene bastante estable al variar este parámetro y mejora solo ligeramente con un tamaño de kernel de 7×7 para Darknet-53 y 5×5 para MobileNet v3.

6.5.2.2. Comparación con métodos alternativos

Una vez analizados los diferentes parámetros del método propuesto y determinada la mejor configuración, compararemos los resultados obtenidos con los de otros métodos del estado del arte, incluidas las cuatro redes de detección de objetos evaluadas anteriormente (FRCNN (Ren et al., 2015), RetinaNet (Lin et al., 2017), YOLO v3 (Redmon and Farhadi, 2018), y SelAE (Gallego et al., 2018a)) y el resultado obtenido al aplicar FS al resto de redes categóricas consideradas anteriormente. Para esta comparación, al igual que hicimos en el capítulo anterior, mostramos el valor promedio del IoU obtenido

junto con la Precisión Promedio (*Average Precision* o AP), dado que estas métricas son ampliamente utilizadas para evaluar métodos de detección de objetos. Para medir el mAP, se calcula AP para cada clase y luego se promedia para todas las clases.

La Tabla 6.5 muestra los resultados de esta comparación. Como se puede observar, el método propuesto para llevar a cabo esta tarea (Darknet-53 + FS) es el que mejores resultados obtiene, seguido de YOLO v3 (arquitectura a la que pertenece el *backbone* utilizado). En general, los enfoques de detección de objetos obtienen resultados bastante buenos. Sin embargo, FS, además de mejorar ligeramente el resultado, es una solución más eficiente, ya que no agrega ninguna capa de procesamiento a la red, simplemente aprovecha las activaciones de los filtros ya calculados por el *backbone* para realizar esta detección.

Tabla 6.5.: Comparación de los resultados obtenidos utilizando el enfoque propuesto (Darknet-53 + FS) y otros métodos del estado del arte, incluidos los métodos de detección de objetos y el resultado de aplicar FS al resto de redes categóricas consideradas anteriormente. El mejor resultado para cada métrica está marcado en negrita, los segundos mejores están subrayados.

Aproximación	Método	Avg(Iou)	mAP
Detección de objetos	FRCNN	77,95	85,46
	RetinaNet	83,35	90,08
	YOLO v3	84,71	95,63
	SelAE	83,38	91,21
Selección de filtros	SqueezeNet + FS	65,25	70,12
	ResNet-50 + FS	74,74	83,99
	VGG16 + FS	71,07	79,46
	VGG19 + FS	71,91	81,77
	Inception v3 + FS	58,89	78,30
	MobileNet v1 + FS	71,13	84,28
	MobileNet v2 + FS	74,51	85,61
	MobileNet v3 + FS	75,40	88,93
	EfficientNetB0 + FS	77,40	85,25
	EfficientNetB1 + FS	79,57	91,25
	DenseNet121 + FS	80,26	91,20
	Xception + FS	83,66	92,14
	Darknet-53 + FS	85,77	96,32

La Figura 6.13 muestra un ejemplo de los filtros obtenidos para una imagen de entrada, junto con el proceso de sumar el resultado hasta alcanzar la predicción final. La primera fila de esta imagen muestra el fotograma de entrada y el proceso realizado para el primer filtro, mientras que la segunda, tercera y cuarta fila muestran, además del proceso realizado sobre el filtro, el resultado de la suma incremental con los filtros anteriores.

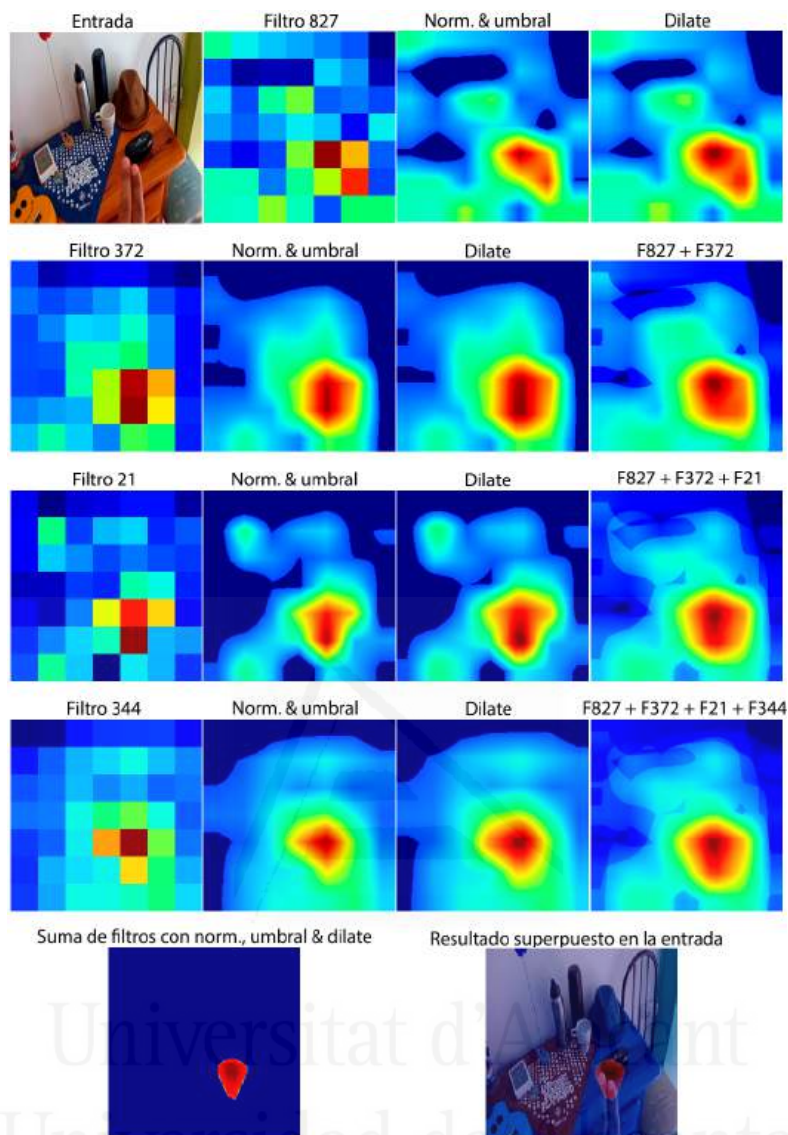


Figura 6.13.: Ejemplo del proceso realizado para calcular la ubicación de las puntas de los dedos utilizando el enfoque propuesto (Darknet-53 + FS). La primera imagen en la primera fila es la entrada. Se muestran los resultados intermedios obtenidos para cada uno de los cuatro filtros seleccionados, así como su suma incremental con los filtros anteriores. La última fila muestra el resultado final, incluida una superposición de la localización prevista con la imagen de entrada. El color rojo oscuro indica un valor de activación más alto.

6.5.3. Evaluación del gesto de lupa

Para evaluar las descripciones generadas por la cabeza dedicada al gesto de la lupa, recurriremos a la ampliamente adoptada métrica *Bi-Lingual Evaluation Understudy* (BLEU) (Papineni et al., 2002). Generalmente se utiliza para evaluar la calidad de oraciones generadas por máquinas comparándolas con oraciones de referencia en problemas relacionados con la generación de lenguaje, *captioning* (descripción de imágenes), resúmenes de texto o reconocimiento de voz, entre otros. El resultado de esta métrica está en el rango $[0, 1]$, donde los valores más cercanos a 1 representan textos más similares. Una puntuación de 1 indica que las oraciones son iguales, sin embargo, no es necesario alcanzar este valor para que el texto sea correcto. Consideraremos el uso de n-gramas de longitud entre 1 y 4 para el cálculo de esta métrica. Esta longitud se refiere a la cantidad de palabras seguidas que deben coincidir, por lo que la longitud de 4 (indicada como BLEU-4) supone un mayor reto. Es conveniente destacar que para este experimento, además de las imágenes con el gesto de la lupa, se evaluaron también el gesto de señalar y las imágenes sin gesto. Estos dos últimos casos se agregaron para considerar muestras de evaluación adicional y también para evaluarlas en caso de que también tengan que generar una descripción de imagen en la aplicación final.

La Tabla 6.6 muestra los resultados de esta evaluación, en la que se compara el método propuesto (Darknet-53 + *captioning*) con el método *merge-model* propuesto por Tanti et al. (2018) (en el que se basa la cabeza especializada de nuestra propuesta) y con el resultado que se obtendría al cambiar Darknet-53 por DenseNet121 o MobileNet v3. Además, también se considera el caso de entrenar de forma independiente la red formada por Darknet-53 seguida de las capas utilizadas por la cabeza de *captioning*. En este último caso, la red se inicializa con los pesos pre-entrenados para ILSVRC y luego se entrena (sin congelar el *backbone*) con el conjunto de datos creado para esta tarea. Como se puede observar en la tabla, el mejor resultado se obtiene con el método propuesto pero entrenando de forma independiente. Sin embargo, este resultado es solo ligeramente mejor que el obtenido al considerar toda la arquitectura propuesta (resultado de la última fila). Por tanto, esta pequeña mejora no justifica el uso de una red independiente para llevar a cabo esta tarea, ya que supondría un aumento considerable de los recursos necesarios. El peor resultado se obtiene con el método de *merge-model*, quizás porque considera un *backbone* más simple (VGG-19). Como referencia, los resultados reportados por Tanti et al. (2018) para el conjunto de datos Flickr8k son 0,191, 0,287, 0,424, 0,611 para BLEU-4, BLEU-3, BLEU-2 y BLEU-1, respectivamente.

La Figura 6.14 muestra diez ejemplos de los *captions* generados para nuestro conjunto de datos. Las dos primeras columnas de estos ejemplos incluyen casos en los que los textos generados mencionan la mano o los dedos y el resultado obtenido tras su procesamiento. Como se puede observar, el método genera descripciones que detallan correctamente las escenas y los objetos que aparecen en ellas.

Tabla 6.6.: Comparación de los resultados obtenidos (en términos de BLEU) para la tarea de *captioning* de imágenes.

Método	BLEU-4	BLEU-3	BLEU-2	BLEU-1
Merge-model (Tanti et al., 2018)	0,1518	0,1864	0,3151	0,4987
DenseNet121 + captioning	0,1997	0,3117	0,3904	0,5882
MobileNet v3 + captioning	0,1803	0,2921	0,3477	0,5614
Backbone independiente + captioning	0,2163	0,3236	0,4408	0,6135
Método propuesto (Darknet-53 + captioning)	0,2088	0,3181	0,4390	0,6027

6.5.4. Evaluación del gesto de pellizco

Finalmente, vamos a evaluar el gesto de pellizcar. Como se indica en la metodología, este es un gesto dinámico que según el movimiento puede indicar si el usuario desea acercar, alejar o si se debe mantener el nivel de *zoom* actual. Como en secciones anteriores, nos basaremos en las métricas *Precision*, *Recall* y F_1 para evaluar el rendimiento de la detección de estas acciones.

Para esta evaluación, se compara el enfoque propuesto con el uso de MediaPipe y con la misma arquitectura de red pero entrenada por separado exclusivamente para este gesto (es decir, considerando solo el *backbone* y la cabeza de pellizco). En este último caso, se inicializa el *backbone* con los pesos obtenidos para ILSVRC, se añaden las capas utilizadas para la cabeza especializada en el gesto de pellizco, y por último se ajustan los pesos de toda la red para el conjunto de datos preparado para esta acción. En el caso de MediaPipe, se calcula la distancia entre los dedos pulgar e índice para determinar qué gesto se realiza. De esta manera, es posible distinguir el gesto dinámico realizado: acercar cuando la distancia aumenta, alejar cuando la distancia disminuye y mantener el nivel de *zoom* cuando la distancia se mantiene estable (permitiendo un pequeño umbral de ± 3 px de variación).

La Tabla 6.7 muestra los resultados de esta comparación. Como se puede observar, MediaPipe obtiene las puntuaciones más bajas, ya que, como se argumentó anteriormente, este método depende bastante de la visibilidad de la mano. El enfoque propuesto logra un F_1 de 90,64 %, que es solo un 0,97 % menos que el resultado obtenido por esta misma arquitectura pero entrenada específicamente para este gesto. Este experimento demuestra que el proceso de entrenamiento propuesto no supone un deterioro notable en el resultado. Sin embargo, a cambio, permite entrenar y conseguir un sistema eficiente para el procesamiento simultáneo de diferentes acciones. Por tanto, esta pequeña diferencia no justificaría el uso de una red paralela para procesar este gesto, ya que ello implicaría una reducción de la eficiencia y un aumento de los recursos necesarios.

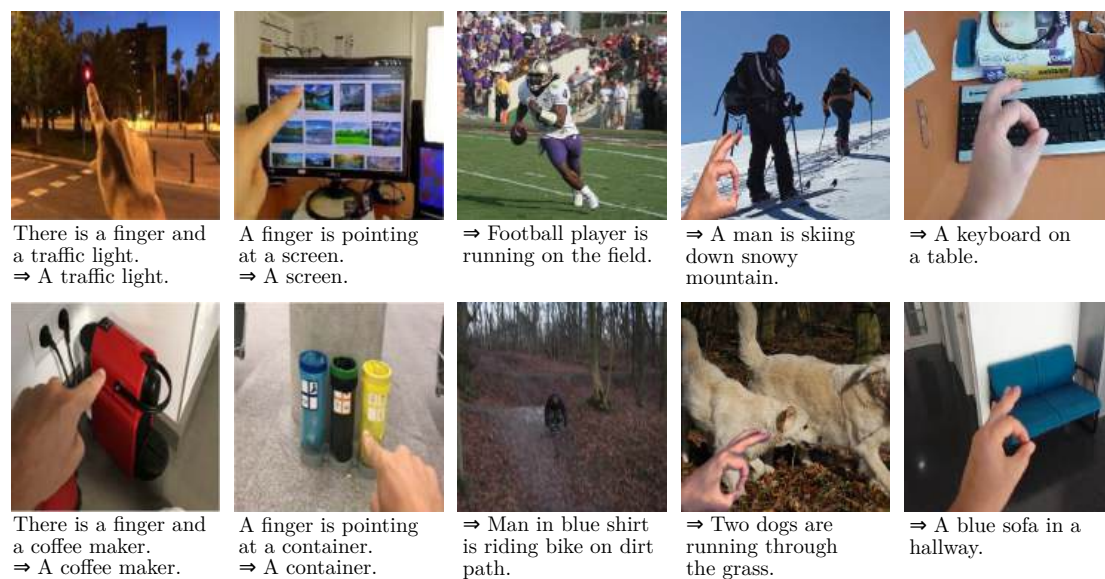


Figura 6.14.: Ejemplos de resultados de *captioning* con el conjunto de datos de gestos propuesto. Las dos primeras columnas muestran los casos en los que la descripción menciona la mano o el dedo, y el resultado obtenido después del paso de posprocesamiento.

Tabla 6.7.: Comparación de los resultados obtenidos para el gesto de pellizco.

Aproximación	Precision	Recall	F_1
MediaPipe	63,50	63,57	63,53
Backbone independiente + cabeza de pellizco	93,27	90,01	91,61
Método propuesto	92,06	89,27	90,64

6.6. Conclusiones

En este capítulo se ha propuesto un método eficiente que pueda servir como base para la construcción de un sistema interactivo que pueda ser controlado mediante el uso de gestos de las manos. Para ello, se han realizado las siguientes aportaciones:

- Se ha definido una interfaz basada en gestos para nuestro sistema interactivo. Cada gesto (estático o dinámico) está asociado a una determinada acción (describir el objeto señalado, hacer *zoom*, etc.).
- Se ha propuesto una arquitectura *multi-head* que integra las diferentes tareas, aprovechando las características aprendidas por un *backbone* común. Con esto, cum-

plimos el objetivo de investigar el desarrollo de metodologías capaces de integrar múltiples tareas de forma eficiente, planteado inicialmente.

- Se han creado nuevos conjuntos de datos, reales y sintéticos, para entrenar y evaluar el método propuesto, con un total de 44 mil imágenes etiquetadas a diferentes niveles: categoría, posición de las manos y punta de los dedos, posición y categoría de los objetos señalados y descripción de las escenas. Estos nuevos conjuntos de datos extienden y mejoran los construidos en etapas previas de la investigación, para contar con una mayor variabilidad de datos y con ejemplos que permitan entrenar y validar las diferentes funciones de nuestro sistema.
- Se ha integrado y optimizado el método de selección de filtros propuesto en el capítulo anterior, evaluándolo en esta ocasión con los conjuntos de datos de gestos de la mano.
- Se han realizando experimentos comparando los resultados de cada una de las cabezas especializadas con enfoques alternativos del estado del arte.

El sistema propuesto ha obtenido muy buenos resultados tanto en cuanto a precisión como en eficiencia. En la comparación realizada para cada una de las cabezas especializadas con métodos alternativos del estado del arte, siempre nuestra propuesta ha obtenido los mejores resultados, o casi los mejores. En los casos en los que no se obtuvieron los mejores resultados, el resultado obtenido fue muy cercano al mejor, por lo que la ligera mejora no justificaría tener que entrenar y utilizar una arquitectura adicional específica para realizar dicha función.

Por otro lado hay que destacar también que la arquitectura propuesta ha obtenido un buen rendimiento en los experimentos realizados con dispositivos móviles de gama media, con tasas de procesamiento de entre 3 y 4 FPS. Esto permite que la solución propuesta sea integrada en aplicaciones móviles, con lo que podría utilizarse para dotar de una interfaz accesible a aplicaciones dirigidas a asistir a personas invidentes o con dificultades visuales.

CAPÍTULO 7

CONCLUSIONES

Resumen

En este capítulo se resumen las conclusiones generales del trabajo realizado a lo largo de esta tesis doctoral. Además, se presentan las posibles líneas de trabajo futuro, que incluyen ideas y mejoras a desarrollar.

Contenidos

7.1	Conclusiones generales	114
7.2	Trabajo futuro	117

7.1. Conclusiones generales

El reconocimiento de gestos es un campo de investigación que involucra un gran número de diferentes enfoques, tal como se mostró en la revisión del estado del arte, abarcando tanto tecnologías basadas en hardware específico de captura, como métodos basados en la visión por computador, donde las redes neuronales y el aprendizaje profundo juegan un papel clave en la actualidad. La detección precisa, robusta y eficiente de gestos sigue siendo un reto en visión por computador, especialmente cuando buscamos trasladar estos métodos a dispositivos limitados, como es el caso de los teléfonos inteligentes. Si además buscamos que el reconocimiento de gestos vaya ligado a diferentes acciones que también requieren el uso de métodos de visión por computador de elevado coste computacional, se hace necesario buscar soluciones que integren todas estas tareas de forma eficiente. Por este motivo, como contribución principal de este trabajo hemos propuesto y evaluado un arquitectura de red *multi-head* que integra tanto la clasificación de los gestos, como la localización de la mano y de la punta de los dedos, la descripción de la escena, y el reconocimiento de un gesto dinámico para modificar el *zoom*. A continuación se exponen de forma desglosada las diferentes aportaciones realizadas:

- **Arquitectura de red *multi-head***, que cuenta con un *backbone* común que aprende una serie de características que son utilizadas por las diferentes cabezas especializadas. Esta arquitectura nos permite reducir notablemente el número de parámetros de la red, ya que todas las cabezas aprovechan las mismas características comunes y, además, solo se ejecuta la cabeza correspondiente al gesto que se haya detectado, obteniendo así una solución eficiente que realiza todas las tareas del sistema de forma integrada. Esta aportación cumple con el objetivo que nos planteamos inicialmente de investigar en el desarrollo de metodologías capaces de realizar múltiples tareas de forma eficiente.
- **Definición de una interfaz accesible basada en gestos**, para la cual se ha definido un conjunto de gestos, estáticos y dinámicos, y acciones asociadas a cada uno de ellos. Para la selección de este conjunto de gestos se estudiaron todas las posibles acciones requeridas, escogiendo finalmente un conjunto reducido que permitía controlar de manera natural e intuitiva un dispositivo a través de esta propuesta de interfaz. No obstante, la metodología desarrollada se ha planteado de manera que resulte sencillo ampliar el conjunto de gestos considerados.
- **Conjuntos de datos** con un total de 44 mil imágenes reales y sintéticas, con anotaciones sobre la categoría del gesto, la localización de la mano y de la punta de los dedos, así como la descripción de la imagen y de los objetos señalados. Las imágenes reales fueron extraídas de vídeos obtenidos con teléfonos móviles, utilizando diferentes modelos de teléfono y localizaciones de interior y exterior con diferentes condiciones de iluminación, para conseguir así un conjunto con alta

variabilidad. Con esto cumplimos el objetivo que nos planteamos inicialmente de validar la metodología con datos reales que incluyan todas las posibles casuísticas en las que debería funcionar el modelo. Además, se ha desarrollado una herramienta (basada en la librería LibHand), para generar de forma automática conjuntos de datos sintéticos con el etiquetado correspondiente.

- **Selección del *backbone*** y de la cabeza de clasificación, para lo que se compararon diferentes arquitecturas usando los conjuntos de datos anteriores. Dado que buscamos una solución que maximice tanto la precisión de la clasificación como su eficiencia, esta evaluación se planteó como un problema de optimización multiobjetivo (MOP), de este modo cumplimos también con el propósito inicial de desarrollar métodos eficientes.
- **Método de selección de filtros** débilmente supervisado para la localización precisa de objetos. Esta propuesta aprovecha los filtros aprendidos por una red categórica para estimar la localización de los objetos, de modo que una vez entrenada la red solo se requiere un reducido número de imágenes etiquetadas con la localización para realizar la selección de filtros y permitir el uso de la misma red también para la tarea de detección. Esto cumple con el objetivo planteado de explorar soluciones que requieran pocos datos etiquetados para su entrenamiento y, además, con el de buscar una solución eficiente, ya que se aprovecha la misma red sin tener que añadir capas adicionales.
- **Experimentación y evaluación** de las propuestas realizadas, comparándolas con los principales métodos del estado del arte. Se han analizado los resultados para las diferentes tareas (clasificación, localización, *captioning* y reconocimiento del gesto dinámico de *zoom*) y se han comparado con los de los principales enfoques alternativos, incluyendo arquitecturas completas específicas para abordar estas tareas, e incluso otros enfoques como la obtención del esqueleto de la mano mediante la librería MediaPipe.

Considerando las aportaciones anteriores podemos concluir que se han podido alcanzar todos los objetivos planteados al inicio de la investigación, habiendo obtenido una solución eficiente y competitiva con respecto a los métodos alternativos del estado del arte que son específicos de cada una de las tareas evaluadas, tal y como se ha observado en la experimentación realizada.

Una de las conclusiones extraídas de la experimentación, referente al proceso de entrenamiento de las redes categóricas, ha sido que pre-entrenar con el conjunto de datos sintético mejora de forma notable los resultados obtenidos. También se han observado mejoras de forma general al utilizar modelos inicializados con los pesos aprendidos del conjunto de datos ILSVRC así como al aplicar la técnica de aumentado de datos.

En cuanto a la aplicación de librerías como MediaPipe para la obtención del esqueleto de la mano, se ha observado, en consonancia con otros trabajos, que este enfoque tiene dificultades para reconstruir el esqueleto cuando la mano no aparece completamente dentro de la imagen. Dado que en nuestro enfoque estamos realizando el reconocimiento desde un punto de vista egocéntrico, estos casos pueden darse de forma habitual, por lo que esta aproximación no resulta adecuada para nuestra propuesta, habiendo obtenido resultados que no son competitivos con los obtenidos por las redes categóricas que toman como entrada la imagen capturada.

Los mejores resultados para la clasificación de gestos que optimizan precisión y eficiencia (soluciones no dominadas según un enfoque MOP), se han obtenido mediante las arquitecturas MobileNet v2, Darknet-53 y SeIAE. De ellos seleccionamos Darknet-53 como *backbone* por ser el que proporcionaba la mejor precisión. Además, hemos comprobado que los resultados obtenidos se pueden mejorar notablemente introduciendo un módulo de consistencia temporal que valide que se produce la misma respuesta durante al menos 2 fotogramas.

Para la localización de la mano y de la punta de los dedos se han probado diferentes enfoques. La aproximación Grad-CAM, si bien es capaz de localizar correctamente la mano, no proporciona la precisión suficiente para localizar objetos pequeños, como la punta de los dedos. Para conseguir una mejor precisión, se ha propuesto, ajustado y evaluado el método de selección de filtros. Este método se aplicó en primer lugar a un problema complejo de localización de objetos pequeños (localización de barcos en imágenes de satélite del conjunto de datos MASATI v2), para así validar la propuesta y, posteriormente, se ha trasladado al problema de reconocimiento de gestos. Una de las cuestiones más destacables en los experimentos realizados con este método es que es suficiente entrenarlo con 14 imágenes, ya que no se ha observado mejora al aumentar el tamaño del conjunto de datos por encima de ese valor. También se ha observado que los mejores resultados de localización se obtienen cuando se seleccionan filtros de las últimas capas de la red (es decir, las más profundas). En cuanto a la comparación con otros métodos del estado del arte, el método propuesto basado en la selección de filtros es el que ha obtenido mejores resultados, seguido de otras aproximaciones como YOLO v3, RetinaNet o Faster R-CNN.

Para las tareas de *captioning* y de ajuste de nivel de *zoom*, se ha comparado la arquitectura *multi-head* propuesta (en la que los pesos del *backbone* son congelados al entrenar cada cabeza) con la misma arquitectura (*backbone* + cabeza especializada), pero entrenada por separado de forma específica para estas tareas. En ambos casos, los mejores resultados se obtuvieron cuando la arquitectura se entrena por separado, pero con una diferencia muy reducida respecto a la propuesta de utilizar un *backbone* común, por lo que la pequeña mejora en los resultados no justifica el coste computacional que supondría añadir una nueva arquitectura de red independiente.

De forma general, se ha observado que al comparar los resultados de cada una de las cabezas especializadas con otros enfoques del estado del arte, incluyendo opciones específicas para esas mismas tareas, los mejores resultados (o casi los mejores) los obtienen en todos los casos estas cabezas especializadas, lo cual prueba la eficacia de la arquitectura propuesta incluso frente a enfoques específicos. Además, la arquitectura ha mostrado un buen desempeño en dispositivos móviles actuales de gama media, reportando una tasa de procesamiento promedio de entre 3 y 4 FPS en las pruebas realizadas en dispositivos Samsung A51 y Huawei P30 lite.

Con todo lo anterior, podemos concluir que el método propuesto proporciona tanto buenos resultados en cuanto a precisión en las diferentes tareas que realiza, competitivos con métodos alternativos del estado del arte, como en cuanto a su eficiencia, pudiendo ser integrado en aplicaciones móviles. De esta forma, supone una herramienta útil para el desarrollo de aplicaciones de asistencia a personas con dificultades visuales, dotándolas de una interfaz intuitiva y accesible con la que interactuar con este tipo de aplicaciones.

7.2. Trabajo futuro

Una de las cuestiones más destacables que quedarían como trabajo futuro es la integración de la arquitectura propuesta en una aplicación móvil para la asistencia a personas invidentes o con baja visión para, de este modo, dotarla de una interfaz accesible mediante gestos. Sin embargo, como paso previo, deberemos realizar una serie de pruebas de usuario en las que participen personas con discapacidad visual, para así evaluar la interfaz del sistema, su usabilidad y accesibilidad. En estas pruebas se analizaría la interfaz propuesta, el conjunto de gestos considerados y las acciones asociadas a estos. A partir de los resultados de dichas pruebas, y en función de los mismos, podría ser necesario iterar sobre el diseño de la interfaz y realizar nuevos ajustes en el método propuesto.

Otra cuestión abierta sería la identificación de los objetos a los que señalamos. Actualmente la cabeza de *captioning* puede ser utilizada para obtener la descripción de dichos objetos, ya que las descripciones de las imágenes suelen indicar el objeto señalado y, mediante post-procesamiento del texto generado, podemos identificar dicho objeto. Como trabajo futuro se pretende comparar esta aproximación con el resultado que podríamos obtener mediante el método de selección de filtros, el cual podría ser entrenado para las categorías de ILSVRC y determinar el objeto señalado a partir de su distancia con respecto a la punta del dedo.

Respecto al método de *captioning*, los resultados obtenidos son competitivos en comparación con los métodos del estado del arte. Sin embargo, aún están lejos de ser perfectos. Además, actualmente estas descripciones se generan en inglés, pero para una aplicación final sería importante contar con soporte multi-idioma. Por lo tanto, planteamos también

como trabajo futuro el desarrollo de un sistema multi-idioma y universal, que amplíe el abanico de personas que podrían utilizar la aplicación. En todos estos desarrollos futuros es importante mantener el objetivo que planteamos inicialmente de integrar las nuevas funcionalidades de forma eficiente en el sistema.



Universitat d'Alacant
Universidad de Alicante

- Ablavatski, A. and Grishchenko, I. (2019). Real-time ar self-expression with machine learning.
- Ahmetovic, D., Manduchi, R., Coughlan, J. M., and Mascetti, S. (2017). Mind your crossings: Mining gis imagery for crosswalk localization. *ACM Transactions on Accessible Computing (TACCESS)*, 9(4):11.
- Al-Azzawi, N. and Abdullah, W. (2011). Medical image fusion schemes using contourlet transform and pca bases. *Image fusion and its applications*, pages 93–110.
- Amaliya, S., Handayani, A. N., Akbar, M. I., Herwanto, H. W., Fukuda, O., and Kurniawan, W. C. (2021a). Study on hand keypoint framework for sign language recognition. In *2021 7th International Conference on Electrical, Electronics and Information Engineering (ICEEIE)*, pages 446–451.
- Amaliya, S., Handayani, A. N., Akbar, M. I., Herwanto, H. W., Fukuda, O., and Kurniawan, W. C. (2021b). Study on hand keypoint framework for sign language recognition. In *2021 7th International Conference on Electrical, Electronics and Information Engineering (ICEEIE)*, pages 446–451. IEEE.
- Balbin, J. R., Padilla, D. A., Caluyo, F. S., Fausto, J. C., Hortinela, C. C., Manlises, C. O., Bernardino, C. K. S., Fiñones, E. G., and Ventura, L. T. (2016). Sign language word translator using neural networks for the aurally impaired as a tool for communication. In *2016 6th IEEE International Conference on Control System, Computing and Engineering (ICCSCE)*, pages 425–429.

- Bamwenda, J. and Özerdem, M. (2019). Recognition of static hand gesture with using ann and svm. *Dicle University Journal of Engineering*.
- Barla, A., Odone, F., and Verri, A. (2003). Histogram intersection kernel for image classification. In *Proceedings 2003 international conference on image processing (Cat. No. 03CH37429)*, volume 3, pages III–513. IEEE.
- Bengio, Y., Lamblin, P., Popovici, D., and Larochelle, H. (2006). Greedy layer-wise training of deep networks. In *Proceedings of the 19th International Conference on Neural Information Processing Systems, NIPS'06*, page 153–160, Cambridge, MA, USA. MIT Press.
- Benitez-Garcia, G., Prudente-Tixteco, L., Castro-Madrid, L. C., Toscano-Medina, R., Olivares-Mercado, J., Sanchez-Perez, G., and Villalba, L. J. G. (2021). Improving real-time hand gesture recognition with semantic segmentation. *Sensors*, 21(2).
- Bheda, V. and Radpour, D. (2017). Using deep convolutional networks for gesture recognition in american sign language. *CoRR*, abs/1710.06836.
- Bottou, L. (2010). Large-scale machine learning with stochastic gradient descent. In *Proceedings of COMPSTAT'2010*, pages 177–186. Springer.
- Cao, Z., Hidalgo Martinez, G., Simon, T., Wei, S., and Sheikh, Y. A. (2019). Openpose: Realtime multi-person 2d pose estimation using part affinity fields. *IEEE Transactions on Pattern Analysis and Machine Intelligence*.
- Ch, R., Gadekallu, T. R., Abidi, M. H., and Al-Ahmari, A. (2020). Computational system to classify cyber crime offenses using machine learning. *Sustainability*, 12(10):4087.
- Chatfield, K., Simonyan, K., Vedaldi, A., and Zisserman, A. (2014a). Return of the Devil in the Details: Delving Deep into Convolutional Nets. In *British Machine Vision Conference*, pages 1–11.
- Chatfield, K., Simonyan, K., Vedaldi, A., and Zisserman, A. (2014b). Return of the devil in the details: Delving deep into convolutional nets. *CoRR*, abs/1405.3531.
- Chen, L.-C., Papandreou, G., Kokkinos, I., Murphy, K., and Yuille, A. L. (2017). Deeplab: Semantic image segmentation with deep convolutional nets, atrous convolution, and fully connected crfs. *IEEE transactions on pattern analysis and machine intelligence*, 40(4):834–848.
- Chen, Z., Cao, Y., Yao, H., Lu, X., Peng, X., Mei, H., and Liu, X. (2021). Emoji-powered sentiment and emotion detection from software developers' communication data. *ACM*

- Transactions on Software Engineering and Methodology (TOSEM)*, 30(2):1–48.
- Cheng, J., Zhou, W., Lei, X., Adamo, N., and Benes, B. (2020). The effects of body gestures and gender on viewer’s perception of animated pedagogical agent’s emotions. In Kurosu, M., editor, *Human-Computer Interaction. Multimodal and Natural Interaction*, pages 169–186, Cham. Springer International Publishing.
- Chollet, F. (2016). Xception: Deep learning with depthwise separable convolutions. *CoRR*, abs/1610.02357.
- Chung, J., Gulcehre, C., Cho, K., and Bengio, Y. (2014). Empirical evaluation of gated recurrent neural networks on sequence modeling.
- Cinbis, R. G., Verbeek, J., and Schmid, C. (2014). Multi-fold ml training for weakly supervised object localization. In *Computer Vision and Pattern Recognition (CVPR), 2014 IEEE Conference on*, pages 2409–2416. IEEE.
- Condit-Schultz, N., Ju, Y., and Fujinaga, I. (2018). A flexible approach to automated harmonic analysis: Multiple annotations of chorales by bach and prætorius. In Gómez, E., 0001, X. H., Humphrey, E., and Benetos, E., editors, *Proceedings of the 19th International Society for Music Information Retrieval Conference, ISMIR 2018, Paris, France, September 23-27, 2018*, pages 66–73.
- Daily, M., Medasani, S., Behringer, R., and Trivedi, M. (2017). Self-driving cars. *Computer*, 50(12):18–23.
- Dalal, N. and Triggs, B. (2005). Histograms of oriented gradients for human detection. In *2005 IEEE computer society conference on computer vision and pattern recognition (CVPR’05)*, volume 1, pages 886–893. Ieee.
- De Smedt, Q., Wannous, H., Vandeborre, J.-P., Guerry, J., Saux, B. L., and Filliat, D. (2017). 3d hand gesture recognition using a depth and skeletal dataset: Shrec’17 track. In *Proceedings of the Workshop on 3D Object Retrieval*, pages 33–38.
- Deng, J. (2012). Large scale visual recognition. phd thesis, princeton university.
- Deng, J., Dong, W., Socher, R., Li, L. J., Li, K., and Fei-Fei, L. (2009). Imagenet: A large-scale hierarchical image database. In *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pages 248–255.
- Desai, S. and Desai, A. (2017). Human computer interaction through hand gestures for home automation using microsoft kinect. In *Proceedings of International Conference on Communication and Networks*, pages 19–29. Springer.

- Devineau, G., Moutarde, F., Xi, W., and Yang, J. (2018). Deep learning for hand gesture recognition on skeletal data. In *2018 13th IEEE International Conference on Automatic Face & Gesture Recognition (FG 2018)*, pages 106–113. IEEE.
- Dinh, D.-L., Kim, J. T., and Kim, T.-S. (2014). Hand gesture recognition and interface via a depth imaging sensor for smart home appliances. *Energy Procedia*, 62:576–582.
- Elsken, T., Metzen, J. H., and Hutter, F. (2019). Neural architecture search: A survey. *The Journal of Machine Learning Research*, 20(1):1997–2017.
- Etten, A. V. (2018). You only look twice: Rapid multi-scale object detection in satellite imagery. *CoRR*, abs/1805.09512.
- Everingham, M., Eslami, S. A., Van Gool, L., Williams, C. K., Winn, J., and Zisserman, A. (2015). The pascal visual object classes challenge: A retrospective. *International journal of computer vision*, 111(1):98–136.
- Fan, H., Su, H., and Guibas, L. J. (2017). A point set generation network for 3d object reconstruction from a single image. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 605–613.
- Felzenszwalb, P., McAllester, D., and Ramanan, D. (2008). A discriminatively trained, multiscale, deformable part model. In *2008 IEEE conference on computer vision and pattern recognition*, pages 1–8. Ieee.
- Gallego, A.-J., Gil, P., Pertusa, A., and Fisher, R. B. (2018a). Segmentation of oil spills on side-looking airborne radar imagery with autoencoders. *Sensors*, 18(3).
- Gallego, A.-J., Pertusa, A., and Gil, P. (2018b). Automatic ship classification from optical aerial images with convolutional neural networks. *Remote Sensing*, 10(4):511.
- Ge, C., Wang, J., Wang, J., Qi, Q., Sun, H., and Liao, J. (2020). Towards automatic visual inspection: A weakly supervised learning method for industrial applicable object detection. *Computers in Industry*, 121:103232.
- Ge, L., Ren, Z., Li, Y., Xue, Z., Wang, Y., Cai, J., and Yuan, J. (2019). 3d hand shape and pose estimation from a single rgb image. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 10833–10842.
- Gershgorn, D. (2017). The data that transformed ai research—and possibly the world, quartz. Accessed: 2020-10-23.
- Girshick, R. (2015). Fast r-cnn.

- Girshick, R., Donahue, J., Darrell, T., and Malik, J. (2014). Rich feature hierarchies for accurate object detection and semantic segmentation. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 580–587.
- Girshick, R., Donahue, J., Darrell, T., and Malik, J. (2015). Region-based convolutional networks for accurate object detection and segmentation. *IEEE transactions on pattern analysis and machine intelligence*, 38(1):142–158.
- Glorot, X., Bordes, A., and Bengio, Y. (2011). Deep Sparse Rectifier Neural Networks. *Journal of Machine Learning Research (JMLR) W&CP*, 15:315–323.
- Golubova, E., Starke, S. D., Crossland, M. D., and Wolffsohn, J. S. (2021). Design considerations for the ideal low vision aid: insights from de-brief interviews following a real-world recording study. *Ophthalmic and Physiological Optics*, 41(2):266–280.
- Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press. <http://www.deeplearningbook.org>.
- Greff, K., Srivastava, R. K., Koutník, J., Steunebrink, B. R., and Schmidhuber, J. (2017). Lstm: A search space odyssey. *IEEE Transactions on Neural Networks and Learning Systems*, 28(10):2222–2232.
- Grossberg, S. and Levine, D. (1975). Some developmental and attentional biases in the contrast enhancement and short term memory of recurrent neural networks. *Journal of Theoretical Biology*, 53(2):341–380.
- Guida, G. and Mauri, G. (1986). Evaluation of natural language processing systems: Issues and approaches. *Proceedings of the IEEE*, 74(7):1026–1035.
- Hannun, A. Y., Rajpurkar, P., Haghpanahi, M., Tison, G. H., Bourn, C., Turakhia, M. P., and Ng, A. Y. (2019). Cardiologist-level arrhythmia detection and classification in ambulatory electrocardiograms using a deep neural network. *Nature medicine*, 25(1):65–69.
- Hasan, M. M. and Mishra, P. K. (2012). Hand gesture modeling and recognition using geometric features: a review. *Canadian journal on image processing and computer vision*, 3(1):12–26.
- Hazelwood, K., Bird, S., Brooks, D., Chintala, S., Diril, U., Dzhulgakov, D., Fawzy, M., Jia, B., Jia, Y., Kalro, A., et al. (2018). Applied machine learning at facebook: A datacenter infrastructure perspective. In *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pages 620–629. IEEE.

- He, K., Zhang, X., Ren, S., and Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778.
- He, K., Zhang, X., Ren, S., and Sun, J. (2016). Deep residual learning for image recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778.
- Hinton, G. E., Osindero, S., and Teh, Y.-W. (2006). A Fast Learning Algorithm for Deep Belief Nets. *Neural Computation*, 18(7):1527–1554.
- Hochreiter, S., Bengio, Y., Frasconi, P., Schmidhuber, J., et al. (2001). Gradient flow in recurrent nets: the difficulty of learning long-term dependencies.
- Hochreiter, S. and Schmidhuber, J. (1997). Long short-term memory. *Neural computation*, 9(8):1735–1780.
- Hodosh, M., Young, P., and Hockenmaier, J. (2013). Framing image description as a ranking task: Data, models and evaluation metrics. *Journal of Artificial Intelligence Research*, 47:853–899.
- Howard, A., Sandler, M., Chu, G., Chen, L.-C., Chen, B., Tan, M., Wang, W., Zhu, Y., Pang, R., Vasudevan, V., Le, Q. V., and Adam, H. (2019). Searching for mobilenetv3.
- Howard, A. G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M., and Adam, H. (2017). Mobilenets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861*.
- Hu, J., Shen, L., and Sun, G. (2018). Squeeze-and-excitation networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 7132–7141.
- Huang, G., Liu, Z., Van Der Maaten, L., and Weinberger, K. Q. (2017). Densely connected convolutional networks. In *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2261–2269.
- Huang, G., Wan, Z., Liu, X., Hui, J., Wang, Z., and Zhang, Z. (2019a). Ship detection based on squeeze excitation skip-connection path networks for optical remote sensing images. *Neurocomputing*, 332:215 – 223.
- Huang, L., Wang, W., and Wang, G. (2019b). Image captioning with two cascaded agents. In *ICASSP 2019-2019 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 4110–4114. IEEE.

- Huang, Y., Li, C., Li, T., Wan, W., and Chen, J. (2019c). Image captioning with attribute refinement. In *2019 IEEE International Conference on Image Processing (ICIP)*, pages 1820–1824. IEEE.
- Hwang, S. J. and Grauman, K. (2012). Reading between the lines: Object localization using implicit cues from image tags. *IEEE transactions on pattern analysis and machine intelligence*, 34(6):1145–1158.
- Iandola, F. N., Han, S., Moskewicz, M. W., Ashraf, K., Dally, W. J., and Keutzer, K. (2016). Squeezenet: Alexnet-level accuracy with 50x fewer parameters and 0.5 mb model size. *arXiv preprint arXiv:1602.07360*.
- Ioffe, S. and Szegedy, C. (2015a). Batch normalization: Accelerating deep network training by reducing internal covariate shift. *arXiv preprint arXiv:1502.03167*.
- Ioffe, S. and Szegedy, C. (2015b). Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. *JMLR W&CP*, 37.
- James, G., Witten, D., Hastie, T., and Tibshirani, R. (2013). *An introduction to statistical learning*, volume 112. Springer.
- Kaur, H. and Rani, J. (2016). A review: Study of various techniques of hand gesture recognition. In *2016 IEEE 1st International Conference on Power Electronics, Intelligent Control and Energy Systems (ICPEICES)*, pages 1–5. IEEE.
- Kim, H., Tang, Z., and Bansal, M. (2020). Dense-caption matching and frame-selection gating for temporal localization in videoqa. *arXiv preprint arXiv:2005.06409*.
- Kim, M.-S. and Lee, C. H. (2016). Hand gesture recognition for kinect v2 sensor in the near distance where depth data are not provided. *Int. J. Softw. Eng. Its Appl*, 10(12):407–418.
- Kleinbaum, D. G., Dietz, K., Gail, M., Klein, M., and Klein, M. (2002). *Logistic regression*. Springer.
- Kohavi, R. (1995). Proceedings of the 14th international joint conference on artificial intelligence. *Montreal, Morgan Kaufmann Publishers Inc*, 2:1137–1143.
- Köpüklü, O., Gunduz, A., Kose, N., and Rigoll, G. (2019). Real-time hand gesture detection and classification using convolutional neural networks. In *2019 14th IEEE International Conference on Automatic Face & Gesture Recognition (FG 2019)*, pages 1–8. IEEE.

- Koskenniemi, K. (1984). A general computational model for word-form recognition and production. In *Proceedings of the 4th Nordic Conference of Computational Linguistics (NODALIDA 1983)*, pages 145–154.
- Krizhevsky, A., Sutskever, I., and Hinton, G. E. (2012). ImageNet Classification with Deep Convolutional Neural Networks. *Advances In Neural Information Processing Systems*, pages 1–9.
- Lai, Z., Yao, Z., Wang, C., Liang, H., Chen, H., and Xia, W. (2016). Fingertips detection and hand gesture recognition based on discrete curve evolution with a kinect sensor. In *2016 Visual Communications and Image Processing (VCIP)*, pages 1–4. IEEE.
- Lamberti, L. and Camastra, F. (2011). Real-time hand gesture recognition using a color glove. In *International Conference on Image Analysis and Processing*, pages 365–373. Springer.
- LeCun, Y., Bengio, Y., and Hinton, G. (2015). Deep learning. *Nature*, 521(7553):436–444.
- Lee, S. and Kim, I. (2018). Multimodal feature learning for video captioning. *Mathematical Problems in Engineering*, 2018.
- Li, Q., Mou, L., Liu, Q., Wang, Y., and Zhu, X. X. (2018). Hsf-net: Multiscale deep feature embedding for ship detection in optical remote sensing imagery. *IEEE Transactions on Geoscience and Remote Sensing*, 56(12):7147–7161.
- Lienhart, R. and Maydt, J. (2002). An extended set of haar-like features for rapid object detection. In *Proceedings. international conference on image processing*, volume 1, pages I–I. IEEE.
- Likas, A., Vlassis, N., and Verbeek, J. J. (2003). The global k-means clustering algorithm. *Pattern recognition*, 36(2):451–461.
- Lin, H. I., Hsu, M. H., and Chen, W. K. (2014a). Human hand gesture recognition using a convolution neural network. In *2014 IEEE International Conference on Automation Science and Engineering (CASE)*, pages 1038–1043.
- Lin, M., Chen, Q., and Yan, S. (2014b). Network in network. In *International Conference on Learning Representations*, volume abs/1312.4400 of *ICLR*.
- Lin, T., Goyal, P., Girshick, R. B., He, K., and Dollár, P. (2017). Focal loss for dense object detection. *CoRR*, abs/1708.02002.

- Lin, T.-Y., Goyal, P., Girshick, R., He, K., and Dollár, P. (2018). Focal loss for dense object detection. *IEEE transactions on pattern analysis and machine intelligence*.
- Liu, W., Anguelov, D., Erhan, D., Szegedy, C., Reed, S., Fu, C.-Y., and Berg, A. C. (2016). Ssd: Single shot multibox detector. In Leibe, B., Matas, J., Sebe, N., and Welling, M., editors, *Computer Vision – ECCV 2016*, pages 21–37, Cham. Springer International Publishing.
- Lowe, D. G. (2004). Distinctive image features from scale-invariant keypoints. *International journal of computer vision*, 60(2):91–110.
- Lu, W., Tong, Z., and Chu, J. (2016). Dynamic hand gesture recognition with leap motion controller. *IEEE Signal Processing Letters*, 23(9):1188–1192.
- Lure, F. Y. M. and Rau, Y.-C. (1994). Detection of Ship Tracks in AVHRR Cloud Imagery with Neural Networks. In *Proceedings of IGARSS '94 - 1994 IEEE International Geoscience and Remote Sensing Symposium*, pages 1401–1403.
- Manduchi, R. and Coughlan, J. (2012). (computer) vision without sight. *Communications of the ACM*, 55(1):96–104.
- Mazumdar, D., Talukdar, A. K., and Sarma, K. K. (2013). Gloved and free hand tracking based hand gesture recognition. In *2013 1st International Conference on Emerging Trends and Applications in Computer Science*, pages 197–202.
- McCulloch, W. S. and Pitts, W. (1943). A logical calculus of the ideas immanent in nervous activity. *The bulletin of mathematical biophysics*, 5(4):115–133.
- Meijer, P. B. (2017). The voice for android.
- Miettinen, K. (1999). *Nonlinear multiobjective optimization*. Kluwer Academic Publishers, Boston.
- Mikolov, T., Yih, W.-t., and Zweig, G. (2013). Linguistic regularities in continuous space word representations. In *Proceedings of the 2013 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 746–751, Atlanta, Georgia. Association for Computational Linguistics.
- Mitchell, T. M. et al. (1997). *Machine learning*. McGraw-hill New York.
- Mohammed, A. A., Minhas, R., Wu, Q. J., and Sid-Ahmed, M. A. (2011). Human face recognition based on multidimensional pca and extreme learning machine. *Pattern Recognition*, 44(10-11):2588–2597.

- Moin, A., Zhou, A., Rahimi, A., Menon, A., Benatti, S., Alexandrov, G., Tamakloe, S., Ting, J., Yamamoto, N., Khan, Y., et al. (2021). A wearable biosensing system with in-sensor adaptive machine learning for hand gesture recognition. *Nature Electronics*, 4(1):54–63.
- Molchanov, P., Gupta, S., Kim, K., and Kautz, J. (2015). Hand gesture recognition with 3d convolutional neural networks. In *2015 IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pages 1–7.
- Molina, J., Pajuelo, J. A., and Martínez, J. M. (2017). Real-time motion-based hand gestures recognition from time-of-flight video. *Journal of Signal Processing Systems*, 86(1):17–25.
- Mummadi, C. K., Leo, F. P. P., Verma, K. D., Kasireddy, S., Scholl, P. M., Kempfle, J., and Laerhoven, K. V. (2018). Real-time and embedded detection of hand gestures with an imu-based glove. In *Informatics*, volume 5, page 28. Multidisciplinary Digital Publishing Institute.
- Mustafa, M. K., Allen, T., and Appiah, K. (2019). A comparative review of dynamic neural networks and hidden markov model methods for mobile on-device speech recognition. *Neural Computing and Applications*, 31(2):891–899.
- Narayanan, U., Unnikrishnan, A., Paul, V., and Joseph, S. (2017). A survey on various supervised classification algorithms. In *2017 International Conference on Energy, Communication, Data Analytics and Soft Computing (ICECDS)*, pages 2118–2124.
- Neto, P., Simão, M., Mendes, N., and Safeea, M. (2019). Gesture-based human-robot interaction for human assistance in manufacturing. *The International Journal of Advanced Manufacturing Technology*, 101(1-4):119–135.
- Nguyen, D. H., Do, T. N., Na, I.-S., and Kim, S.-H. (2019). Hand segmentation and fingertip tracking from depth camera images using deep convolutional neural network and multi-task segnet. *arXiv preprint arXiv:1901.03465*.
- Oggier, T., Lustenberger, F., and Blanc, N. (2006). Miniature 3d tof camera for real-time imaging. In *International Tutorial and Research Workshop on Perception and Interactive Technologies for Speech-Based Systems*, pages 212–216. Springer.
- ONCE, L. (2022). Afiliados a la once 2021.
- Pansare, J. R., Gawande, S. H., and Ingle, M. (2012). Real-time static hand gesture recognition for american sign language (asl) in complex background.

- Papineni, K., Roukos, S., Ward, T., and Zhu, W.-J. (2002). Bleu: a method for automatic evaluation of machine translation. In *Proceedings of the 40th annual meeting on association for computational linguistics*, pages 311–318. Association for Computational Linguistics.
- Perimal, M., Basah, S., Safar, M., and Yazid, H. (2018). Hand-gesture recognition-algorithm based on finger counting. *Journal of Telecommunication, Electronic and Computer Engineering (JTEC)*, 10(1-13):19–24.
- Pigou, L., Dieleman, S., Kindermans, P.-J., and Schrauwen, B. (2015). Sign language recognition using convolutional neural networks. In Agapito, L., Bronstein, M. M., and Rother, C., editors, *Computer Vision - ECCV 2014 Workshops*, pages 572–578, Cham. Springer International Publishing.
- Pisharady, P. K. and Saerbeck, M. (2015). Recent methods and databases in vision-based hand gesture recognition: A review. *Computer Vision and Image Understanding*, 141:152–165. Pose & Gesture.
- Prakash, J. and Gautam, U. K. (2019). Hand gesture recognition. *International Journal of Recent Technology and Engineering (IJRTE)*, 7(6).
- Prakash, R. M., Deepa, T., Gunasundari, T., and Kasthuri, N. (2017). Gesture recognition and finger tip detection for human computer interaction. In *2017 International Conference on Innovations in Information, Embedded and Communication Systems (ICIIECS)*, pages 1–4. IEEE.
- Prechelt, L. (1998). Early stopping - but when?
- Pun, C.-M., Zhu, H.-M., and Feng, W. (2011). Real-time hand gesture recognition using motion tracking. *International Journal of Computational Intelligence Systems*, 4(2):277–286.
- Raheja, J., Minhas, M., Prashanth, D., Shah, T., and Chaudhary, A. (2015). Robust gesture recognition using kinect: A comparison between dtw and hmm. *Optik*, 126(11-12):1098–1104.
- Raj, B., Kalgaonkar, K., Harrison, C., and Dietz, P. (2012). Ultrasonic doppler sensing in hci. *IEEE Pervasive Computing*, 11(2):24–29.
- Rajendran, P. and Madheswaran, M. (2010). Hybrid medical image classification using association rule mining with decision tree algorithm. *arXiv preprint arXiv:1001.3503*.
- Rajesh, R. J., Nagarjunan, D., Arunachalam, R., and Aarthi, R. (2012). Distance trans-

- form based hand gestures recognition for powerpoint presentation navigation. *Advanced Computing*, 3(3):41.
- Ranzato, M. A., Boureau, Y.-L., and LeCun, Y. (2007). Sparse feature learning for deep belief networks. In *Proceedings of the 20th International Conference on Neural Information Processing Systems*, NIPS'07, page 1185–1192, Red Hook, NY, USA. Curran Associates Inc.
- Redmon, J. (2013–2016). Darknet: Open source neural networks in c. <http://pjreddie.com/darknet/>.
- Redmon, J., Divvala, S., Girshick, R., and Farhadi, A. (2016). You only look once: Unified, real-time object detection. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 779–788.
- Redmon, J., Divvala, S. K., Girshick, R. B., and Farhadi, A. (2016). You only look once: Unified, real-time object detection. In *2016 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2016, Las Vegas, NV, USA, June 27-30, 2016*, pages 779–788.
- Redmon, J. and Farhadi, A. (2017). YOLO9000: better, faster, stronger. In *2017 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2017, Honolulu, HI, USA, July 21-26, 2017*, pages 6517–6525.
- Redmon, J. and Farhadi, A. (2018). Yolov3: An incremental improvement. *CoRR*, abs/1804.02767.
- Ren, S., He, K., Girshick, R., and Sun, J. (2015). Faster R-CNN: Towards real-time object detection with region proposal networks. In *Advances in Neural Information Processing Systems (NIPS)*.
- Ren, Z., Meng, J., and Yuan, J. (2011). Depth camera based hand gesture recognition and its applications in human-computer-interaction. In *2011 8th International Conference on Information, Communications & Signal Processing*, pages 1–5. IEEE.
- Risvik, H. (2007). Principal component analysis (pca) & nipals algorithm.
- Rituerto, A., Fusco, G., and Coughlan, J. M. (2016). Towards a sign-based indoor navigation system for people with visual impairments. In *Proceedings of the 18th International ACM SIGACCESS Conference on Computers and Accessibility*, pages 287–288. ACM.
- Rosenblatt, F. (1958). The perceptron: a probabilistic model for information storage

- and organization in the brain. *Psychological review*, 65 6:386–408.
- Rozantsev, A., Lepetit, V., and Fua, P. (2015). On rendering synthetic images for training an object detector. *Computer Vision and Image Understanding*, 137:24–37.
- Rumelhart, D., Hinton, G., and Williams, R. (1985). *Learning Internal Representations by Error Propagation*. ICS report. Institute for Cognitive Science, University of California, San Diego.
- Russakovsky, O., Deng, J., Su, H., Krause, J., Satheesh, S., Ma, S., Huang, Z., Karpathy, A., Khosla, A., Bernstein, M., Berg, A. C., and Fei-Fei, L. (2015). ImageNet Large Scale Visual Recognition Challenge. *International Journal of Computer Vision (IJCV)*, 115(3):211–252.
- Sáez, J. M., Escolano, F., and Lozano, M. A. (2015). Aerial obstacle detection with 3-d mobile devices. *IEEE journal of biomedical and health informatics*, 19(1):74–80.
- Sáez, J. M., Lozano, M. A., Escolano, F., and Lozano, J. P. (2020). An efficient, dense and long-range marker system for the guidance of the visually impaired. *Mach. Vis. Appl.*, 31(7):57.
- Samuel, A. L. (1959). Some studies in machine learning using the game of checkers. *IBM Journal of research and development*, 3(3):210–229.
- Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., and Chen, L.-C. (2018). Mobilenetv2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 4510–4520.
- Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., and Batra, D. (2016). Grad-cam: Visual explanations from deep networks via gradient-based localization. See <https://arxiv.org/abs/1610.02391> v3, 7(8).
- Seo, H., Back, S., Lee, S., Park, D., Kim, T., and Lee, K. (2020). Intra- and inter-epoch temporal context network (iitnet) using sub-epoch features for automatic sleep scoring on raw single-channel eeg. *Biomedical Signal Processing and Control*, 61:102037.
- Shao, F., Chen, L., Shao, J., Ji, W., Xiao, S., Ye, L., Zhuang, Y., and Xiao, J. (2021). Deep learning for weakly-supervised object detection and object localization: A survey. *arXiv preprint arXiv:2105.12694*.
- Shirke, Y. S., Doshi, P., Hegde, T., and Dhanvij, P. (2018). Text reader for visually impaired using google cloud vision api.

- Shuster, K., Humeau, S., Hu, H., Bordes, A., and Weston, J. (2019). Engaging image captioning via personality. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 12516–12526.
- Simon, T., Joo, H., Matthews, I., and Sheikh, Y. (2017). Hand keypoint detection in single images using multiview bootstrapping. In *CVPR*.
- Simonyan, K., Vedaldi, A., and Zisserman, A. (2013). Deep inside convolutional networks: Visualising image classification models and saliency maps. *arXiv preprint arXiv:1312.6034*.
- Simonyan, K. and Zisserman, A. (2014). Very Deep Convolutional Networks for Large-Scale Image Recognition. *ImageNet Challenge*, pages 1–10.
- Singh, K. K. and Lee, Y. J. (2017). Hide-and-seek: Forcing a network to be meticulous for weakly-supervised object and action localization.
- Son, G., Kwon, S., and Park, N. (2019). Gender classification based on the non-lexical cues of emergency calls with recurrent neural networks (rnn). *Symmetry*, 11(4):525.
- Sonkusare, J. S., Chopade, N. B., Sor, R., and Tade, S. L. (2015). A review on hand gesture recognition system. In *2015 International Conference on Computing Communication Control and Automation*, pages 790–794. IEEE.
- Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., and Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56):1929–1958.
- Supervision (2016). Supervision for Google Cardboard. Advanced magnifier for the visually impaired based on Google Cardboard. <http://supervisioncardboard.com>. Accessed: 6/4/2021.
- Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J., and Wojna, Z. (2015). Rethinking the inception architecture for computer vision. *CoRR*, abs/1512.00567.
- Tan, M. and Le, Q. (2019). EfficientNet: Rethinking model scaling for convolutional neural networks. In Chaudhuri, K. and Salakhutdinov, R., editors, *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 6105–6114. PMLR.
- Tang, D., Qin, B., and Liu, T. (2015). Document modeling with gated recurrent neural network for sentiment classification. In *Proceedings of the 2015 conference on empirical methods in natural language processing*, pages 1422–1432.

- Tang, H., Liu, H., Xiao, W., and Sebe, N. (2019). Fast and robust dynamic hand gesture recognition via key frames extraction and feature fusion.
- Tanti, M., Gatt, A., and Camilleri, K. P. (2018). Where to put the image in an image caption generator. *Natural Language Engineering*, 24(3):467–489.
- Tekin, B., Bogo, F., and Pollefeys, M. (2019). H+ o: Unified egocentric recognition of 3d hand-object poses and interactions. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 4511–4520.
- Vakunov, A. and Lagun, D. (2020). Mediapipe iris: Real-time iris tracking & depth estimation.
- Van den Bergh, M., Carton, D., De Nijs, R., Mitsou, N., Landsiedel, C., Kuehnlentz, K., Wollherr, D., Van Gool, L., and Buss, M. (2011). Real-time 3d hand gesture interaction with a robot for understanding directions from humans. In *2011 Ro-Man*, pages 357–362. IEEE.
- Van der Maaten, L. and Hinton, G. (2008). Visualizing data using t-sne. *Journal of machine learning research*, 9(11).
- Vathsala, M. and Holi, G. (2020). Rnn based machine translation and transliteration for twitter data. *International Journal of Speech Technology*, 23(3):499–504.
- Viola, P. and Jones, M. (2001). Rapid object detection using a boosted cascade of simple features. In *Proceedings of the 2001 IEEE computer society conference on computer vision and pattern recognition. CVPR 2001*, volume 1, pages I–I. Ieee.
- Waldrop, M. M. (2019). News feature: What are the limits of deep learning? *Proceedings of the National Academy of Sciences of the United States of America*, 116(4):1074–1077.
- Wang, L., Xiong, Y., Lin, D., and Gool, L. V. (2017). Untrimmednets for weakly supervised action recognition and detection.
- Wang, R. Y. and Popović, J. (2009). Real-time hand-tracking with a color glove. *ACM transactions on graphics (TOG)*, 28(3):1–8.
- Wang, Y., Wang, C., Zhang, H., Dong, Y., and Wei, S. (2019). Automatic ship detection based on retinanet using multi-resolution gaofen-3 imagery. *Remote Sensing*, 11(5).
- Weiss, J., Ruixuan Luo, and Welch, R. (1997). Automatic detection of ship tracks in satellite imagery. In *IGARSS'97. IEEE International Geoscience and Remote Sen-*

- sing Symposium Proceedings. Remote Sensing - A Scientific Vision for Sustainable Development*, volume 1, pages 160–162. IEEE.
- Weiss, K., Khoshgoftaar, T. M., and Wang, D. (2016). A survey of transfer learning. *Journal of Big data*, 3(1):1–40.
- WHO (2020). Informe mundial sobre la visión.
- Wikipedia Commons (2015). Typical CNN architecture. https://commons.wikimedia.org/wiki/File:Typical_cnn.png. Accessed: 19/07/2021.
- Williams, T. (2009). *Thermal imaging cameras: characteristics and performance*. CRC press.
- Xi, C., Chen, J., Zhao, C., Pei, Q., and Liu, L. (2018). Real-time hand tracking using kinect. In *Proceedings of the 2nd International Conference on Digital Signal Processing*, pages 37–42.
- Xiao, Z., Zhao, Y., Li, N., Zhou, S., Xu, H., et al. (2021). Research on key technologies of hand function rehabilitation training evaluation system based on leap motion. *Journal of Computer and Communications*, 9(01):19.
- Xie, C., Li, C., Zhang, B., Chen, C., and Han, J. (2017). Deep fisher discriminant learning for mobile hand gesture recognition.
- Xu, B., Wang, N., Chen, T., and Li, M. (2015). Empirical evaluation of rectified activations in convolutional network. *arXiv preprint arXiv:1505.00853*.
- Xu, M., Qian, F., Mei, Q., Huang, K., and Liu, X. (2018). Deeptype: On-device deep learning for input personalization service with minimal privacy concern. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*, 2(4):1–26.
- Xu, Y. and Goodacre, R. (2018). On splitting training and validation set: A comparative study of cross-validation, bootstrap and systematic sampling for estimating the generalization performance of supervised learning. *Journal of Analysis and Testing*, 2(3):249–262.
- Yan, X. and Su, X. (2009). *Linear regression analysis: theory and computing*. World Scientific.
- Yang, F., Xu, Q., and Li, B. (2017). Ship detection from optical satellite images based on saliency segmentation and structure-lbp feature. *IEEE Geoscience and Remote Sensing Letters*, 14(5):602–606.

- Yang, S.-P., Seo, Y.-H., Kim, J.-B., Kim, H. H., and Jeong, K.-H. (2019). Optical mems devices for compact 3d surface imaging cameras. *Micro and Nano Systems Letters*, 7.
- You, Q., Jin, H., Wang, Z., Fang, C., and Luo, J. (2016). Image captioning with semantic attention. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 4651–4659.
- Zarandi, M. F., Zarinbal, M., and Izadi, M. (2011). Systematic image processing for diagnosing brain tumors: A type-ii fuzzy expert system approach. *Applied soft computing*, 11(1):285–294.
- Zeiler, M. D. (2012a). ADADELTA: an adaptive learning rate method. *CoRR*, abs/1212.5701.
- Zeiler, M. D. (2012b). ADADELTA: an adaptive learning rate method. *CoRR*, abs/1212.5701.
- Zhang, F., Bazarevsky, V., Vakunov, A., Tkachenka, A., Sung, G., Chang, C.-L., and Grundmann, M. (2020a). Mediapipe hands: On-device real-time hand tracking. *arXiv preprint arXiv:2006.10214*.
- Zhang, F., Bazarevsky, V., Vakunov, A., Tkachenka, A., Sung, G., Chang, C.-L., and Grundmann, M. (2020b). Mediapipe hands: On-device real-time hand tracking. *arXiv preprint arXiv:2006.10214*.
- Zhang, J., Huang, K., Yu, Y., and Tan, T. (2011). Boosted local structured hog-lbp for object localization. In *Computer Vision and Pattern Recognition (CVPR), 2011 IEEE Conference on*, pages 1393–1400. IEEE.
- Zhang, X., Wei, Y., Kang, G., Yang, Y., and Huang, T. (2018). Self-produced guidance for weakly-supervised object localization.
- Zhao, Z.-Q., Zheng, P., Xu, S.-t., and Wu, X. (2019). Object detection with deep learning: A review. *IEEE transactions on neural networks and learning systems*, 30(11):3212–3232.
- Zhou, B., Khosla, A., Lapedriza, A., Oliva, A., and Torralba, A. (2016). Learning deep features for discriminative localization. In *Computer Vision and Pattern Recognition (CVPR), 2016 IEEE Conference on*, pages 2921–2929. IEEE.
- Zhou, C., Ding, C., Wang, X., Lu, Z., and Tao, D. (2020). One-pass multi-task networks with cross-task guided attention for brain tumor segmentation. *IEEE Transactions on Image Processing*, 29:4516–4529.

- Zhu, L. and Spachos, P. (2019). Towards image classification with machine learning methodologies for smartphones. *Machine Learning and Knowledge Extraction*, 1:1039–1057.
- Zoph, B., Vasudevan, V., Shlens, J., and Le, Q. (2017). Automl for large scale image classification and object detection. *Google AI Blog*, 2:2017.
- Zou, Z., Shi, Z., Guo, Y., and Ye, J. (2019). Object detection in 20 years: A survey. *arXiv preprint arXiv:1905.05055*.



Universitat d'Alacant
Universidad de Alicante

Contenidos

A	Librerías	137
B	Herramientas	139
C	Plataformas	141
D	Arquitecturas CNN	144

A. Librerías

Hoy en día existen numerosas librerías orientadas a resolver diferentes problemas relacionados con la inteligencia artificial, el aprendizaje automático y el deep learning. A continuación vamos a describir las librerías que se han utilizado en esta tesis doctoral:

- **MediaPipe¹**: Se trata de una librería de código abierto desarrollada por Google que nos ofrece diferentes soluciones de procesamiento de vídeo e imagen en tiempo real. En este trabajo se ha hecho uso de MediaPipe Hands (Zhang et al., 2020b), la cual permite reconocer el esqueleto de la mano a partir de una imagen 2D. Se trata de una solución de seguimiento de la mano y de los dedos, que obtiene 21 puntos de referencia en 3D o 2D a partir de un solo fotograma (ver Figura A.1). Esta librería es capaz de realizar el reconocimiento en tiempo real en dispositivos

¹<https://google.github.io/mediapipe/>

móviles, e incluso puede hacer seguimiento de varias manos. En el Capítulo 6 se compara esta forma de reconocimiento con el método propuesto.

Además, MediaPipe proporciona otras soluciones, como por ejemplo MediaPipe Iris (Vakunov and Lagun, 2020), para un reconocimiento preciso del iris, o MediaPipe Face Mesh (Ablavatski and Grishchenko, 2019), para la obtención de puntos de referencia 3D de caras en tiempo real.

Actualmente, MediaPipe se puede portar a Linux, MacOS, Android e iOS. El núcleo de MediaPipe es una librería C++ que cumple con el estándar C++ 11, por lo que es relativamente fácil de migrar a otras plataformas.



Figura A.1.: Resultado obtenido mediante la librería MediaPipe, que emplea el aprendizaje automático para inferir 21 puntos de referencia 3D o 2D de una mano a partir de un solo fotograma.

- **Core ML²:** Se trata de un *framework* que Apple proporciona para crear aplicaciones de aprendizaje automático y que resulta relativamente fácil de integrar para desarrolladores iOS, permitiéndoles utilizar modelos de terceros en sus aplicaciones. La API de Core ML permite entrenar y ajustar los modelos de ML y hacer predicciones desde el dispositivo de un usuario.

Core ML es relativamente amplia y se puede utilizar para implementar multitud de funcionalidades. Hay dos aplicaciones directas de Core ML: la primera consiste en que los desarrolladores pueden hacer uso de los modelos previamente entrenados proporcionados por la propia Core ML. La segunda permite construir tu propio modelo de ML personalizado usando plataformas como Caffe, Turi o Keras. A partir de estos modelos es necesario realizar una conversión al formato de Core ML mediante la herramienta de Python llamada `coremltools`, la cual transformaría

²<https://developer.apple.com/documentation/coreml>

el modelo y nos permitiría usarlo de forma directa en una aplicación iOS.



Figura A.2.: La plataforma Core ML permite integrar métodos de visión por computador, procesamiento del lenguaje natural, procesamiento de la voz y análisis del sonido, entre otros. Actúa de forma conjunta con otros *frameworks* de Apple, como *Accelerate* y *Metal Performance Shaders (MPS)*, que se utilizan para obtener aplicaciones de alto rendimiento.

- **Open Source Computer Vision Library (OpenCV)**³: Se trata de una librería de visión por computador de código abierto que ha sido utilizada durante nuestra investigación para diversas tareas, como implementar el preprocesamiento de las imágenes de entrada, normalizar los gradientes obtenidos por las redes neuronales, o aplicar operaciones morfológicas para obtener la localización precisa de objetos (ver Capítulos 5 y 6).

Está escrita en C/C++, cuenta con interfaces en C++, C, Python, MATLAB y Java, y es compatible con Windows, Linux, MacOS, iOS y Android. Esta librería contiene más de 2.500 algoritmos que cubren múltiples áreas de la visión por computador, pudiendo ser utilizada en áreas como el análisis de imágenes médicas, la visión estéreo y la robótica.

Los métodos de OpenCV se encuentran optimizados para conseguir una alta eficiencia y gran parte de ellos pueden utilizarse en aplicaciones en tiempo real.

B. Herramientas

Aparte de las librerías anteriores, también se cuenta con numerosas herramientas para facilitar la creación de modelos de ML. A continuación vemos las herramientas que hemos utilizado durante nuestro trabajo:

³<https://opencv.org/>

- **LibHand**⁴: Se trata de una librería de código abierto para generar imágenes sintéticas de manos a partir de un modelo 3D. En este trabajo se ha utilizado para generar un conjunto de datos sintético con gestos de la mano con el que realizar un entrenamiento inicial de los modelos propuestos.

Esta librería proporciona una interfaz de programación simple para generar una pose de un modelo 3D de mano, representarla gráficamente, y analizar la imagen resultante (Figura B.1). Se ha desarrollado una versión modificada de la librería⁵ para que genere de forma aleatoria un *dataset* de gestos a partir de los rangos dentro de los cuales puede moverse cada parámetro (ángulos de las articulaciones de la mano). La gran ventaja que ofrece esta librería es que nos proporciona una forma sencilla y rápida de generar imágenes con un modelo 3D de mano realista, etiquetando las imágenes de forma automática, lo cual puede ser de utilidad para otros investigadores en reconocimiento de gestos. Por este motivo, la modificación desarrollada se ha dejado disponible de forma pública para la comunidad.



Figura B.1.: LibHand es una herramienta que permite generar modelos 3D de manos realistas.

- **LabelImg**⁶: Se trata de una herramienta para etiquetar imágenes mediante cuadros delimitadores con la posición y clase de los objetos (ver Figura B.2). En nuestro trabajo se ha usado para anotar todas las imágenes de nuestros conjuntos de datos, con las posiciones de las manos, puntas de los dedos y objetos apunta-

⁴<https://github.com/libhand/libhand>

⁵<https://github.com/malozano/libhand>

⁶<https://github.com/tzutalin/labelImg>

dos, con el objetivo de utilizar este etiquetado para entrenar y evaluar los modelos desarrollados.

LabellImg está escrita en Python y usa el *framework* Qt para su interfaz gráfica. Las anotaciones se guardan como archivos XML en formato PASCAL VOC, que es el formato también utilizado por ImageNet (Russakovsky et al., 2015). Además, es compatible con el formato YOLO (Redmon et al., 2016) y está disponible para los sistemas operativos Linux, Mac y Windows.

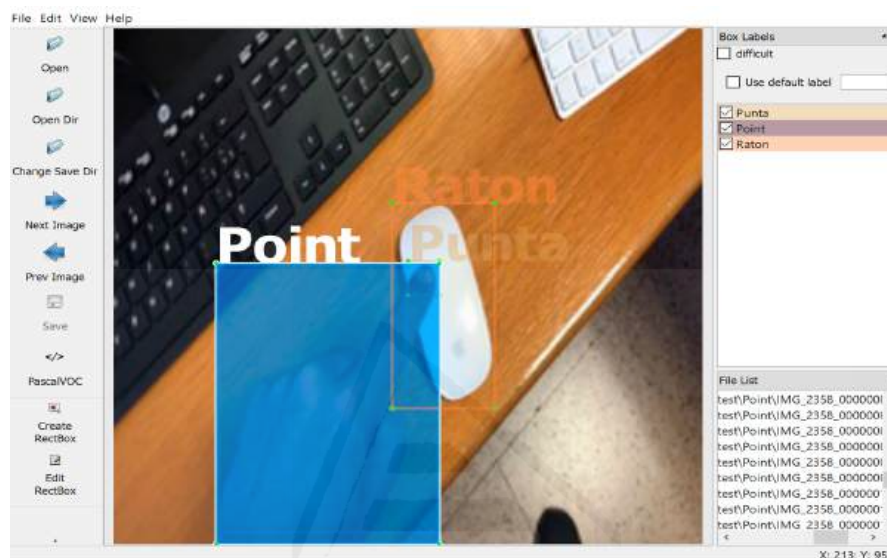


Figura B.2.: Herramienta LabellImg para la anotación de imágenes mediante cuadros delimitadores.

C. Plataformas

Las plataformas de aprendizaje automático nos brindan a los desarrolladores herramientas para implementar y analizar métodos pertenecientes a esta disciplina. Estas plataformas combinan algoritmos inteligentes de toma de decisiones con datos que podemos utilizar para entrenar los diferentes modelos, lo cual simplifica a los desarrolladores la creación de soluciones en el ámbito del ML. Además, permiten a los desarrolladores sin conocimientos avanzados en ML beneficiarse de las posibilidades de estos métodos, ya que facilitan la construcción de modelos basados en redes neuronales.

De las diferentes plataformas que existen, son tres las más comúnmente empleadas:

- **TensorFlow (TF)**⁷: Es una librería de código abierto desarrollada por Google Brain para crear redes neuronales y modelos de aprendizaje profundo. Dispone de una extensa documentación y una gran cantidad de funcionalidades, las cuales además se pueden modificar y personalizar fácilmente. La curva de aprendizaje de esta herramienta es algo más elevada que la de otras alternativas, dado que está escrita como una librería de bajo nivel. Sin embargo, cuenta con la ventaja de estar implementada en Python y C++, por lo que nos permite desarrollar y entrenar los modelos en estos lenguajes y, posteriormente, exportar de forma sencilla y eficiente a diferentes plataformas para su puesta en producción.

Una vez desarrollado y entrenado el modelo, se puede congelar (todas sus variables se convertirán en constantes) y guardar en un formato que contenga tanto su estructura como sus pesos, para así poderlo cargar desde diferentes plataformas, como podría ser nuestro ordenador de trabajo local, un clúster en la nube, dispositivos iOS y Android, y soportando tanto CPU como GPU. Además, es posible usar la propia nube de Google y ejecutar TF en las Unidad de Procesamiento de TensorFlow (TPU) de Google para una mayor aceleración.

También dispone de un conjunto de herramientas reducido (TF Lite) para ayudar a los desarrolladores a ejecutar modelos de TF en dispositivos embebidos, móviles o de *Internet of Things* (IoT), buscando reducir el tamaño del binario y la carga computacional. La Figura C.1 muestra el diagrama de flujo del funcionamiento de TF Lite. Además cuenta con una herramienta de visualización llamada TensorBoard que permite a los usuarios monitorizar sus modelos de ML, los parámetros y las diferentes métricas.

⁷https://www.tensorflow.org/lite/api_docs

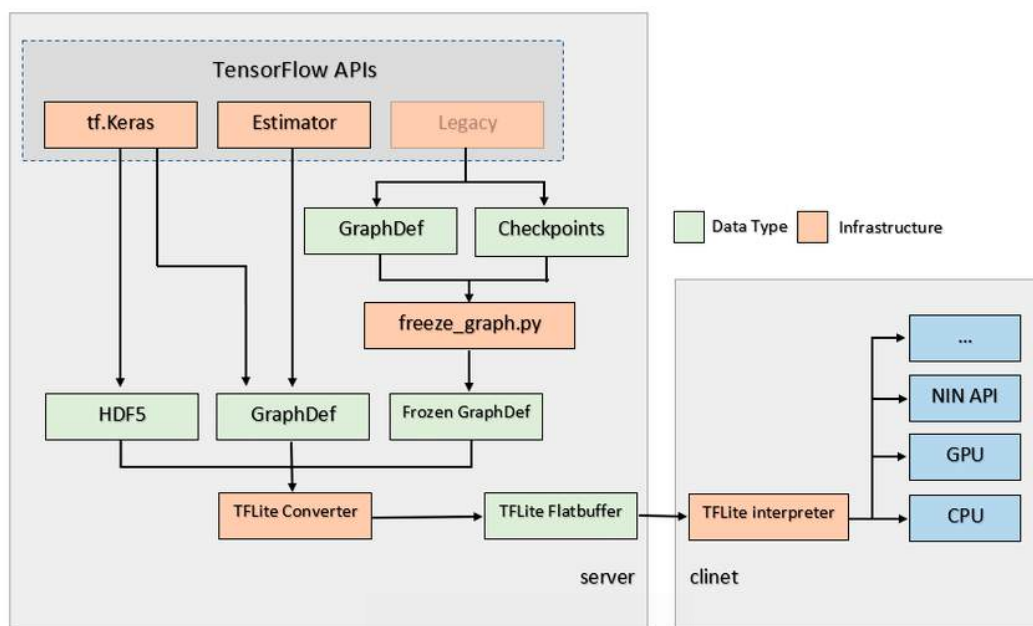


Figura C.1.: Esquema del proceso de conversión de TensorFlow Lite (Zhu and Spachos, 2019).

- **Keras**⁸: Se trata de una librería de alto nivel escrita en Python y construida sobre TF, que proporciona una estructura simplificada para la creación de modelos de aprendizaje profundo. Se sacrifica parte de la flexibilidad de TF a cambio de una mayor simplicidad de desarrollo. Al igual que en el caso de TF, los modelos son fácilmente exportables a diferentes plataformas con una alta escalabilidad. Cuenta con los principales tipos de unidades que podemos necesitar para la construcción de modelos de aprendizaje profundo, permitiéndonos crear estos modelos combinando de forma sencilla las diferentes unidades. El modelo generado también puede ser ejecutado tanto en CPU como en GPU. Todo esto hace de Keras una opción óptima para el desarrollo de aplicaciones de aprendizaje profundo.
- **Torch**⁹: Es un *framework* de código abierto que cuenta con un amplio soporte para algoritmos de aprendizaje automático y que destaca por el uso optimizado de la GPU. Proporciona una elevada flexibilidad para la construcción de algoritmos al mismo tiempo que hace que el proceso sea extremadamente simple. Se pueden construir los modelos de redes neuronales de forma sencilla y paralelizarlos sobre CPU o GPU de una manera eficiente y, además, se puede integrar en plataformas móviles iOS y Android. Aunque Torch ya no está en desarrollo activo, sí lo está PyTorch, una librería basada en Torch e implementada en Python.

⁸<https://keras.io/>

⁹<http://torch.ch/>

D. Arquitecturas CNN

A continuación se describen (ordenados alfabéticamente) algunos de los principales modelos de redes neuronales profundas que encontramos en el estado del arte para la clasificación de imagen y que hemos utilizado durante la elaboración de esta tesis:

- **Darknet-53** (Redmon and Farhadi, 2018): Es el *backbone* utilizado en la red YOLO v3 de detección de objetos. Está implementado en Darknet (Redmon, 2016), un *framework* para el desarrollo de redes neuronales de código abierto escrito en C y CUDA, que facilita la creación de modelos muy eficientes sobre CPU y GPU. Darknet-53 tiene 53 capas que combinan una red residual profunda (He et al., 2016) con la parte para la extracción de características usada en YOLO. Utiliza capas de convolución sucesivas con conexiones residuales, capas de *Batch Normalization* (BN) (Ioffe and Szegedy, 2015a) y activaciones tipo LeakyRelu (Xu et al., 2015).
- **DenseNet** (Huang et al., 2017): Es otra arquitectura del estado del arte para el reconocimiento de objetos. Es bastante similar a ResNet (He et al., 2016) (descrita más adelante), de hecho, los dos métodos intentan resolver el problema del *vanishing gradient* (Hochreiter et al., 2001) mediante el uso de atajos o conexiones desde las primeras capas hasta las últimas. Sin embargo, DenseNet lo hace concatenando los mapas de características en lugar de sumándolos, lo cual ha demostrado obtener mejores resultados al reutilizar los mapas de características. Esto permite usar un menor número de filtros y, por lo tanto, hace que la red necesite menos parámetros. Gracias a esto DenseNet logra una precisión similar a ResNet, pero utilizando menos de la mitad de parámetros.
- **EfficientNet** (Tan and Le, 2019): Como su nombre indica, son redes muy eficientes desde el punto de vista computacional y que además obtienen una precisión del 84,3 % (top-1) en ImageNet (Deng et al., 2009), superando a otros métodos del estado del arte. La idea central detrás de esta propuesta es escalar estratégicamente las redes neuronales profundas existentes con un método llamado “*Compound Scaling*”. Este método sugiere que en lugar de escalar solo un atributo del modelo (profundidad, ancho o resolución), escalar estratégicamente los tres juntos ofrece mejores resultados. La arquitectura EfficientNet-B0 ha sido desarrollada mediante “*Neural Architecture Search*” (Elsken et al., 2019) utilizando el *framework* “AutoML MNAS” (Zoph et al., 2017), que optimiza tanto la precisión como la eficiencia. Tomando EfficientNet-B0 como modelo de referencia, posteriormente se mejora mediante el método *Compound Scaling* para obtener la serie de modelos EfficientNet-B1 a B7. En concreto, EfficientNet-B0 es una arquitectura diseñada con un tamaño orientado a dispositivos móviles, con 11 millones de parámetros. Utiliza 7 bloques residuales invertidos (propuestos originalmente en MobileNet v2), pero en este caso cada uno tiene una configuración diferente. Estos bloques tienen

además un número variable de sub-bloques cuyo número aumenta a medida que pasamos de EfficientNet-B0 a EfficientNet-B7.

- **Inception v3** (Szegedy et al., 2015): Esta arquitectura tiene 6 capas convolucionales seguidas de 3 módulos Inception y una última capa completamente conectada. Tiene menos parámetros que otros modelos similares gracias a los módulos *Inception*, cuyo diseño se basa en dos ideas principales: la aproximación de una estructura dispersa con componentes densos repetidos espacialmente y el uso de la reducción de dimensionalidad para mantener la complejidad computacional dentro de los límites.
- **MobileNet v1** (Howard et al., 2017): Utiliza convoluciones separables en profundidad (*depthwise separable convolutions*) para construir redes neuronales profundas y ligeras. En total tiene 28 capas, combinando este tipo de convoluciones con *Batch Normalization* (Ioffe and Szegedy, 2015b) y funciones de activación ReLU, con la excepción de la capa final completamente conectada, que utiliza Softmax para la clasificación. Este modelo está orientado a la implementación de aplicaciones de visión móvil eficientes, lo que permite aplicar la detección, clasificación y segmentación de imagen en dispositivos móviles.
- **MobileNet v2** (Sandler et al., 2018): Este modelo es una mejora con respecto a la anterior versión, es un 35 % más rápido, siendo más eficiente para funcionar en dispositivos móviles. MobileNet v2 también utiliza en su arquitectura bloques convolucionales separables en profundidad, pero introduce dos novedades principales: *bottlenecks* lineales entre capas y atajos entre los *bottlenecks*. Estos atajos conectan el principio y el final de un bloque convolucional, y permiten entrenar más rápidamente y conseguir una mayor precisión.
- **MobileNet v3** (Howard et al., 2019): Para el desarrollo de esta tercera versión se usó AutoML (Zoph et al., 2017) para encontrar la mejor topología de red neuronal posible dentro del espacio de búsqueda de modelos orientados a computación móvil. En su arquitectura incorpora bloques de tipo *squeeze-and-excitation* (Hu et al., 2018), lo que permitió eliminar tres capas de alto coste computacional de la red predecesora sin sacrificar la precisión. En tareas de detección de objetos MobileNet v3 obtiene un 25 % menos de latencia manteniendo la misma precisión que las versiones anteriores. Es conveniente mencionar también que esta arquitectura se ha incorporado en *frameworks* populares como TensorFlow Lite.
- **ResNet** (*Residual Network*) (He et al., 2016): Esta arquitectura se basa en el uso de conexiones residuales denominadas *skip connections*, que se saltan varios bloques, lo cual permite entrenar redes con una gran cantidad de capas evitando el problema del gradiente evanescente a la vez que se mantiene el rendimiento. Las arquitecturas Resnet más comunes utilizan 50, 101 o 152 capas.

- **SqueezeNet** ([Iandola et al., 2016](#)): Esta pequeña CNN está orientada a integrarse en chips tipo *Field Programmable Gate Array* (FPGA) y en dispositivos móviles. Con 50 veces menos parámetros que la red AlexNet ([Krizhevsky et al., 2012](#)) y un tamaño de solo 0,5 Mb, logra alcanzar un nivel similar de precisión que otras arquitecturas similares.
- **VGG-16 y VGG-19** ([Simonyan and Zisserman, 2014](#)): VGG-16 tiene 13 capas convolucionales y 3 completamente conectadas, mientras que VGG-19 se compone de 16 capas convolucionales y 3 capas completamente conectadas. Ambas topologías utilizan técnicas *Dropout*, *Max-pooling* y funciones de activación ReLU.
- **Xception** ([Chollet, 2016](#)): Este modelo tiene 36 capas convolucionales que incorporan una versión rediseñada de los módulos *Inception* que permite la operación de convolución separable en profundidad. Esta arquitectura supera los resultados de *Inception* utilizando el mismo número de parámetros.



Universitat d'Alacant
Universidad de Alicante

LISTA DE ACRÓNIMOS

A

AP Average Precision.

ATAP Google's Advanced Technology and Projects Group.

B

BLEU Bi-Lingual Evaluation Understudy.

BN Batch Normalization.

BoF Bag of Features.

BP Retropropagación.

C

CAM Class Activation Mapping.

CV Cross Validation.

D

DL Deep Learning.

DPM Deformable Part-based Model.

F

FC Fully Connected.

FPGA Field Programmable Gate Array.

FS Filter Selection.

G

GAP Global Average Pooling.

Grad-CAM Gradient-weighted Class Activation Mapping.

H

HCNF Hidden Conditional Neural Field.

HOG Histogram of Oriented Gradients.

I

IA Inteligencia Artificial.

ILSVRC ImageNet Large Scale Visual Recognition Challenge.

IoT Internet of Things.

IoU Intersection over Union.

M

MASATI MAritime SATellite Imagery.

ML Machine Learning.

MOP Problema de Optimización Multiobjetivo.

O

ONCE Organización Nacional de Ciegos Españoles.

OpenCV Open Source Computer Vision Library.

P

PCA Principal Component Analysis.

R

RCNN Region Based Convolutional Neural Network.

ReLU Rectified Linear Unit.

RGB Red Green Blue.

RPN Region Proposal Network.

S

SAE Selectional Auto-Encoder.

SESPNets Squeeze Excitation Skip-connection Path Networks.

T

t-SNE t-Stochastic Neighbor Embedding.

TF TensorFlow.

ToF Time of Flight.

TPU Unidad de Procesamiento de TensorFlow.

V

VOC2012 Visual Object Classes Challenge 2012.

VRG Gafas de Realidad Virtual.

W

WHO Organización Mundial de la Salud.

Y

YOLT You Only Look Twice.



Universitat d'Alacant
Universidad de Alicante



Universitat d'Alacant
Universidad de Alicante

ED|UA

Escola de Doctorat
Escuela de Doctorado

edua.ua.es

